

BPS5231

Artificial Intelligence

For

Sustainable

Building

Design

L8

Ang Yu Qian

Assistant Professor

L08.1

Unsupervised

Clustering

KMeans

Hierarchical

Principal Component Analysis

t-SNE

L08.2

Supervised

Convolutional Neural Networks

Saliency Maps

Transfer Learning

L08.1

Unsupervised

Clustering

KMeans

Hierarchical

Principal Component Analysis

t-SNE

L08.2

Supervised

Convolutional Neural Networks

Saliency Maps

Transfer Learning

Today, we will cover

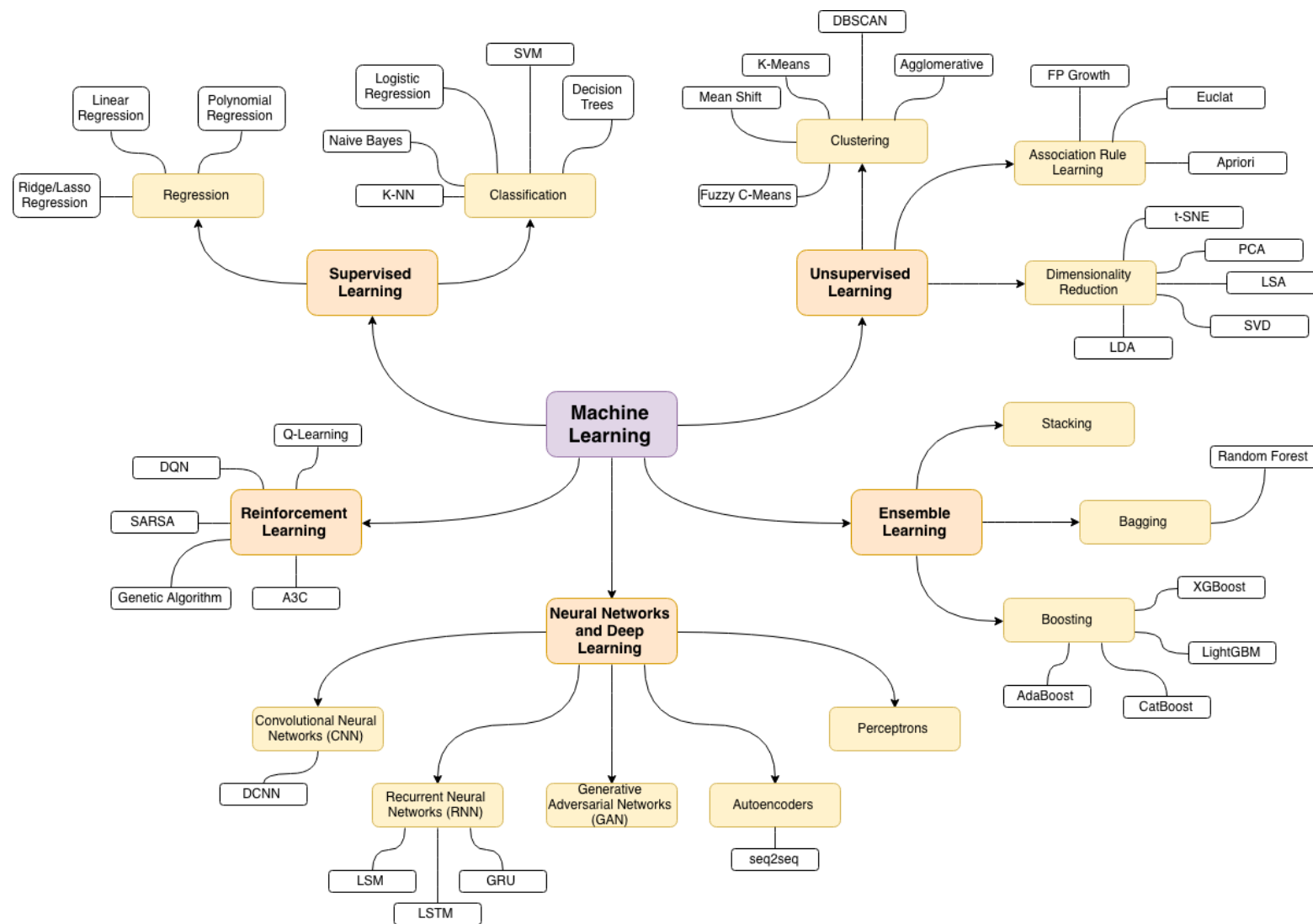


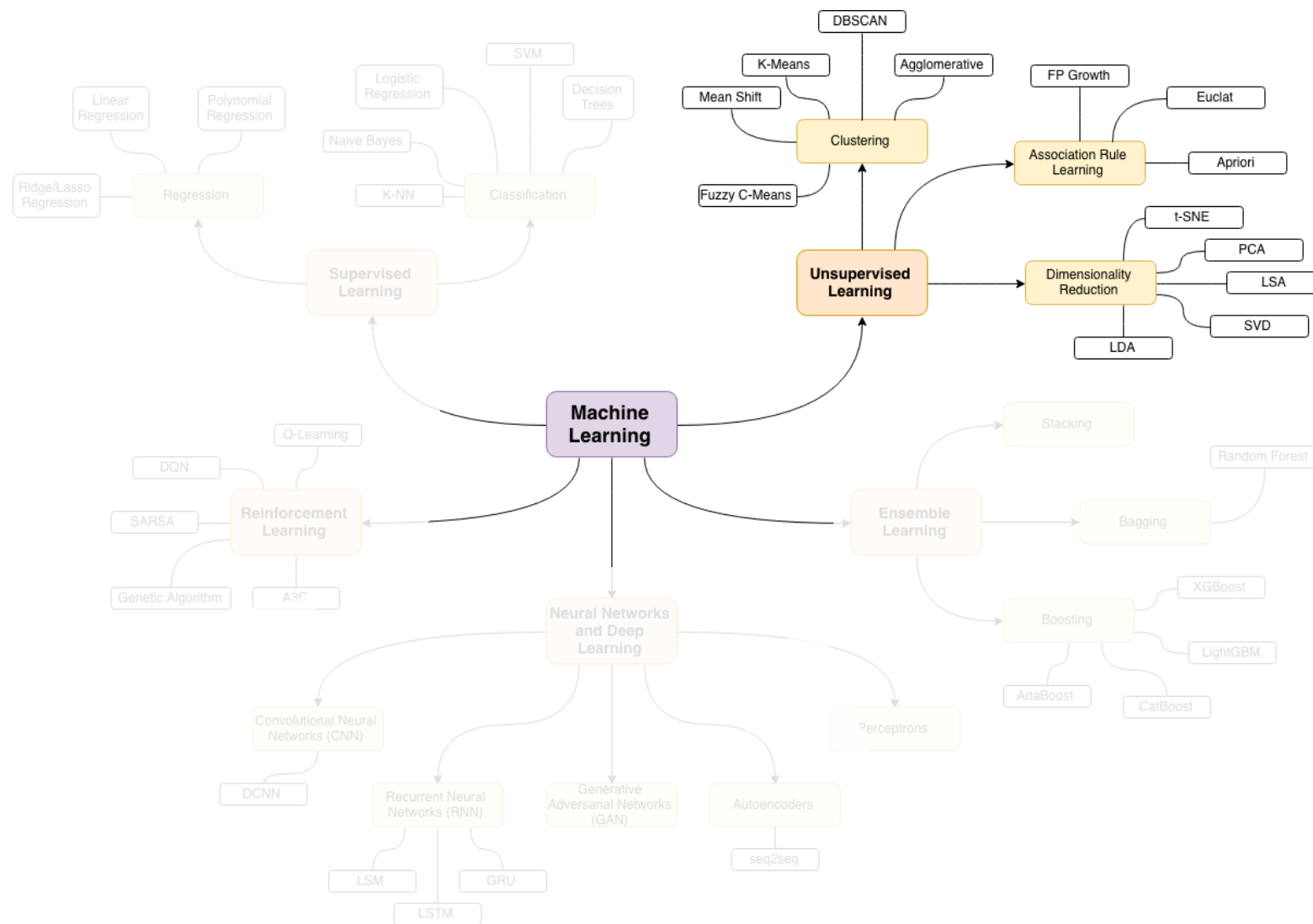
Image from Anil Jain, Google

UNSUPERVISED

SUPERVISED

SIMULATIONS

Today, we will cover



← Today

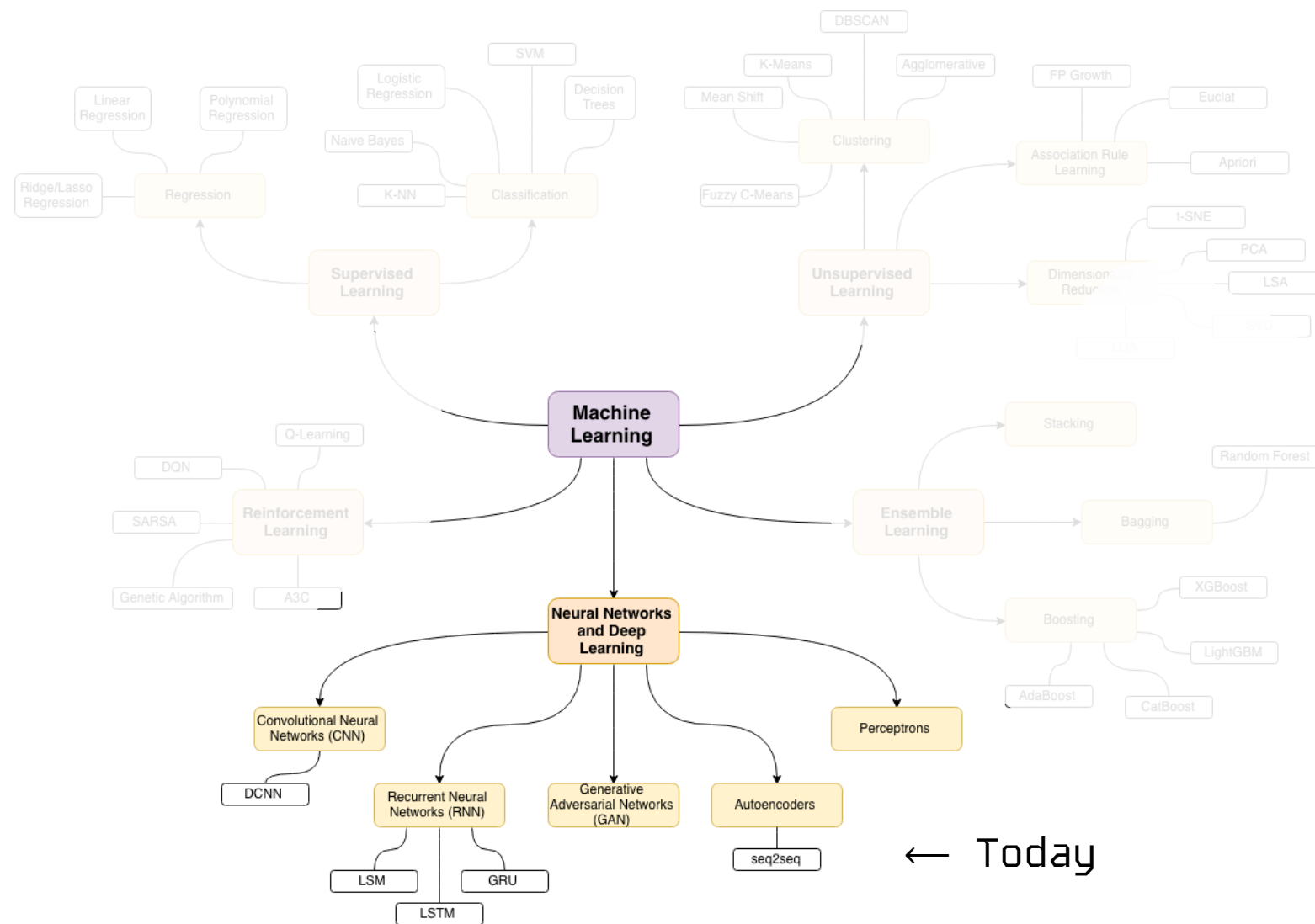
Image from Anil Jain, Google

UNSUPERVISED

SUPERVISED

SIMULATIONS

Today, we will cover



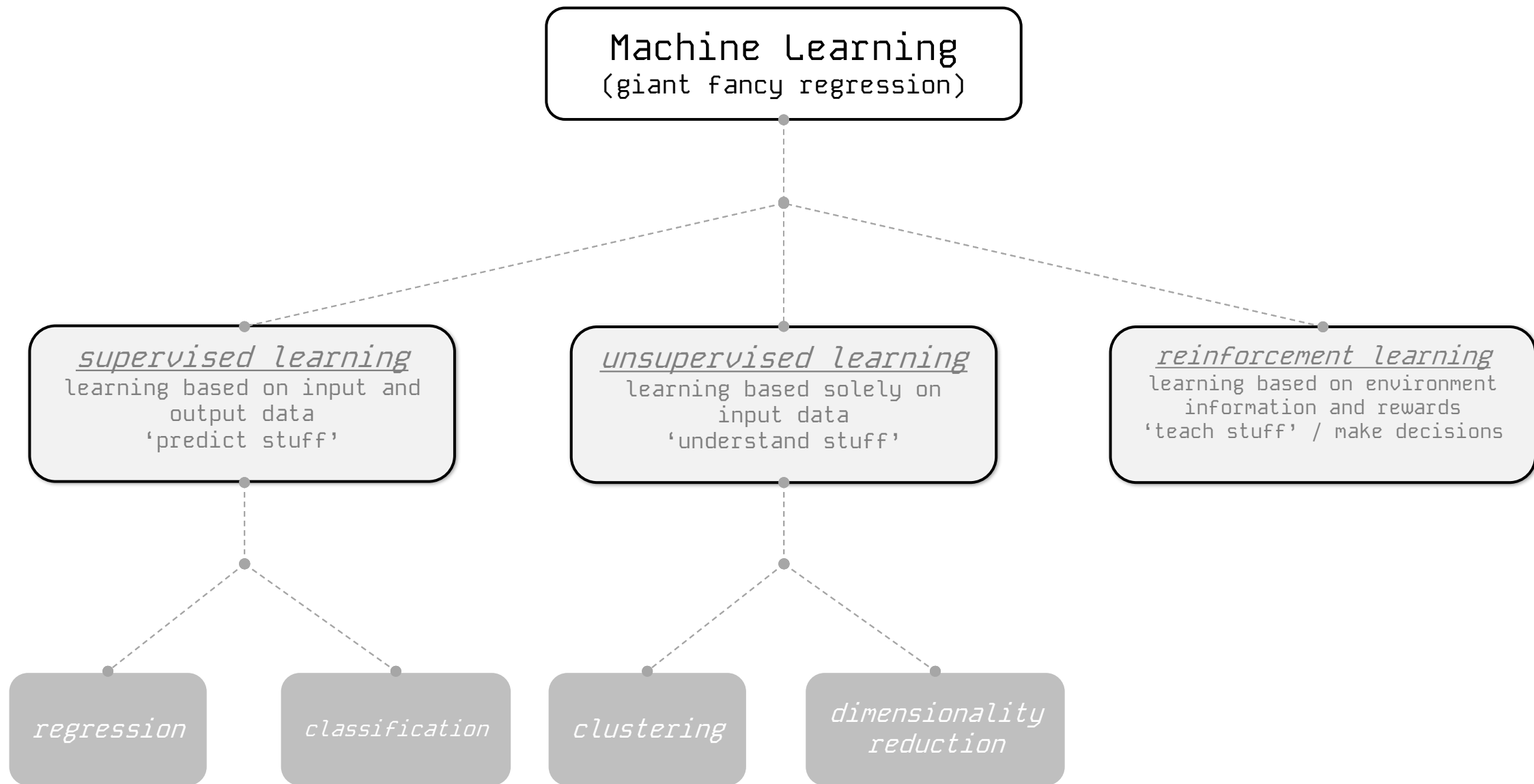
← Today

Image from Anil Jain, Google

UNSUPERVISED

SUPERVISED

SIMULATIONS



UNSUPERVISED

SUPERVISED

SIMULATIONS

Supervised Learning

$S_n = \{(\mathbf{x}^{(t)}, \mathbf{y}^{(t)}), t = 1, \dots, n\}$ = dataset with features and labels (ground truth examples)

Learn relationship between features and target variables

Unsupervised Learning

$S_n = \{\mathbf{x}^{(t)}, t = 1, \dots, n\}$ = dataset with features only

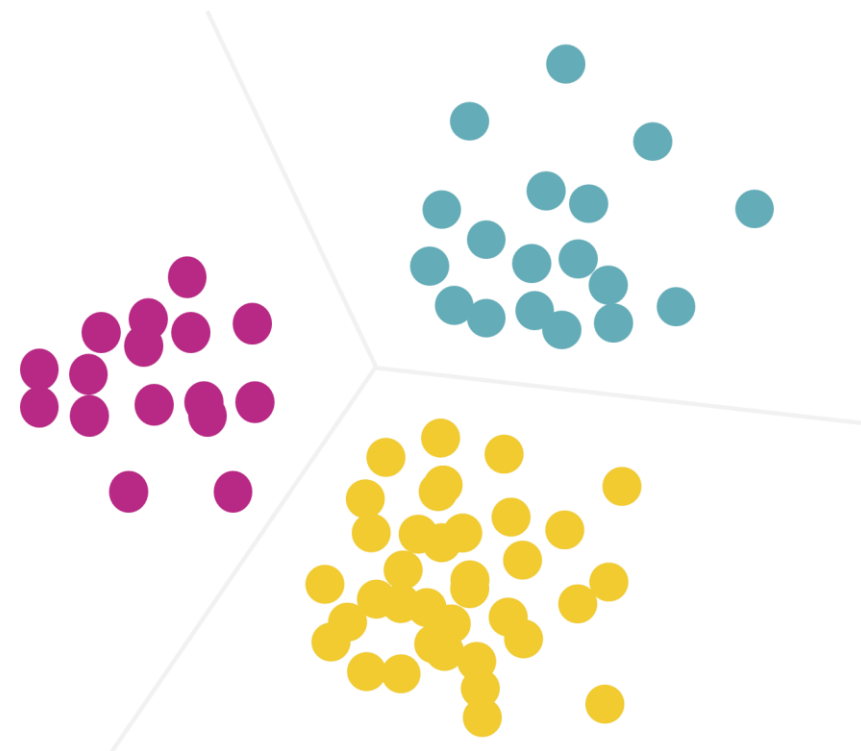
Uncover structure in the data

e.g.:

Identifying groups (cluster analysis)

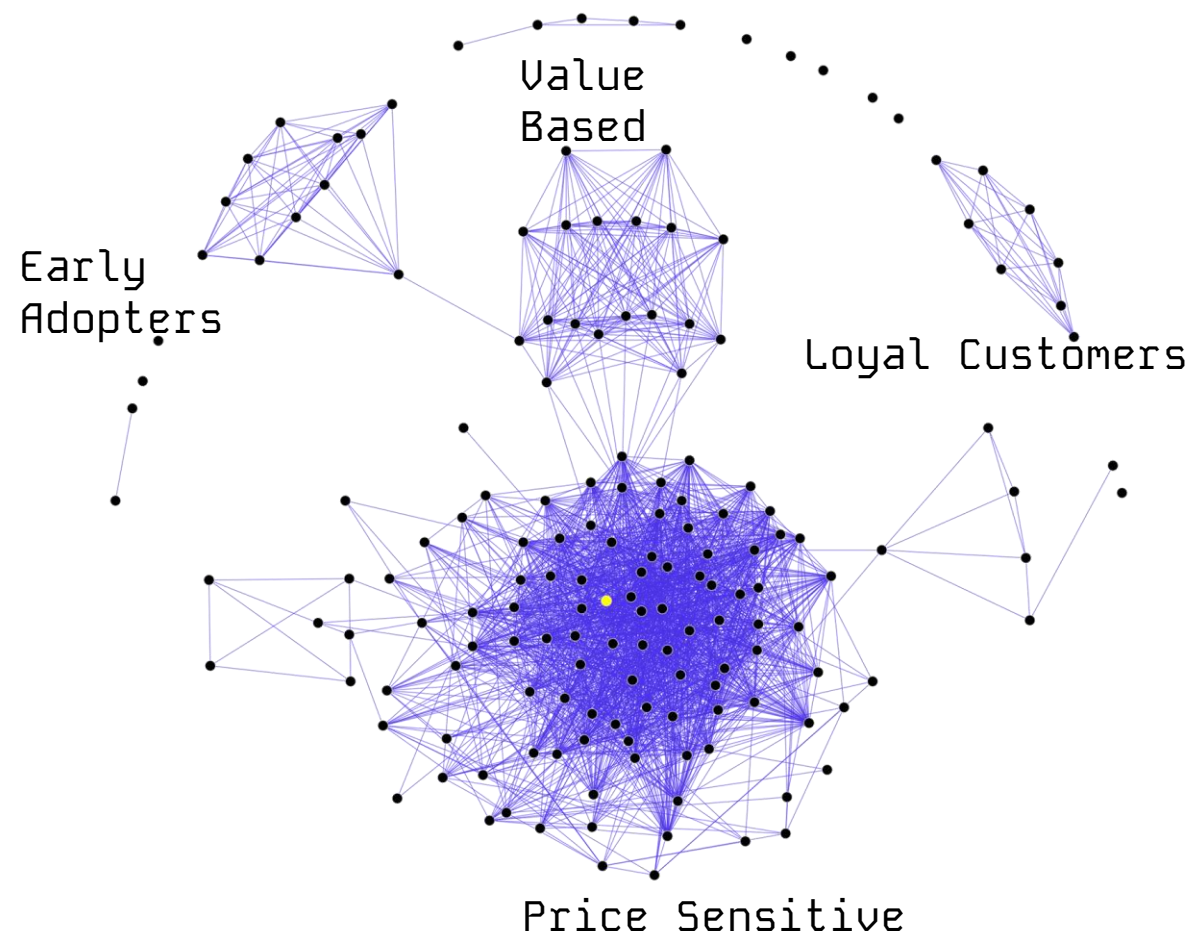
Dimensionality Reduction (visualization)

Latent Variable Models (VAEs, GANs)

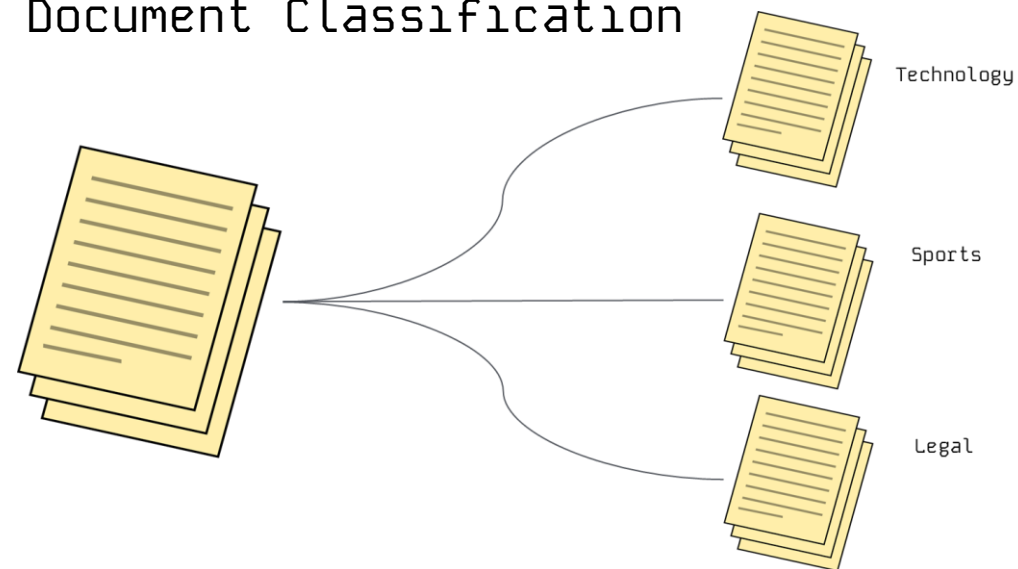


Clustering Use Cases

Customer Segmentation

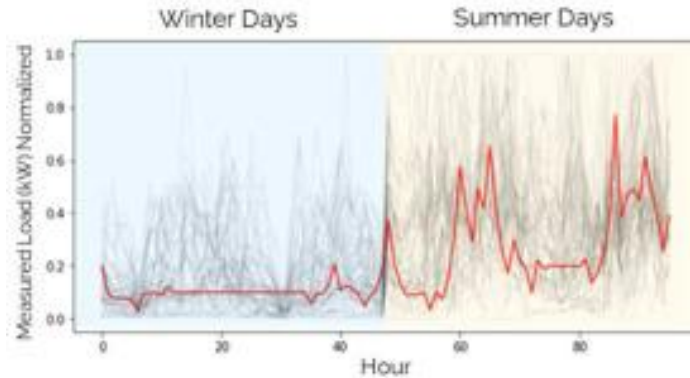


Document Classification

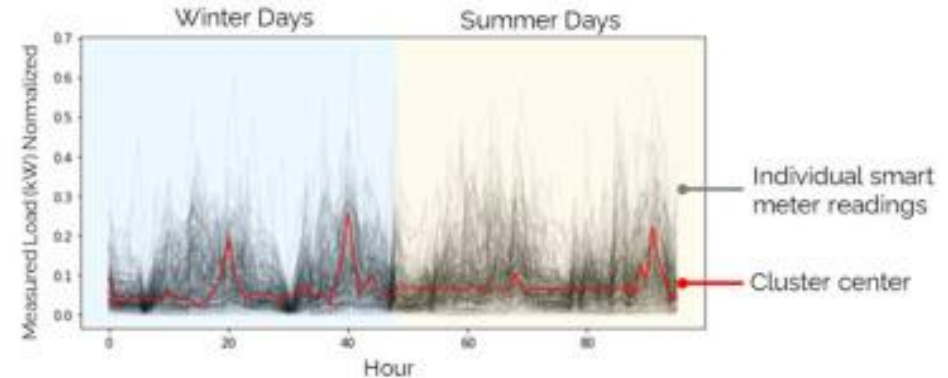


Clustering Use Cases

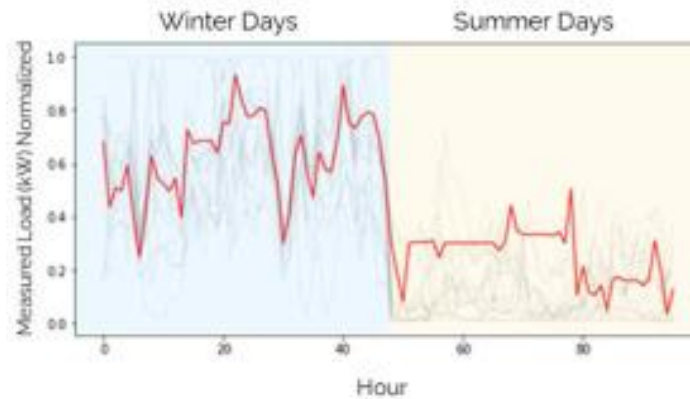
Cluster 1: Exurban, upper-middle-class



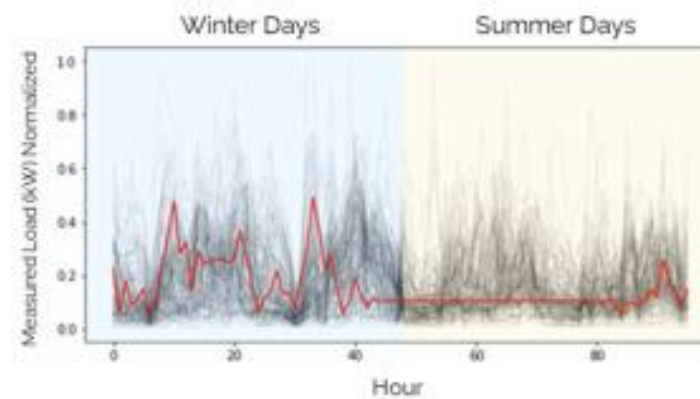
Cluster 2: Working 9-to-5



Cluster 3: young, lower-income

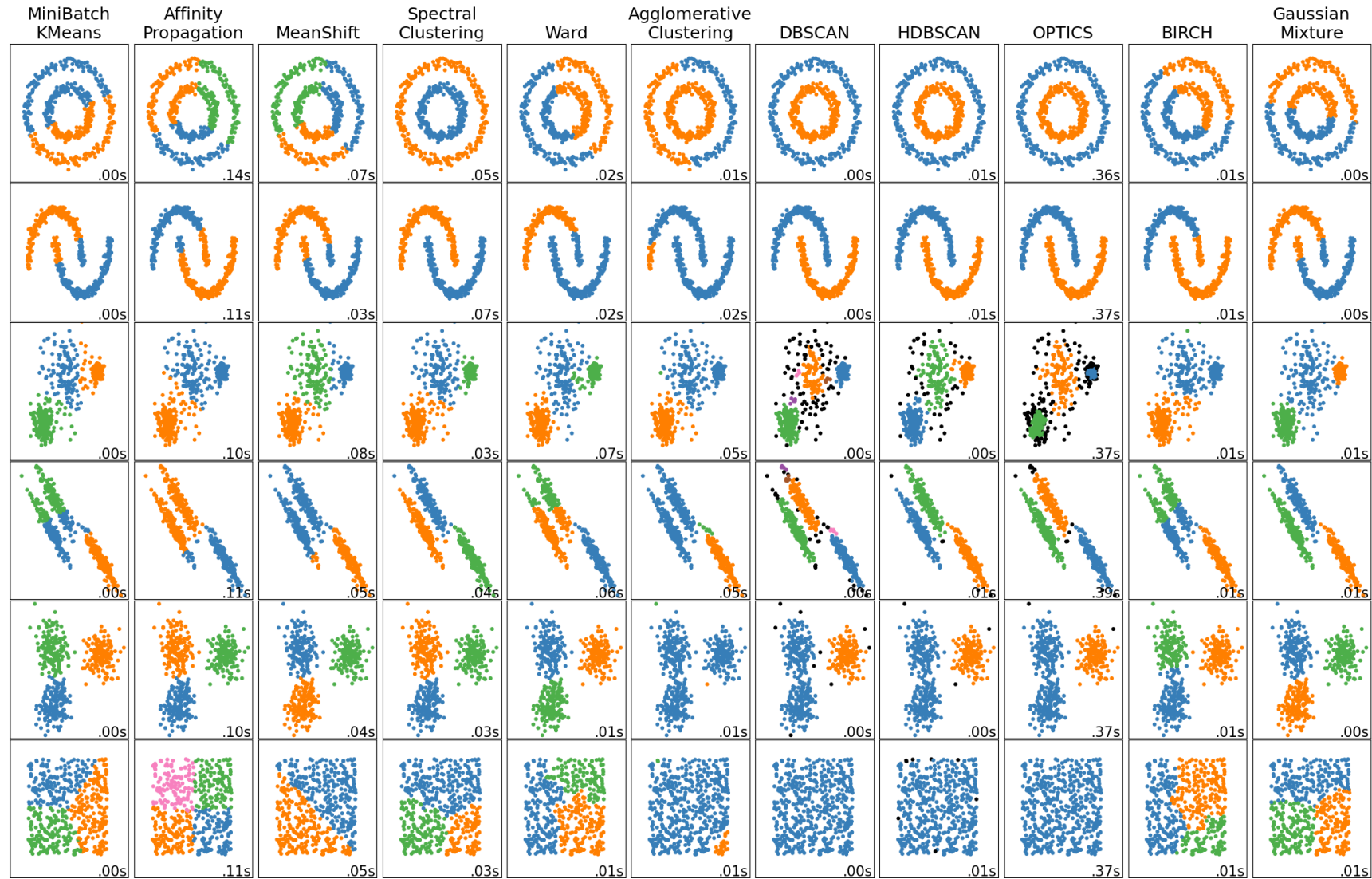


Cluster 4: High energy users



Clustering Use Cases





Clustering Algorithms in Scikit-Learn

UNSUPERVISED

SUPERVISED

SIMULATIONS

Idea: Similar data points should belong to the same group

How to measure similarity?

Euclidean distance or cosine similarity

>>> Need to define an error metric

K-means clustering:

- find K cluster centers

- predict the cluster of a point based on its closest cluster center

Suppose k clusters C_i with center z_i

We want to minimize

$$Cost(C_1, \dots, C_i, \dots, C_k, z_1, \dots, z_i, \dots, z_k) = \sum_{i=1}^k \sum_{j \in C_i} \|x_j - z_i\|^2$$

K-means clustering:

- find K cluster centers

- predict the cluster of a point based on its closest cluster center

Suppose k clusters C_i with center z_i

We want to minimize

$$Cost(C_1, \dots, C_i, \dots, C_k, z_1, \dots, z_i, \dots, z_k) = \sum_{i=1}^k \sum_{j \in C_i} \|x_j - z_i\|^2$$

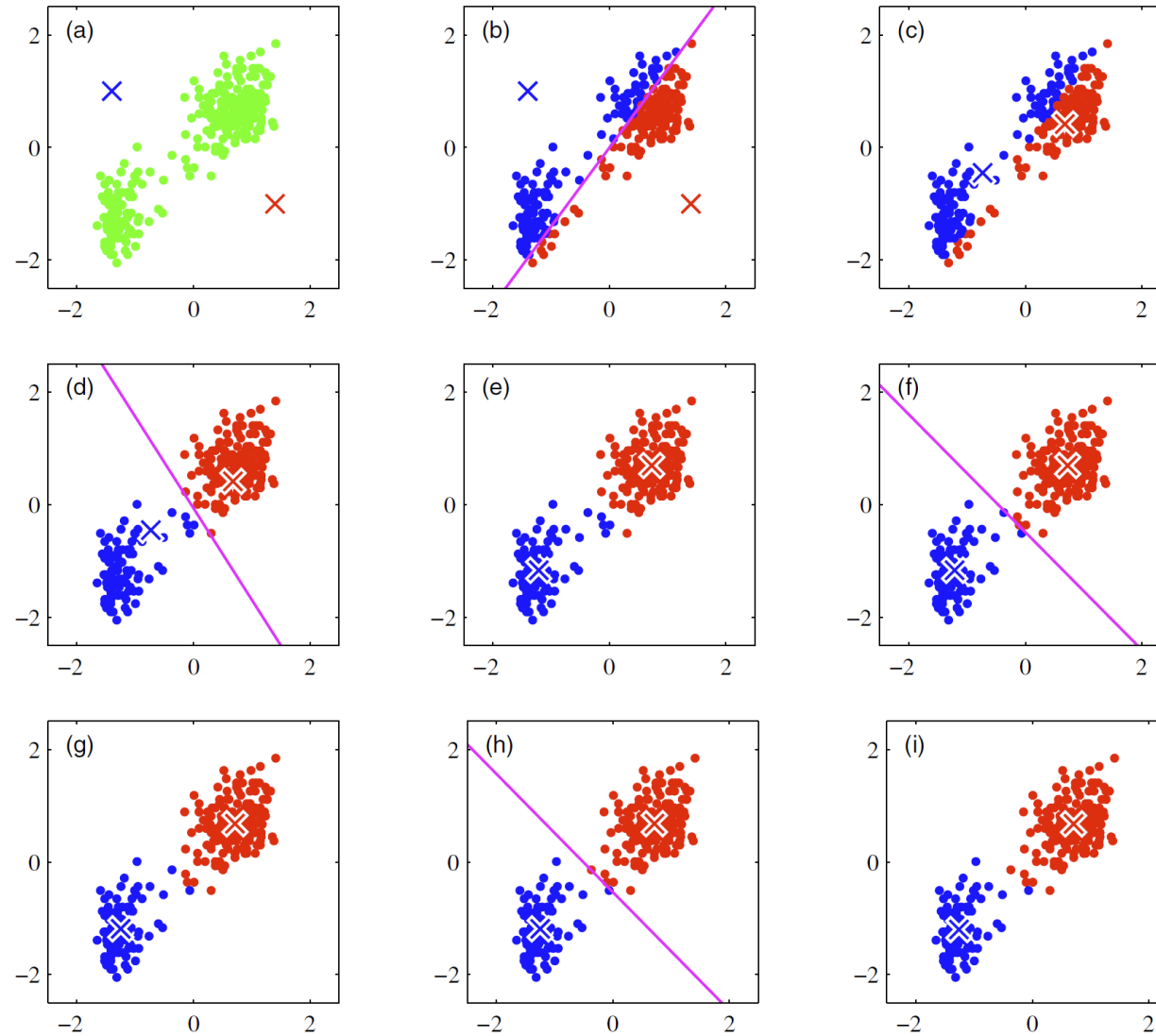
>>> K-Means Algorithm

1. Initialize z_i ($i = 1, \dots, k$)

2. Repeat until cost converges:

- (a) Assign points to the cluster of their closest centers: $C_j = \{i \text{ s.t. } x_i \text{ is closest to } z_j\}$

- (b) Assign cluster centers as means of the points in their clusters $z_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_j$



K-Means Algorithm

Bishop, Pattern Recognition and Machine Learning

1. Choose value for K
2. Initialize centroid locations
3. Compute distances between all points and centroids
4. Assign points to closest centroid
5. Re-compute cluster centroids
6. Repeat Steps 3-5 until convergence

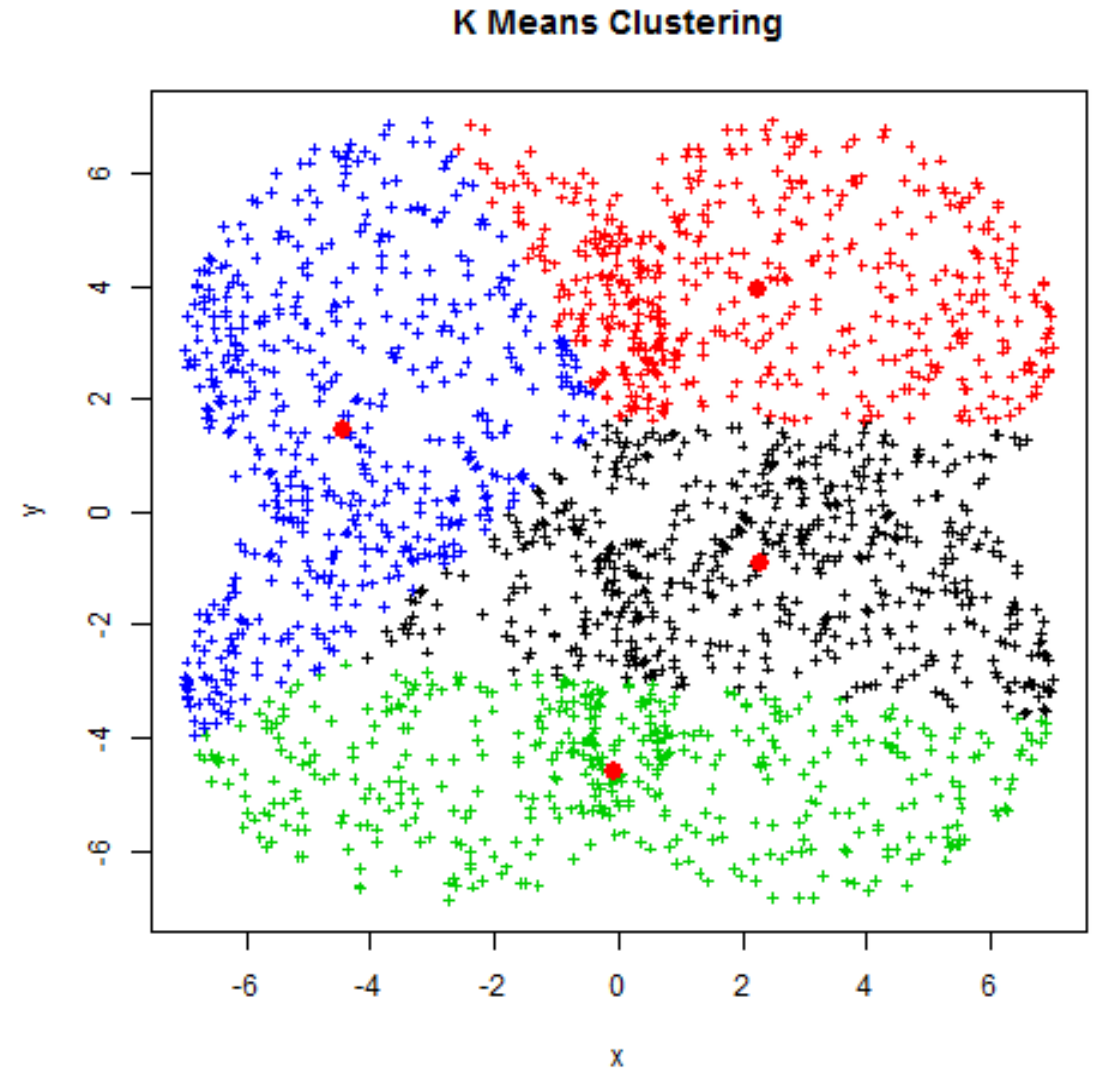




Image Segmentation
Bishop, Pattern Recognition and Machine Learning

Advantages and Disadvantages

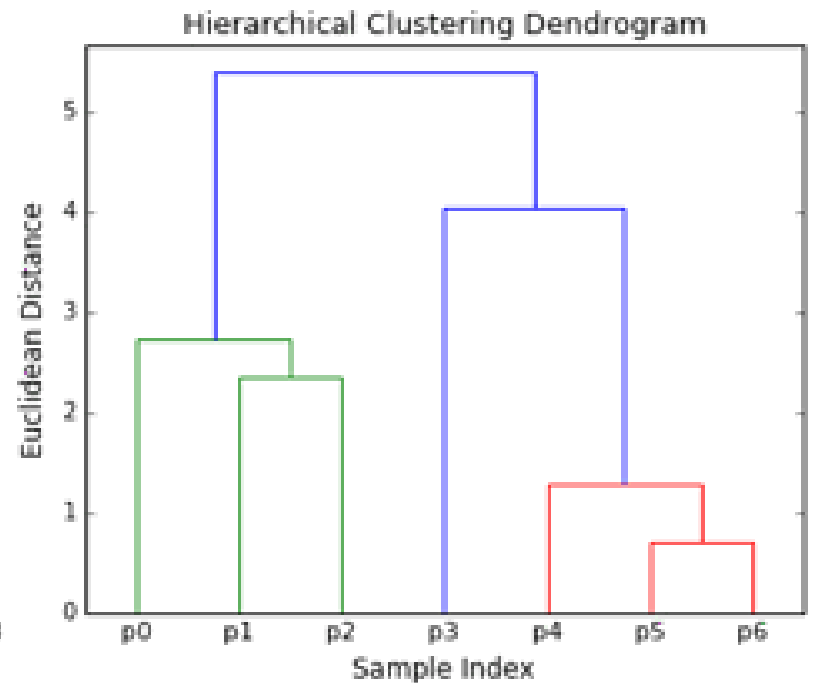
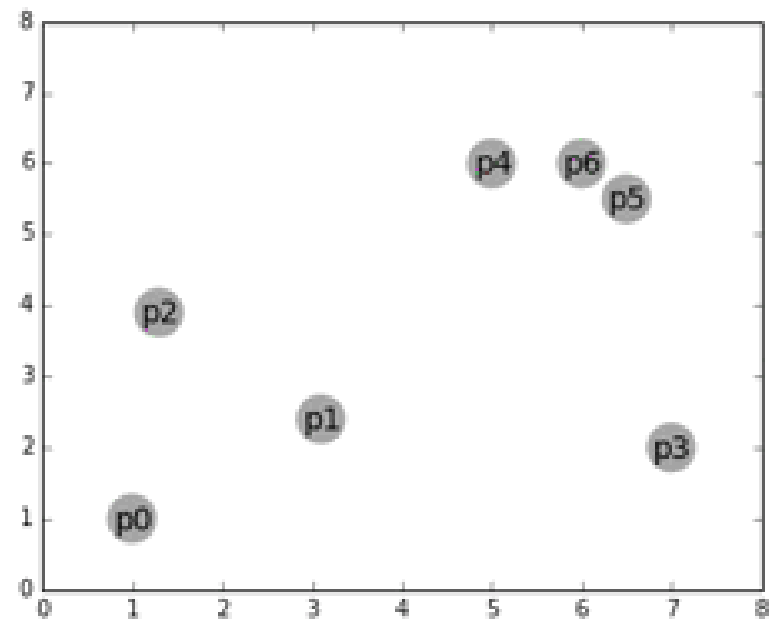
PROS

- Very easy to implement
- Often (given certain conditions) produces better results than hierarchical clustering
- Faster than hierarchical clustering for large data sets

CONS

- Difficult to predict the correct number for k clusters
- Outcome depends on a random initialization step

1. Start with each point in own cluster
2. Merge closest two clusters
3. Repeat 1 and 2 until one cluster



Advantages and Disadvantages

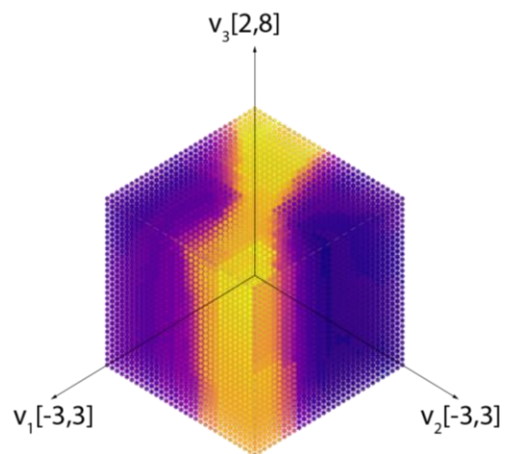
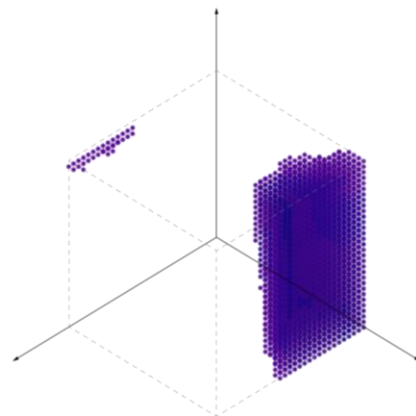
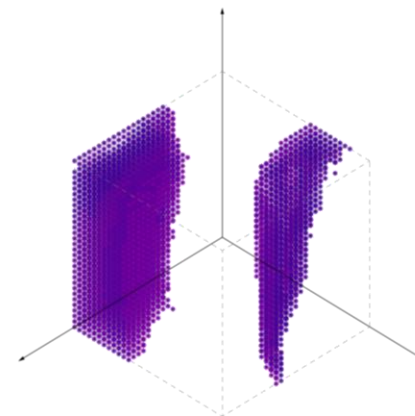
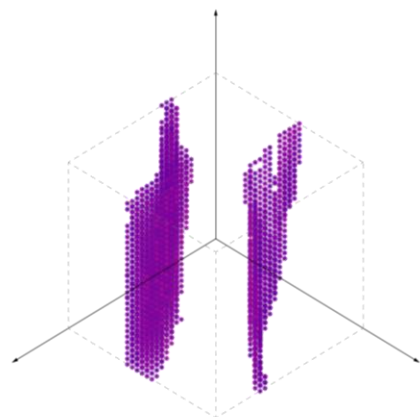
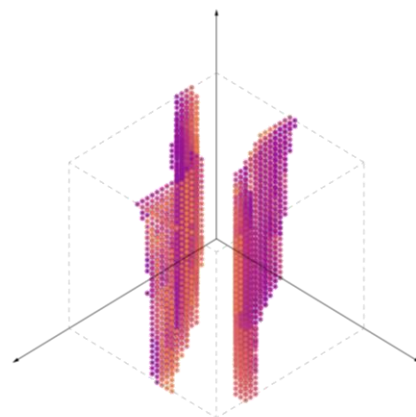
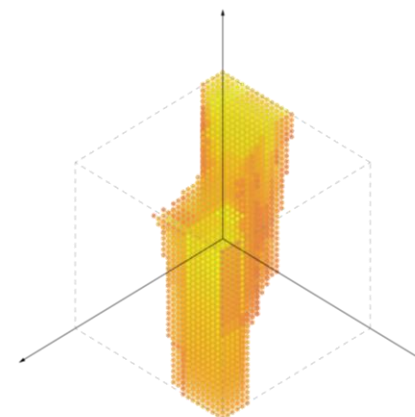
PROS

- Purely data-driven; no a priori parameters to choose
- Dendrogram easy to interpret, shows clustering process

CONS

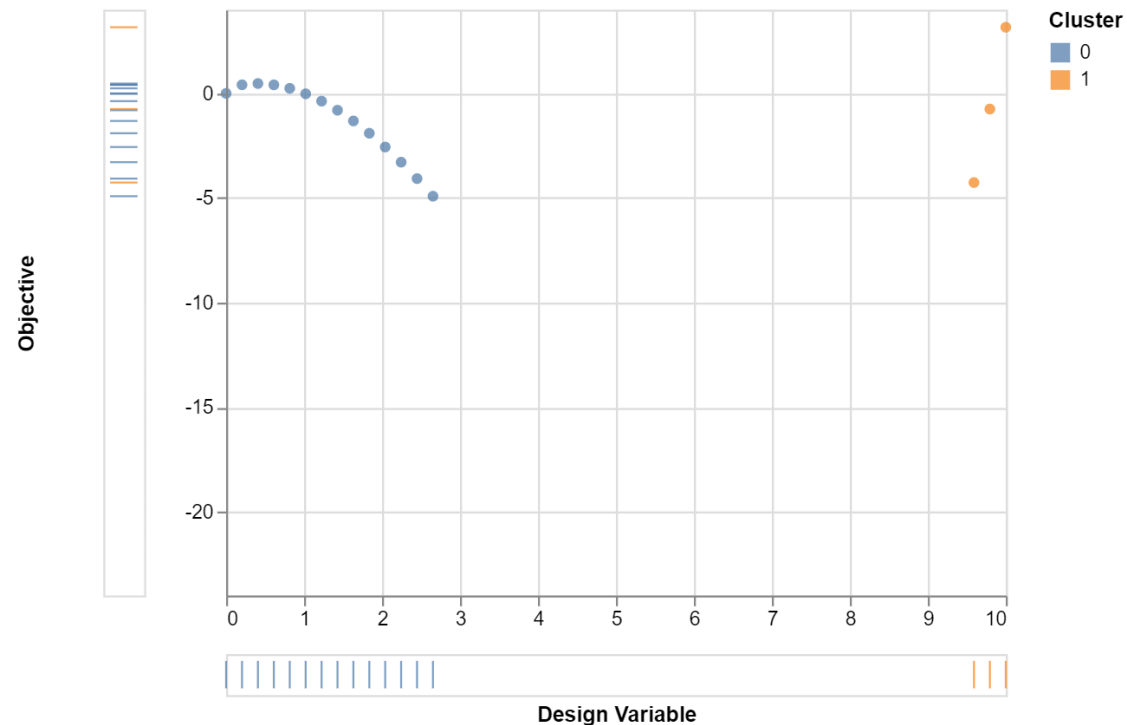
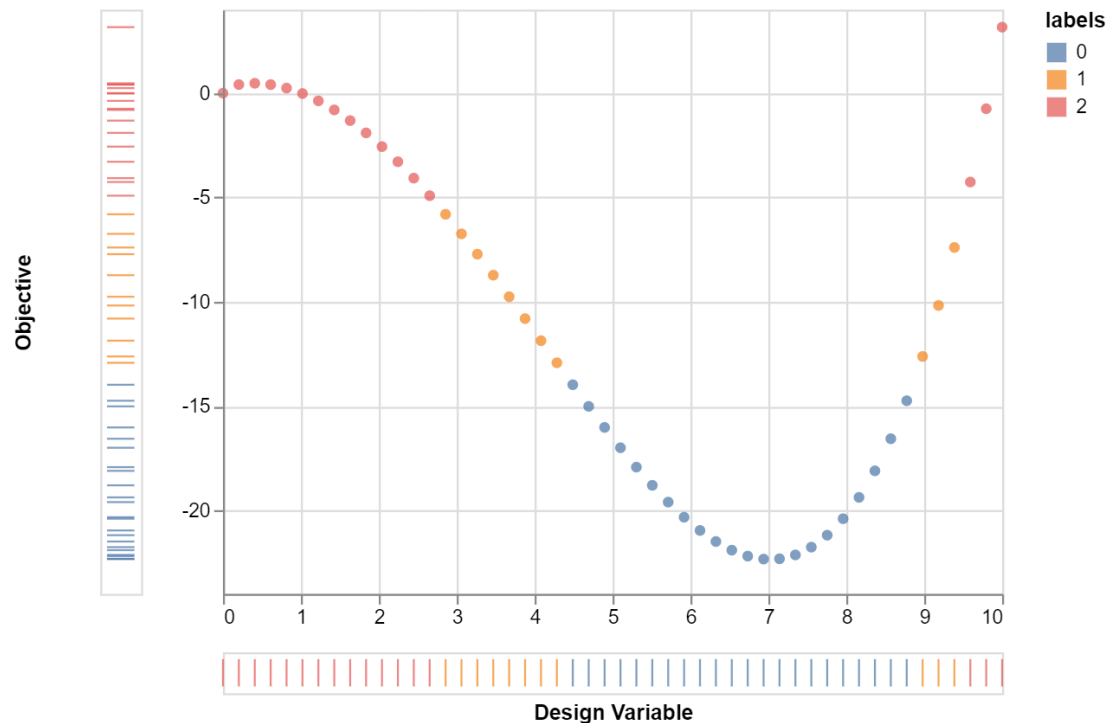
- Makes only one (myopic) pass through the data
- Low stability: can lead to very different results if only a few observations are left out
- Tends to be sensitive to outliers

Design Space Filtering

weight in $[0,1]$ weight in $[0,0.1]$ weight in $[0.1,0.2]$ weight in $[0.2,0.3]$ weight in $[0.3,0.7]$ weight in $[0.7,1]$ *Image from Renaud Danhaive*

Filtering the Design Space

Slice the design space in the objective space and analyze the top slices

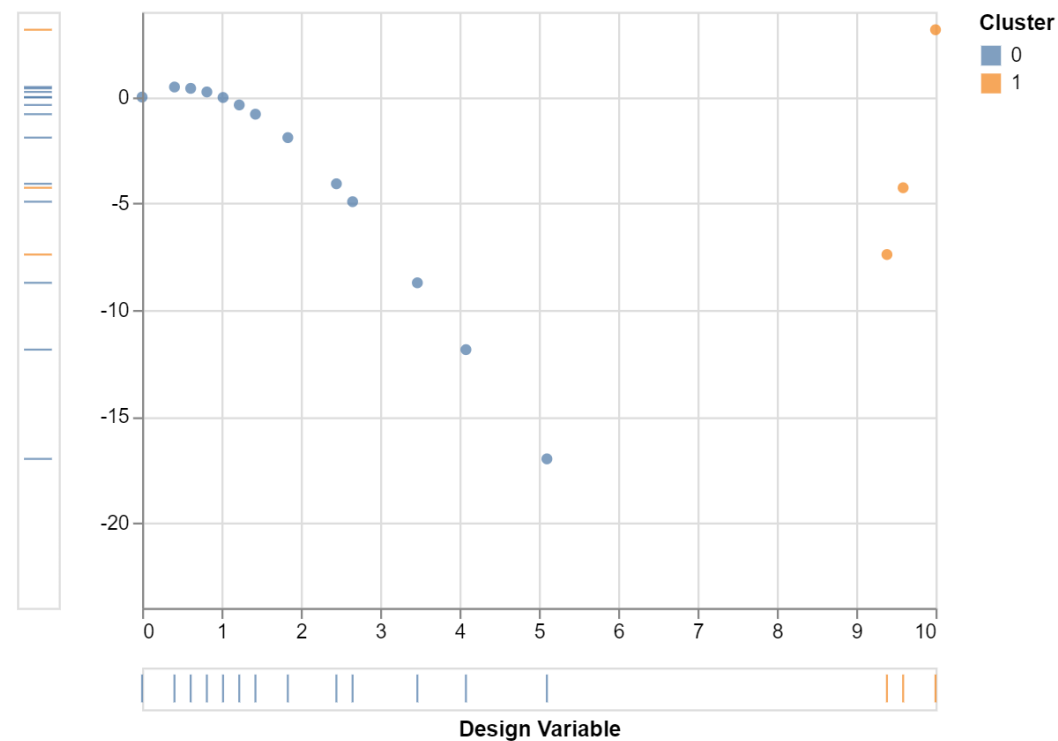
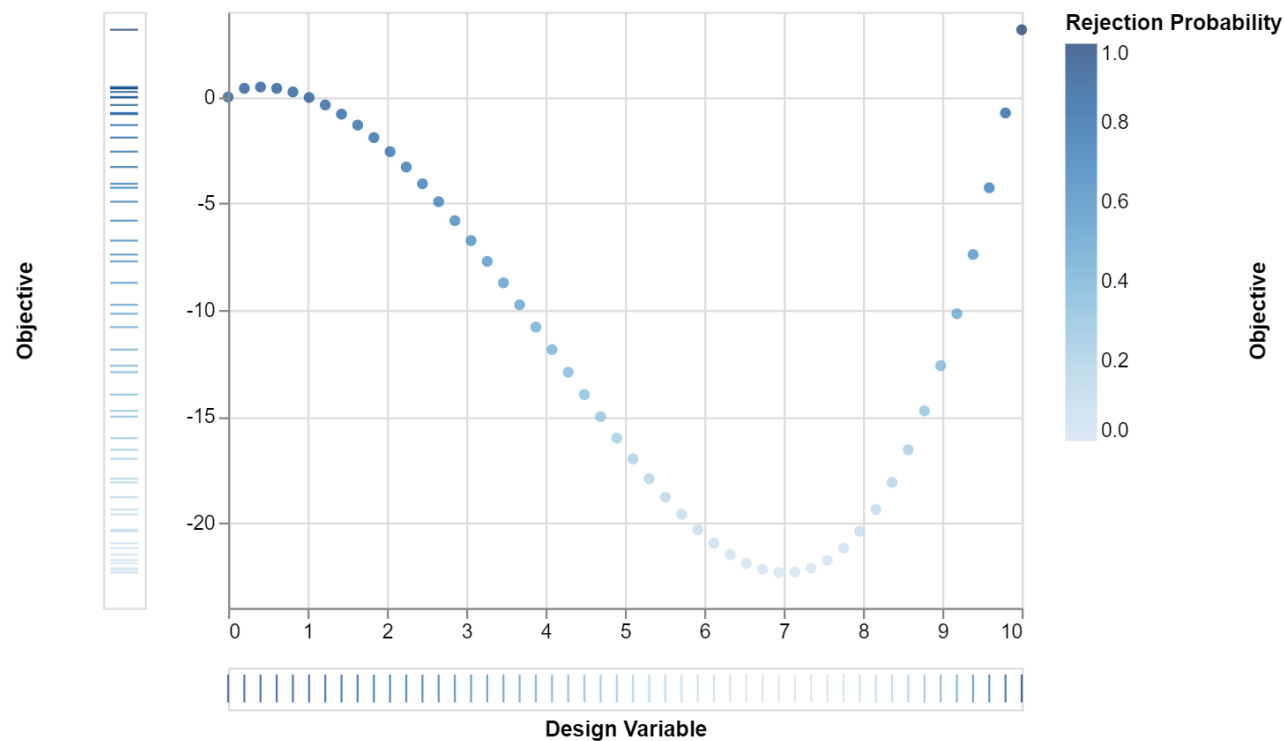


Hard slicing and filtering of 1D design space | Clustering in variable space

Image from Renaud Danhaive

Filtering the Design Space

Assign a rejection probability proportional to the score

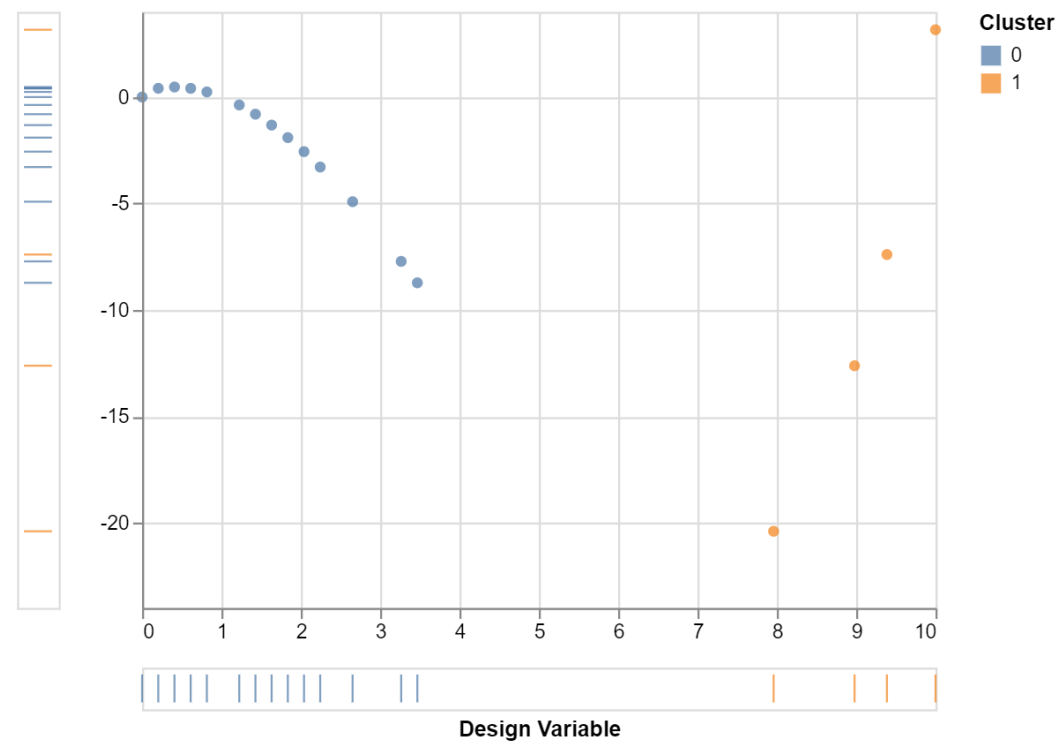
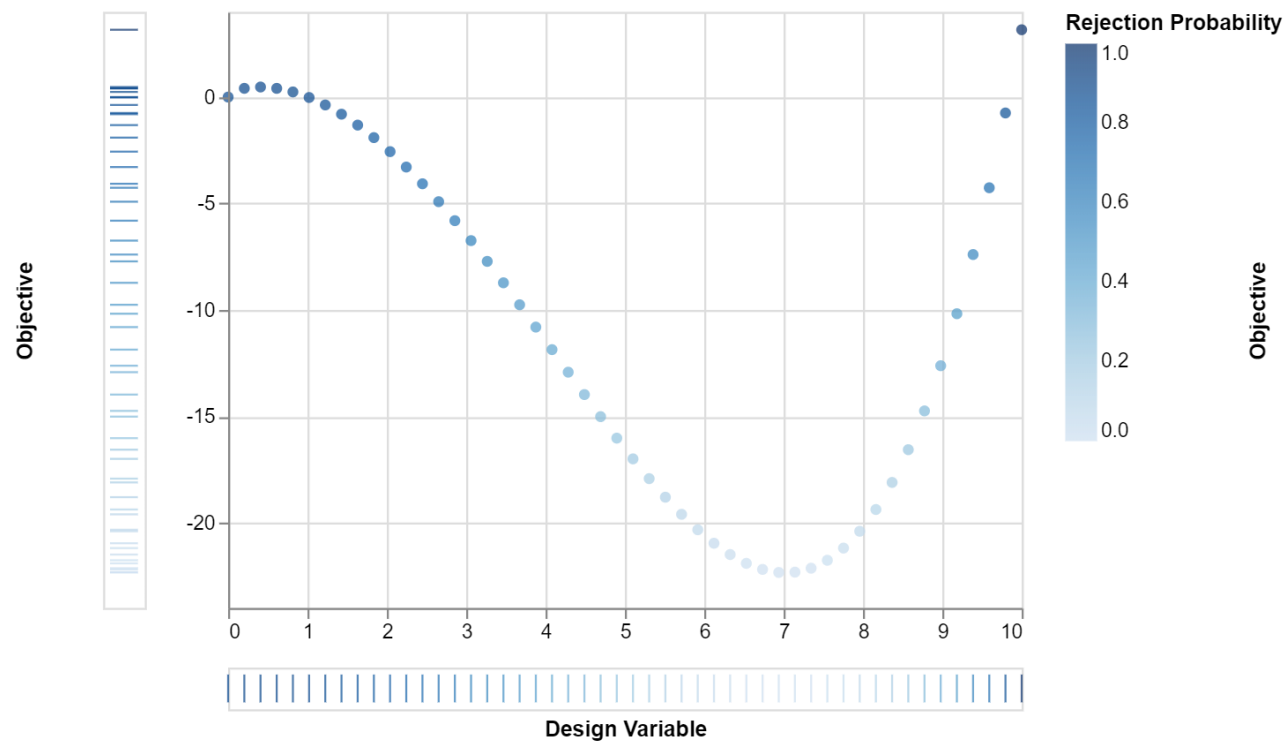


Soft filtering of 1D design space

Image from Renaud Danhaive

Filtering the Design Space

Assign a rejection probability proportional to the score



Soft filtering of 1D design space

Image from Renaud Danhaive

Design Space Visualization

design space links design variables (sometimes encoding geometry) and performance

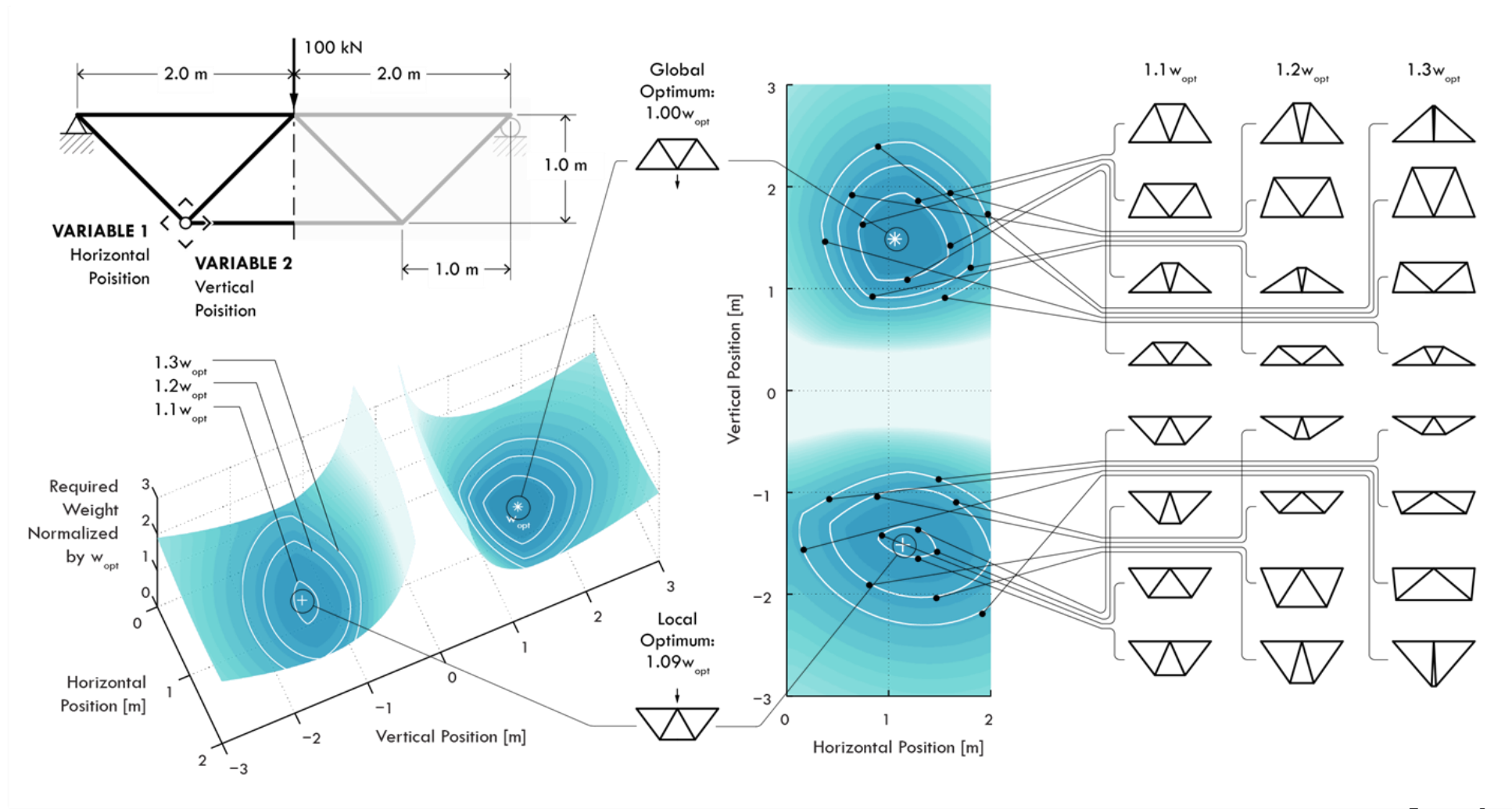


Image from Caitlin Mueller

UNSUPERVISED

SUPERVISE 7

SIMULATIONS

Design Space Visualization

design space links design variables (sometimes encoding geometry) and performance
3D > subpar because of occlusion (+ requires panning and rotating)

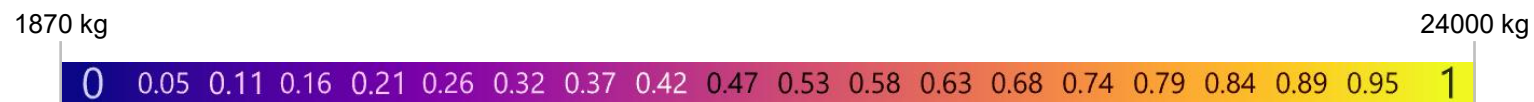
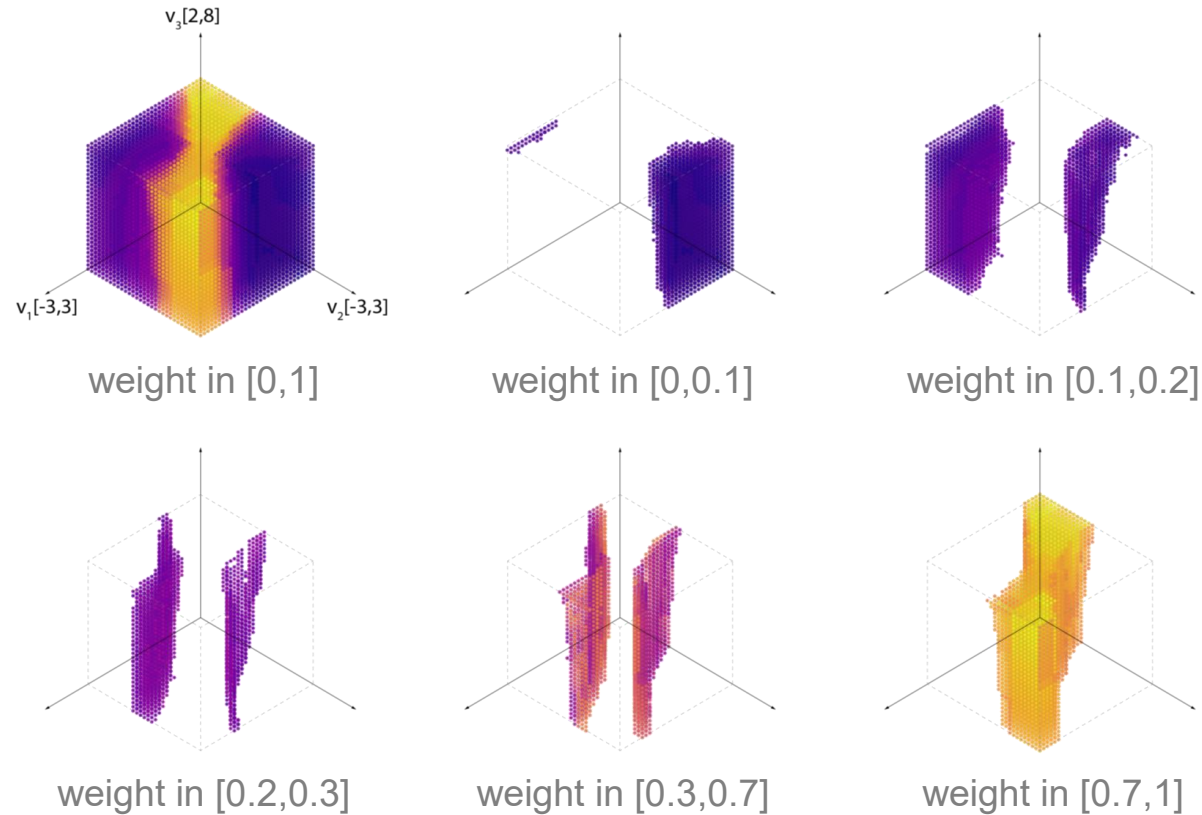


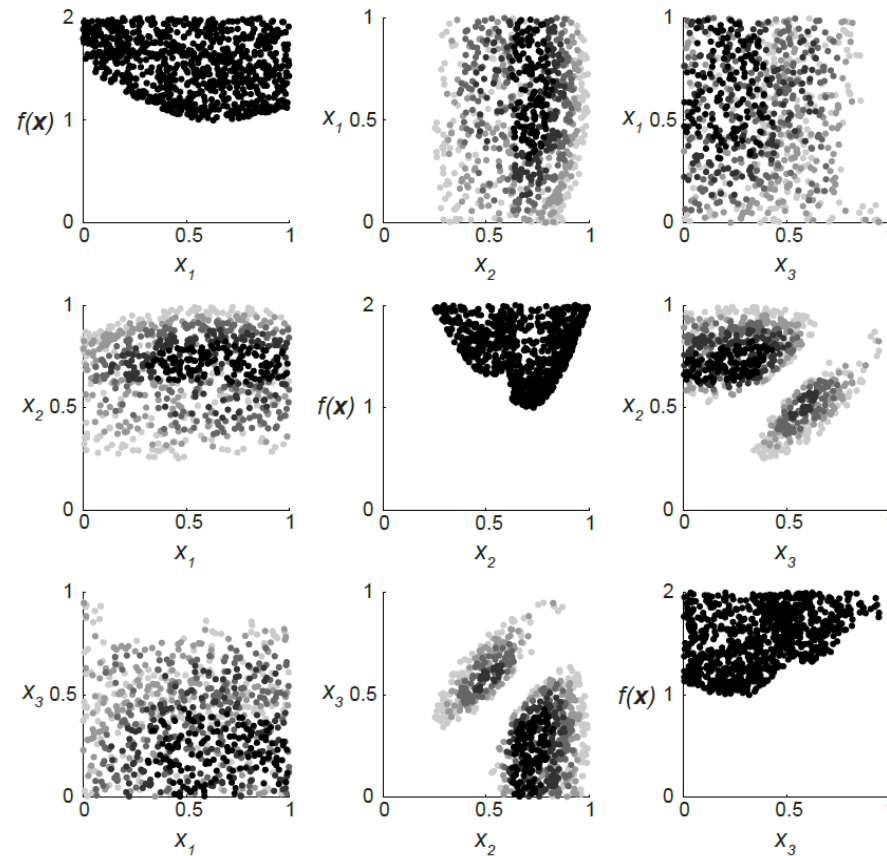
Image from Caitlin Mueller

UNSUPERVISED

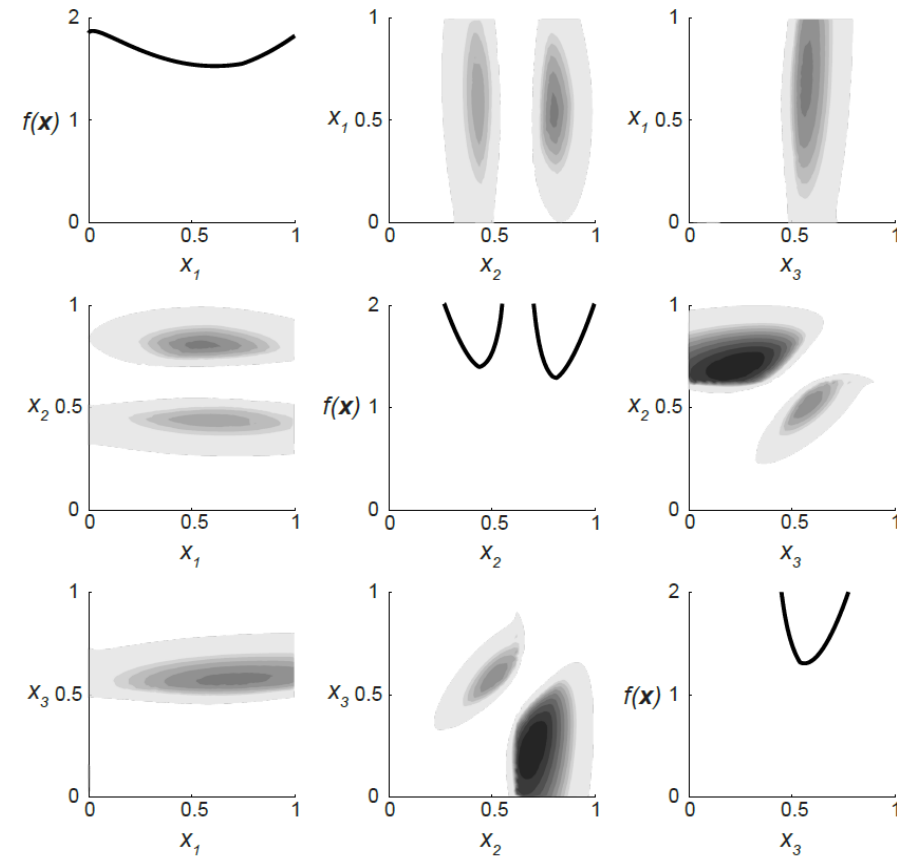
SUPERVISE 7

SIMULATIONS

Filtering and projecting the design space



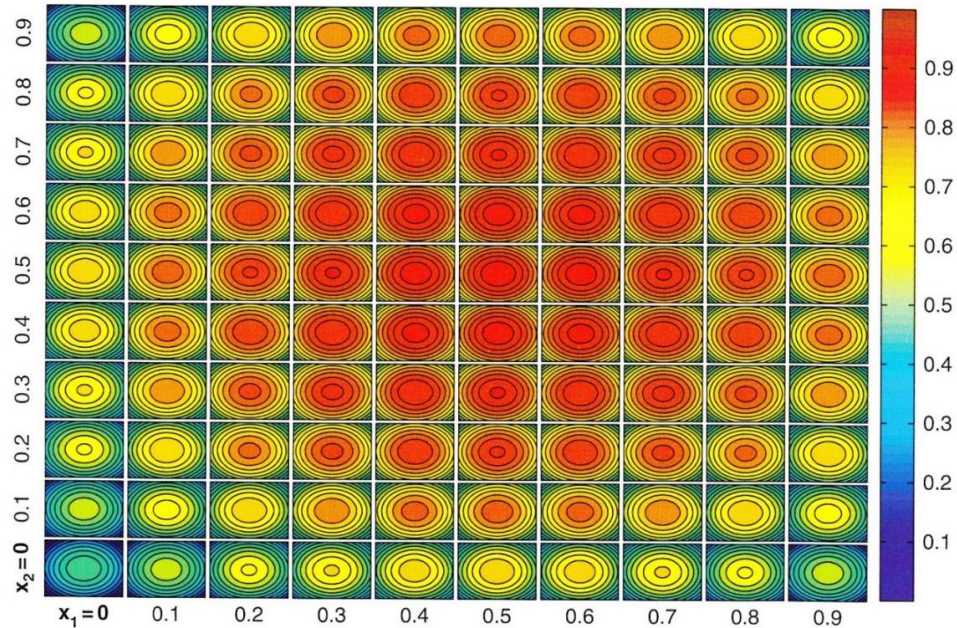
(a)



(b)

Nested Plots

4D



6D

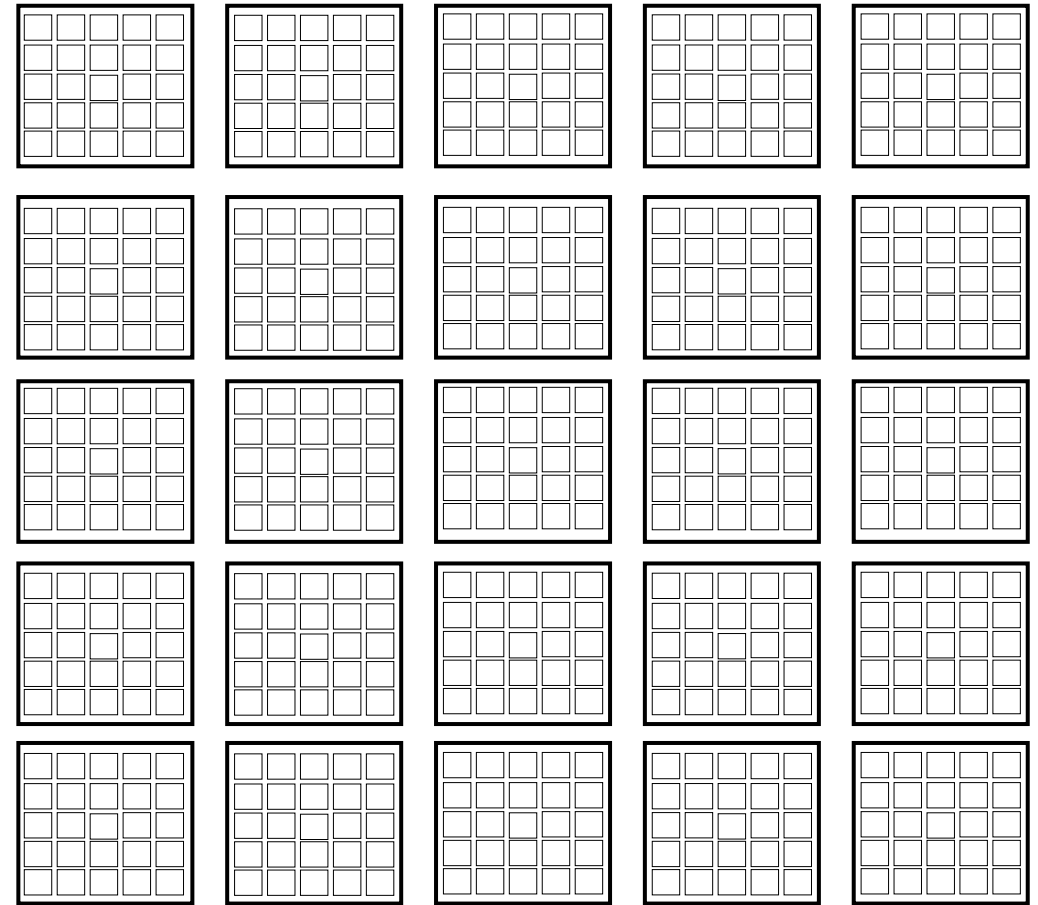
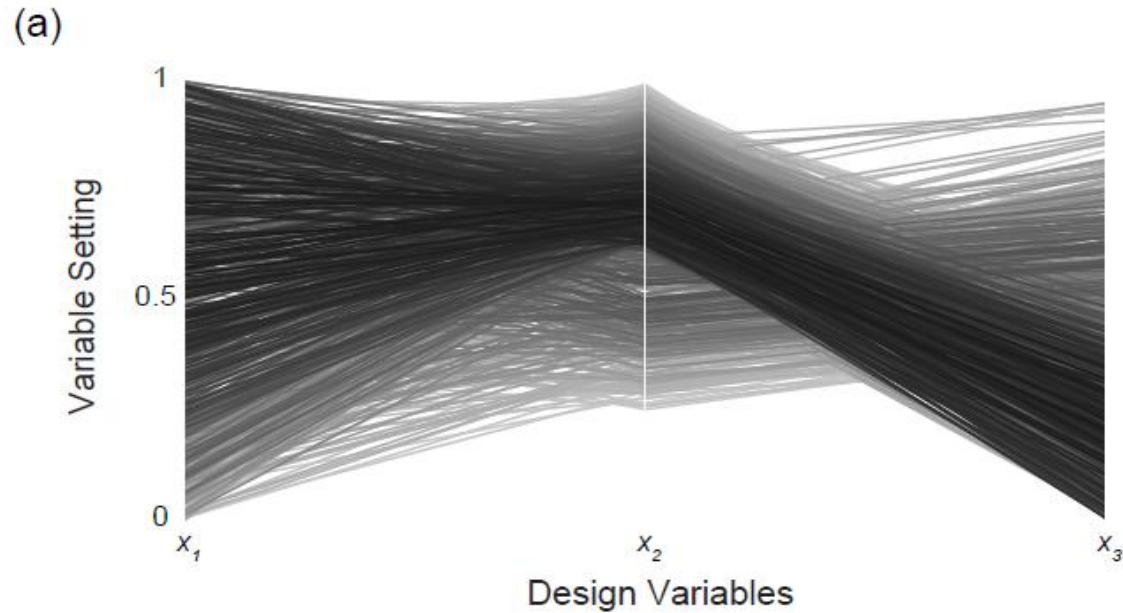


Plate II. (colour version of Figure 4.2) Four-variable nested plot of the surrogate of the function $f(\mathbf{x}) = 1/4 \sum_{i=1}^4 1 - (2x_i - 1)^2$, $\mathbf{x} \in [0, 1]^4$, generated using `nested4.m`. Here x_3 varies along the horizontal axis of each tile, x_4 along the vertical axes, while the values of x_1 and x_2 can be read off the bottom of each column of tiles and the beginning of each row respectively.

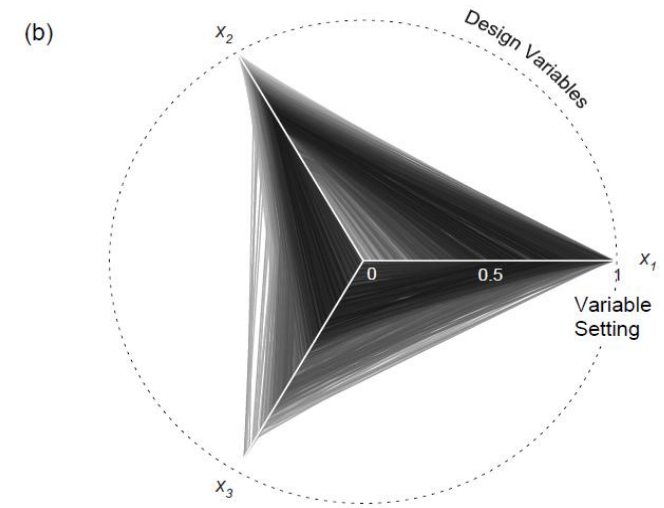
Engineering Design via SurrogateModelling: A Practical Guide
Forrester, Sobester, Keane, 2008

Coordinate Plots

Effective for high(ish) dimensions (~ 10)

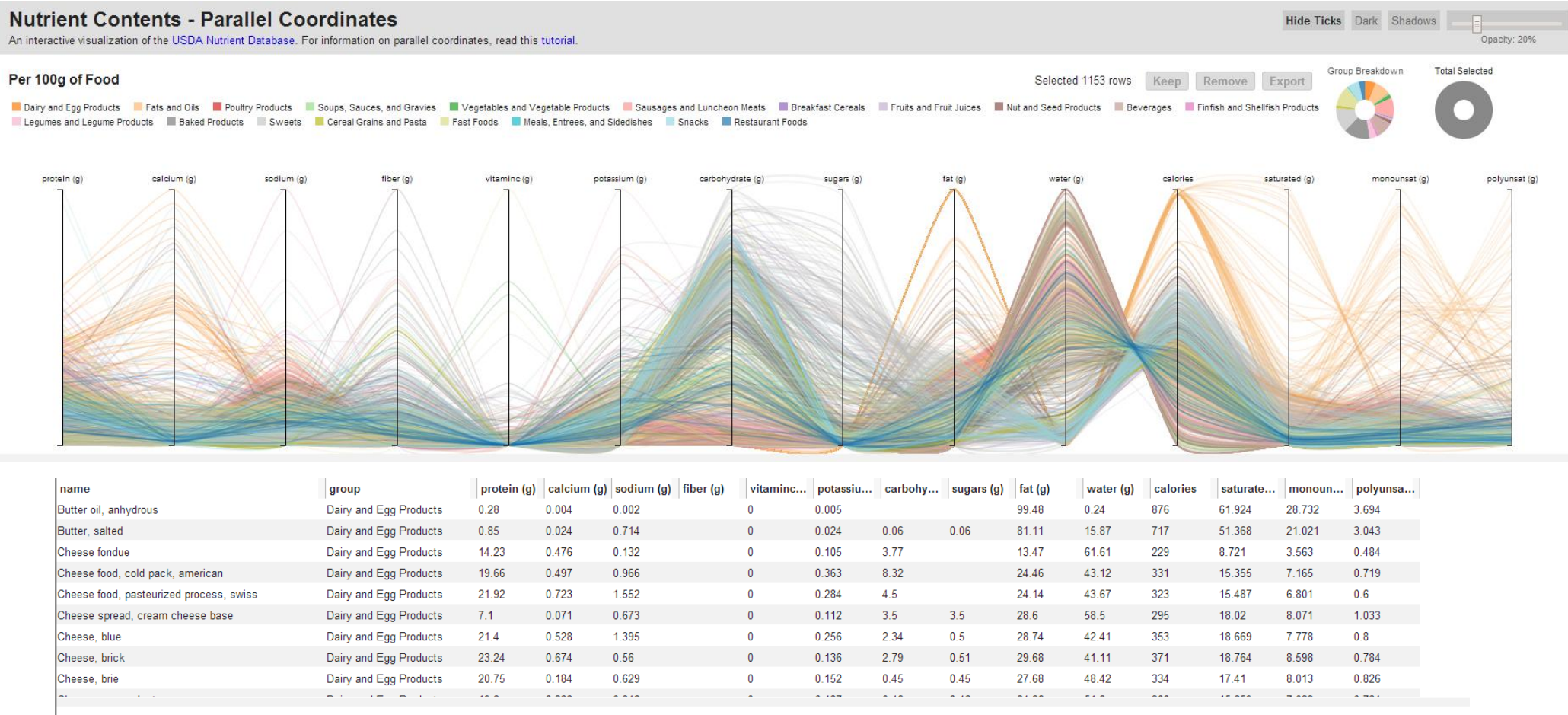


Parallel Coordinate Plots



Radar/Spider Plots

Coordinate Plots
Effective for high(ish) dimensions (~10)



UNSUPERVISED

SUPERVISED

SIMULATIONS

Google Colab
<https://bit.ly/BPS5231-L8>

What is 'Big Data'?

In the world of Data Science, the term *Big Data* gets thrown around a lot.

What does *Big Data* mean?

A rectangular data set has two dimensions: number of observations (n) and the number of predictors (p). Both can play a part in defining a problem as a *Big Data* problem.

What are some issues when:

- n is big (and p is small to moderate)?
- p is big (and n is small to moderate)?
- n and p are both big?

When n is big?

When the sample size is large, this is typically not much of an issue from the statistical perspective, just one from the computational perspective.

- Algorithms can take forever to finish. Estimating the coefficients of a regression model, especially one that does not have closed form (like LASSO), can take a while. Wait until we get to Neural Nets!
- If you are tuning a parameter or choosing between models (using CV), this exacerbates the problem.

What can we do to fix this computational issue?

- Perform 'preliminary' steps (model selection, tuning, etc.) on a subset of the training data set. 10% or less can be justified

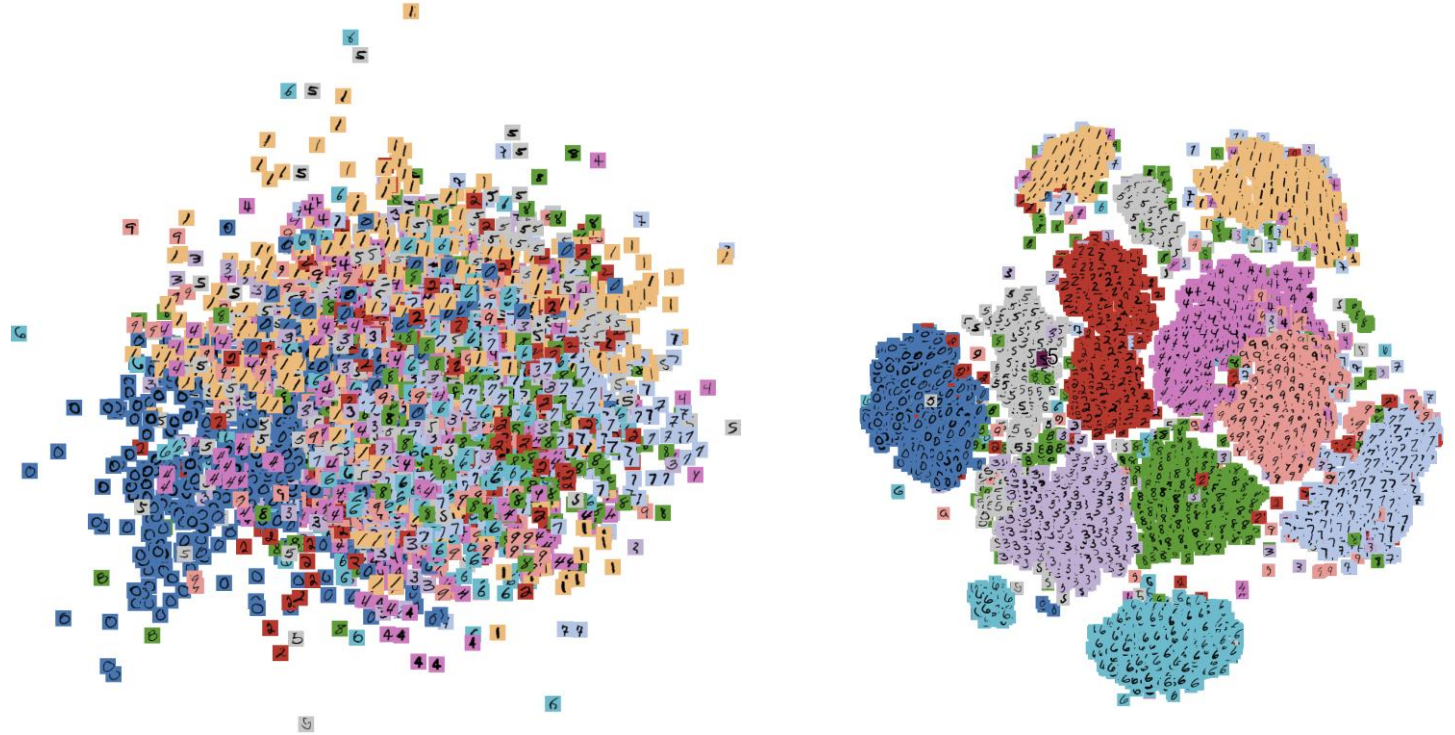
When does high dimensionality occur?

The problem of high dimensionality can occur when the number of parameters exceeds or is close to the number of observations. This can occur when we consider lots of interaction terms, like in our previous example. But this can also happen when the number of main effects is high.

For example:

- When we are performing polynomial regression with a high degree and a large number of predictors.
- When the predictors are genomic markers (and possible interactions) in a computational biology problem.
- When the predictors are the counts of all English words appearing in a text.

Dimensionality Reduction



UNSUPERVISED

SUPERVISE 7

SIMULATIONS

A framework for Dimensionality Reduction

One way to reduce the dimensions of the feature space is to create a new, smaller set of predictors by taking linear combinations of the original predictors.

We choose $Z_1, Z_2, \dots, Z_m$, where $m \leq p$ and where each Z_i is a linear combination of the original p predictors

$$Z_i = \sum_{j=1}^p \phi_{ji} X_j$$

for fixed constants ϕ_{ji} . Then we can build a linear regression model using the new predictors

$$Y = \beta_0 + \beta_1 Z_1 + \dots + \beta_m Z_m + \varepsilon$$

Notice that this model has a smaller number ($m+1 < p+1$) of parameters.

A framework for Dimensionality Reduction

A method of dimensionality reduction includes 2 steps:

- Determine an optimal set of new predictors $Z_1, \dots, Z_m$, for $m < p$.
- Express each observation in the data in terms of these new predictors. The transformed data will have m columns rather than p .

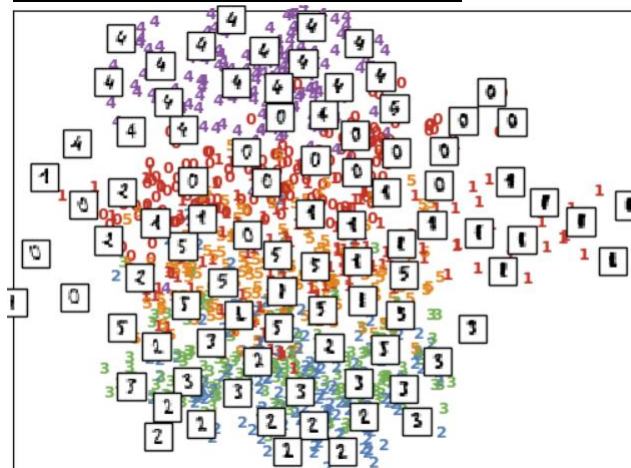
Thereafter, we can fit a model using the new predictors.

The method for determining the set of new predictors (what do we mean by an optimal predictors set?) can differ according to application. We will explore a way to create new predictors that captures the *essential* variations in the observed predictor set.

Dimensionality Reduction

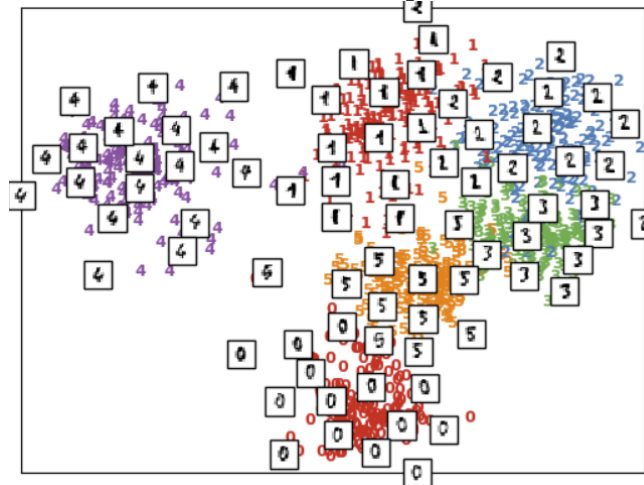
PCA

Principal Component Analysis



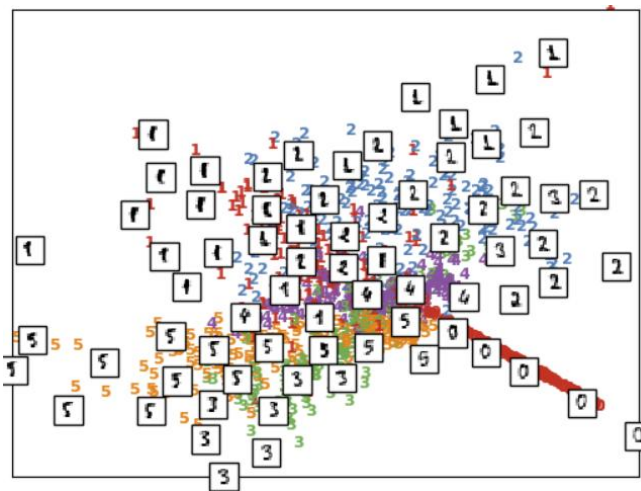
LDA

Linear Discriminant Analysis



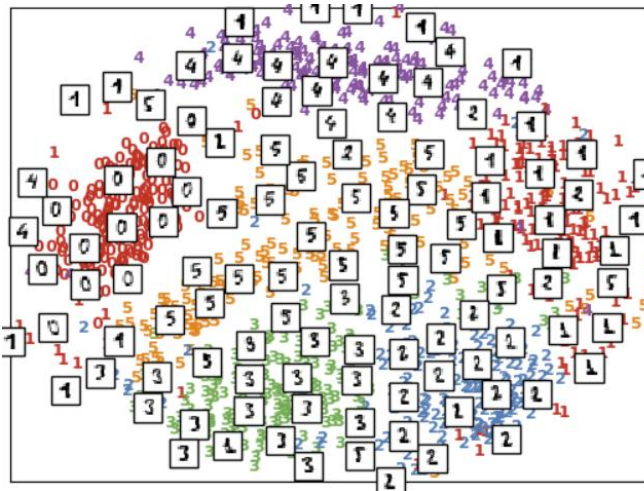
LLE

Locally Linear Embedding



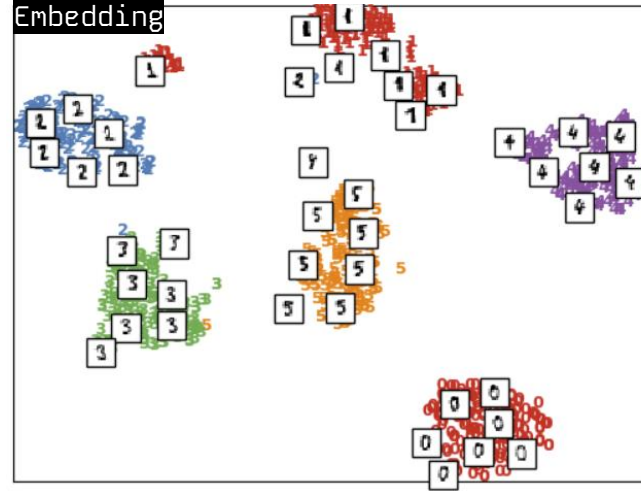
MDS

Multidimensional Scaling



t-SNE

t-Distributed Stochastic Neighbor Embedding



UNSUPERVISED

SUPERVISED

SIMULATION

Selection of dimensionality reduction algorithms in Scikit-Learn

Principal Component Analysis (PCA)

Idea: Find directions that explain the variance of the data the best

Project data (linearly) onto a subspace such that variance of the projected data is maximized

$$S_n = \{\mathbf{x}_i \in \mathbb{R}^d, i = 1, \dots, N\}$$

Project $\mathbf{x}_i \in \mathbb{R}^d$ onto $\mathbb{R}^m$ with $m < d$

What if $m = 1$?

Datapoints are projected onto $\mathbf{u}_1$ where $\mathbf{u}_1^T \mathbf{u}_1 = 1$

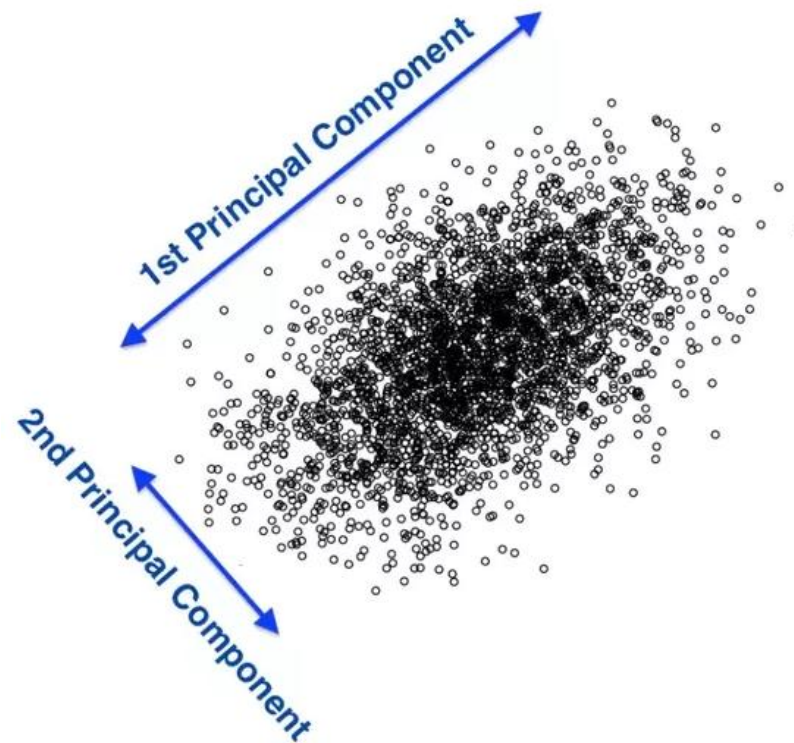
Projection $\mathbf{x}_i \rightarrow \mathbf{u}_1^T \mathbf{x}_i \in \mathbb{R}$

>>> Mean of projected data

$$\mathbf{u}_1^T \bar{\mathbf{x}} \text{ where } \bar{\mathbf{x}} = \frac{1}{N} \sum_i \mathbf{x}_i$$

>>> Variance of projected data

$$\frac{1}{N} \sum_i (\mathbf{u}_1^T \bar{\mathbf{x}} - \mathbf{u}_1^T \mathbf{x}_i)^2$$



Principal Component Analysis (PCA)

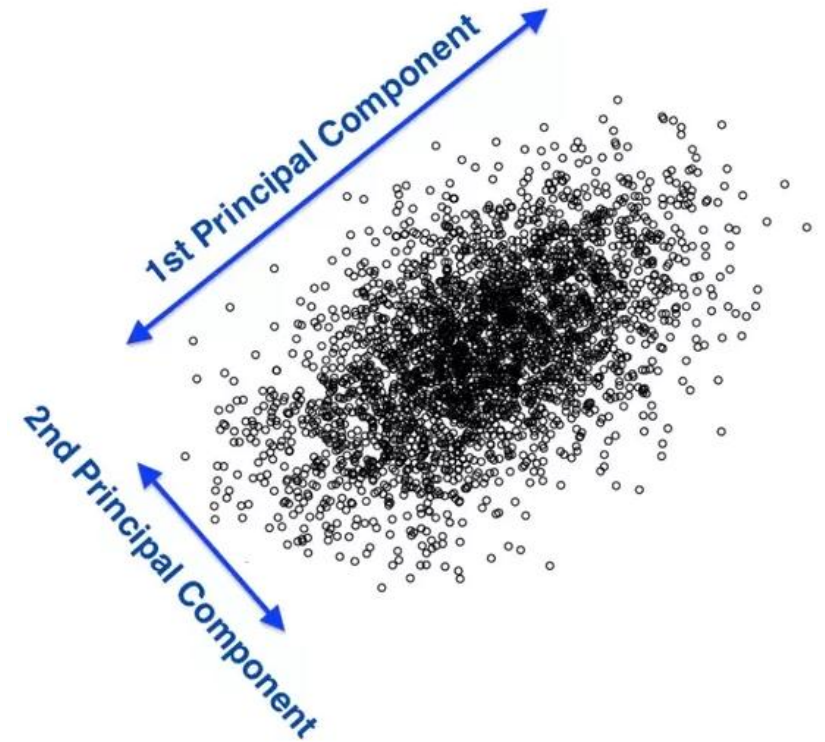
Principal Components Analysis (PCA) is a method to identify a new set of predictors, as linear combinations of the original ones, that captures the 'maximum amount' of variance in the observed data.

Principal Components Analysis (PCA) produces a list of p principle components $Z_1, \dots, Z_p$ such that

- Each Z_i is a linear combination of the original predictors, and its vector norm is 1
- The Z_i 's are pairwise orthogonal
- The Z_i 's are ordered in decreasing order in the amount of captured observed variance.

That is, the observed data shows more variance in the direction of Z_1 than in the direction of Z_2 .

To perform dimensionality reduction we select the top m principle components of PCA as our new predictors and express our observed data in terms of these predictors.

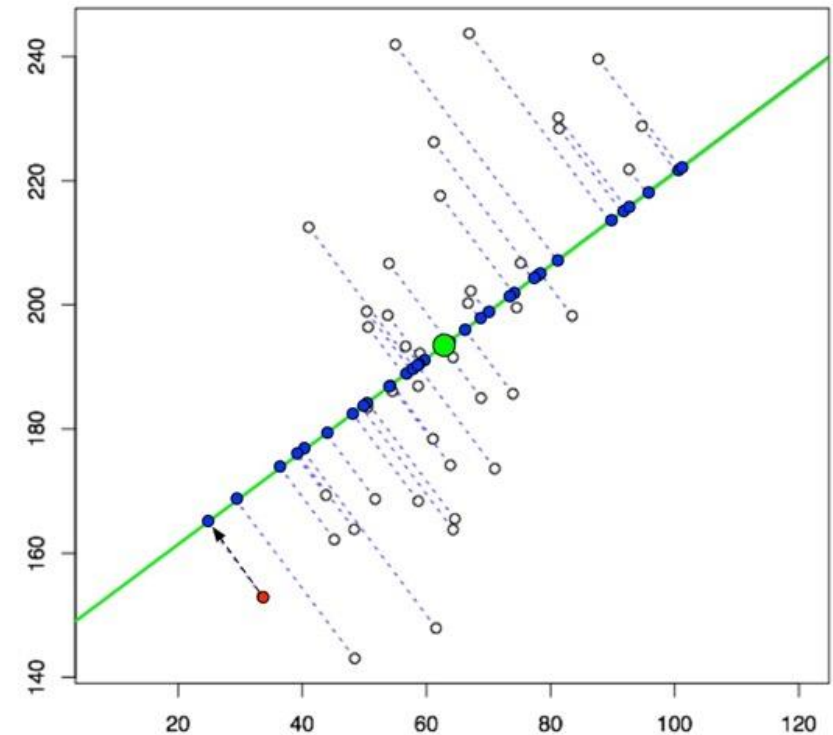


Principal Component Analysis (PCA)

Top PCA components capture the most of amount of variation (interesting features) of the data.

Each component is a linear combination of the original predictors - we visualize them as vectors in the feature space.

Transforming our observed data means projecting our dataset onto the space defined by the top m PCA components, these components are our new predictors.



PCA for Regression (PCR)

PCA is easy to use in Python, so how do we then use it for regression modeling in a real-life problem?

If we use all p of the new Z_j , then we have not improved the dimensionality. Instead, we select the first M PCA variables, $Z_1, \dots, Z_M$, to use as predictors in a regression model.

The choice of M is important and can vary from application to application. It depends on various things, like how collinear the predictors are, how truly related they are to the response, etc ...

What would be the best way to check for a specified problem?

Cross Validation!!!

PCA is an unsupervised algorithm. Meaning? It is done independent of the outcome variable.

- Note: the components as predictors might not be ordered from best to worst!

PCA is not so good because:

1. Direct Interpretation of coefficients in PCR is completely lost. So do not do if interpretation is important.
2. Will not improve predictive ability of a model.

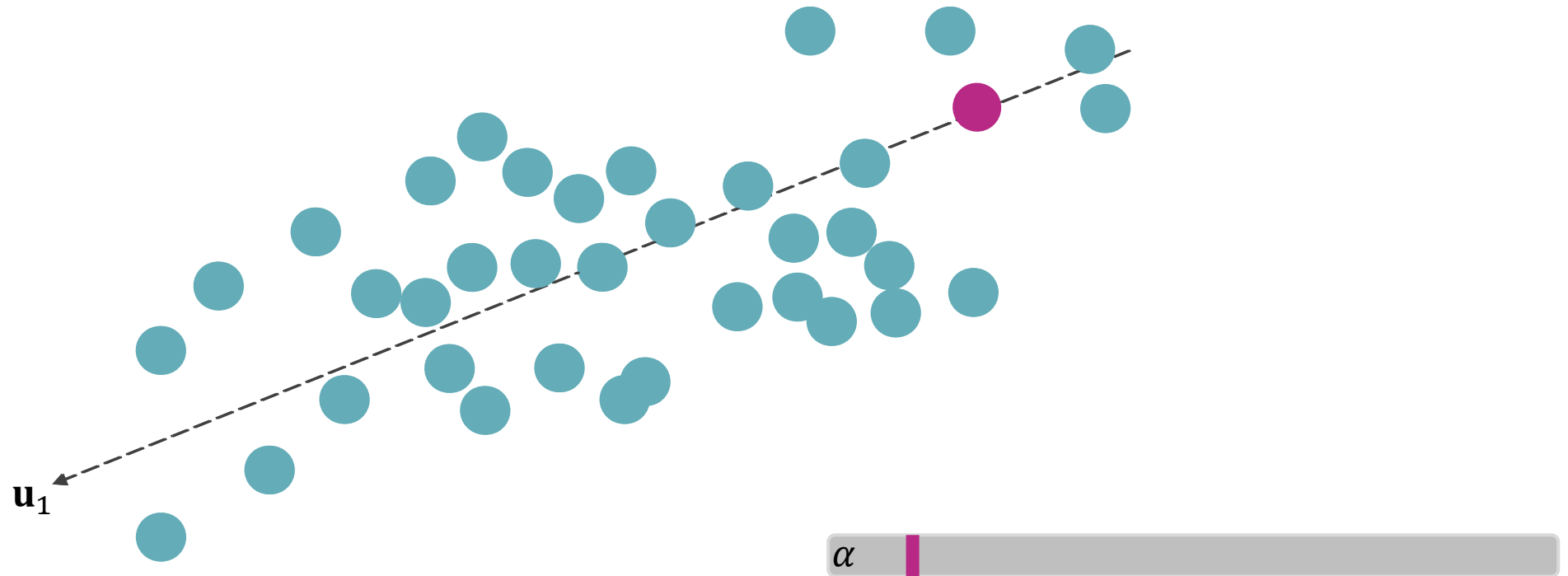
PCA is great for:

1. Reducing dimensionality in very high dimensional settings.
2. Visualizing how predictive your features can be of your response, especially in the classification setting.
3. Reducing multicollinearity, and thus may improve the computational time of fitting models.

PCA >>> the subspace is meaningful

can generate data in the high-dimensional space from low-dimensional input

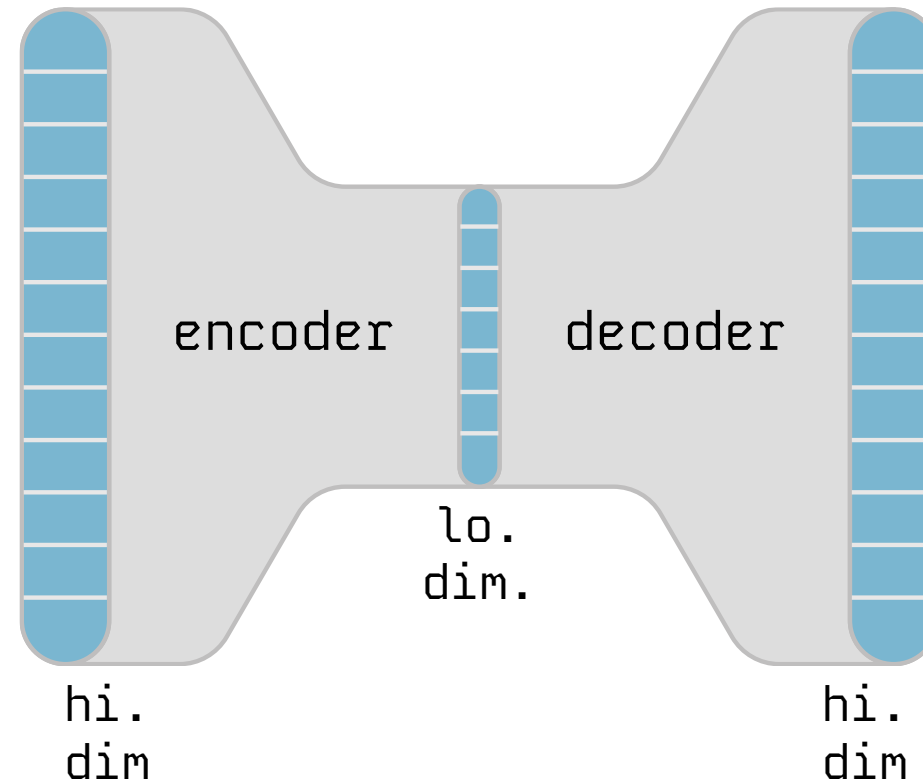
e.g. $\mathbf{x}_{new} = \alpha * \mathbf{u}_1$



PCA >>> the subspace is meaningful

can generate data in the high-dimensional space from low-dimensional input

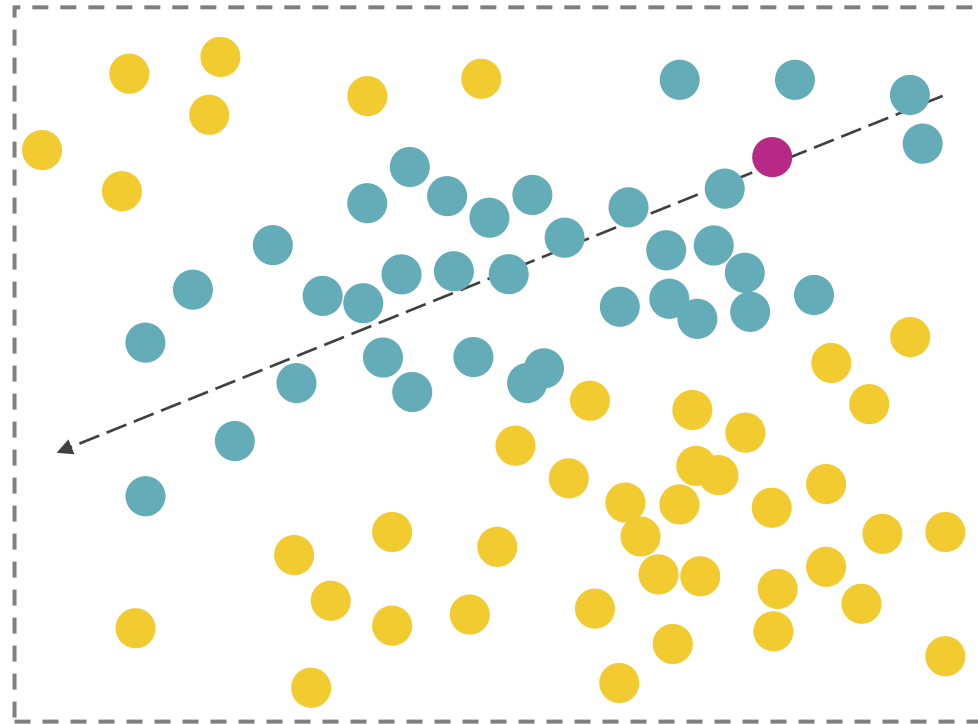
e.g. $\mathbf{x}_{new} = \alpha * \mathbf{u}_1$



General approach:

filter design data (based on objective/properties)

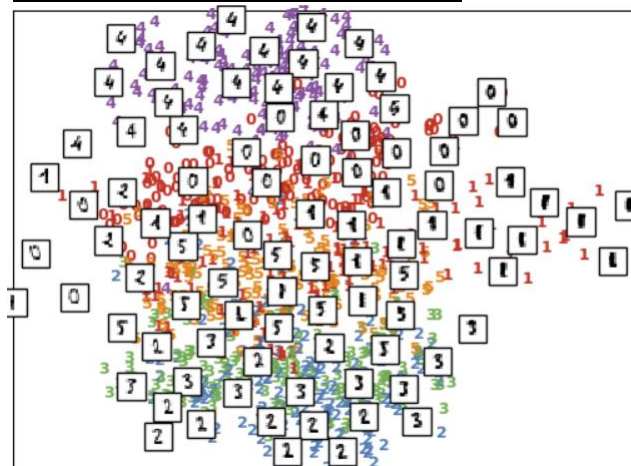
learn a subspace representation > explore that subspace



Dimensionality Reduction

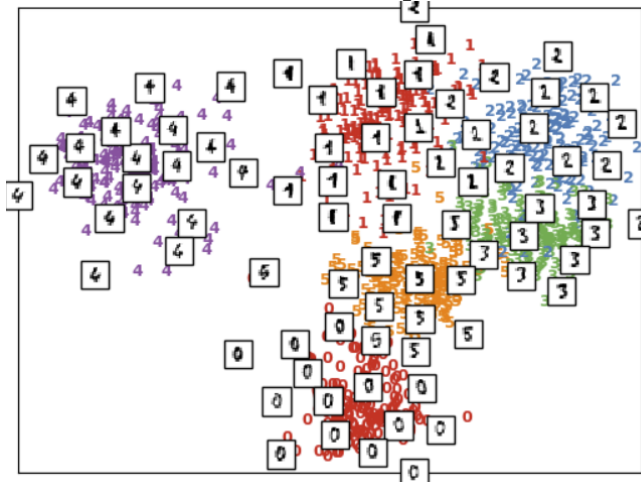
PCA

Principal Component Analysis



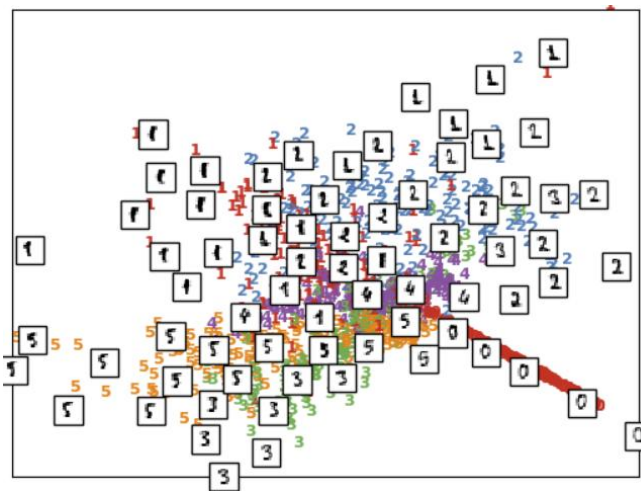
LDA

Linear Discriminant Analysis



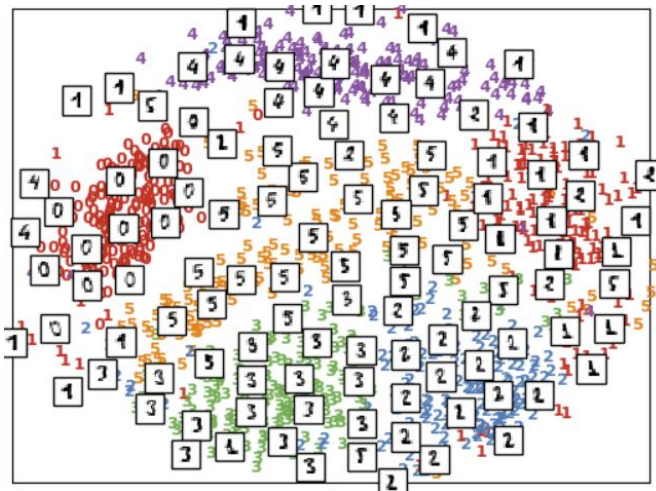
LLE

Locally Linear Embedding



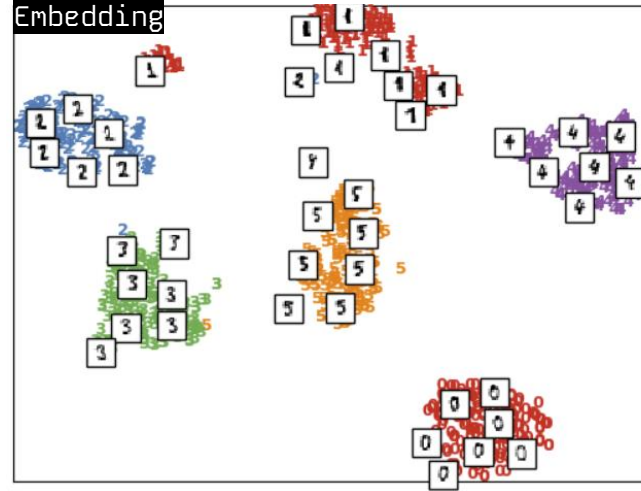
MDS

Multidimensional Scaling



t-SNE

t-Distributed Stochastic Neighbor Embedding

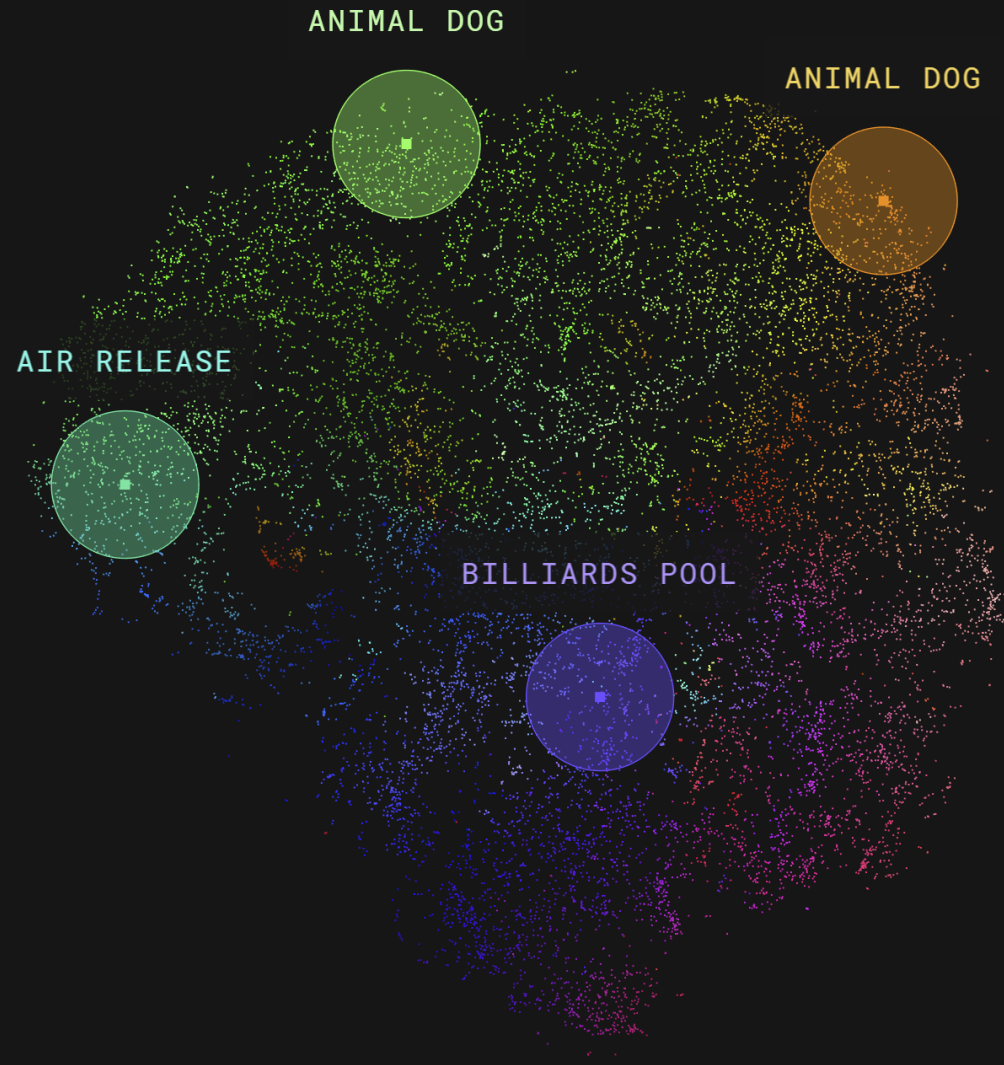


UNSUPERVISED

SUPERVISED

SIMULATION

Selection of dimensionality reduction algorithms in Scikit-Learn



Infinite Drum Machine

UNSUPERVISED

SUPERVISE 7

SIMULATIONS

<https://experiments.withgoogle.com/ai/drum-machine/view/>

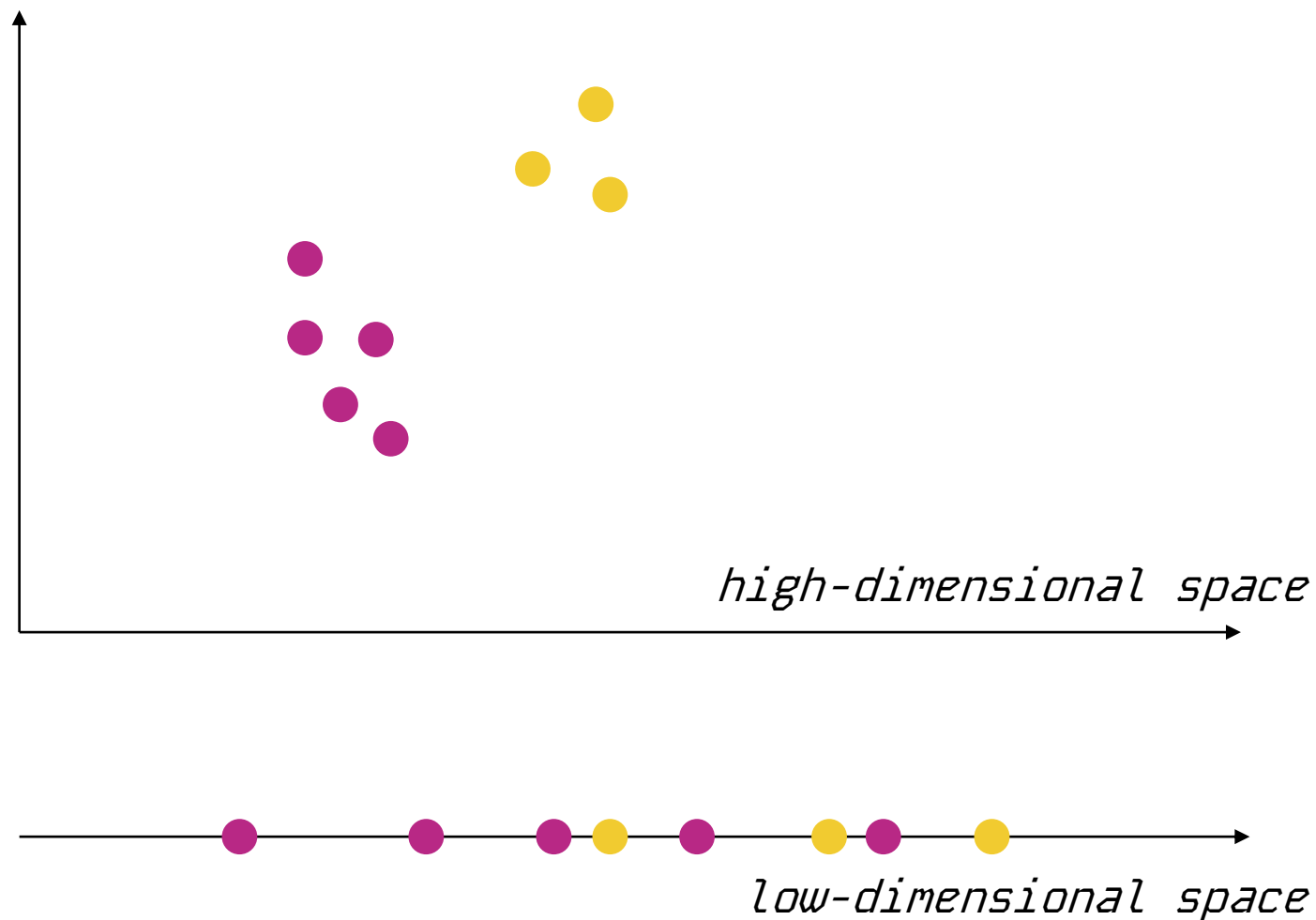
Van der Maaten & Hinton, 2008

Objective

visualize high-dimensional data
in a (much) lower-dimensional
space in a way that preserves
clusters and local structure

Intuition:

Particle physics simulation



Objective

visualize high-dimensional data in a (much) lower-dimensional space in a way that preserves clusters and local structure

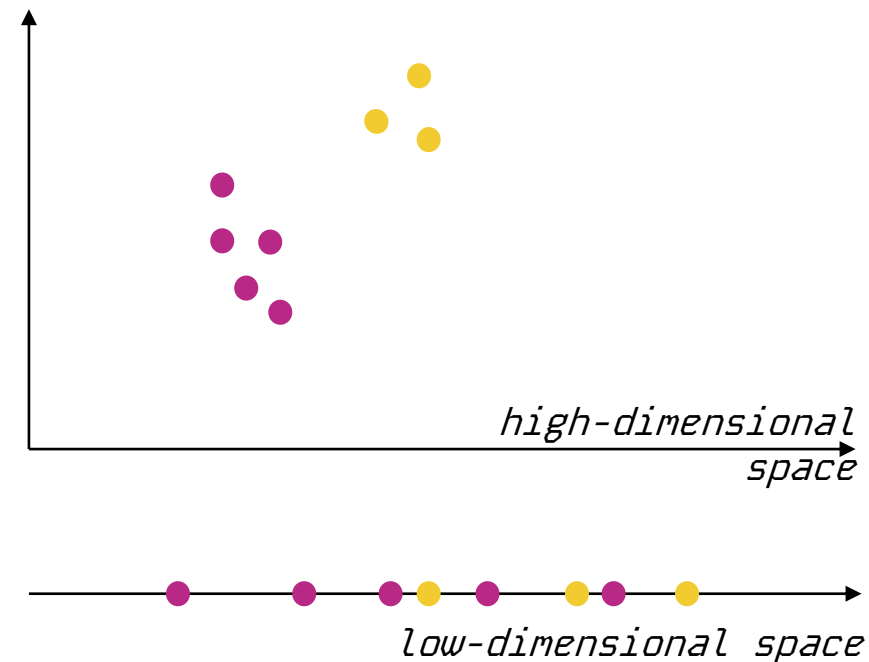
How?

Measure similarity between data points in hi.-dim. and optimize positions in lo.-dim. to attain same (=as close as possible) similarity there

Similarity

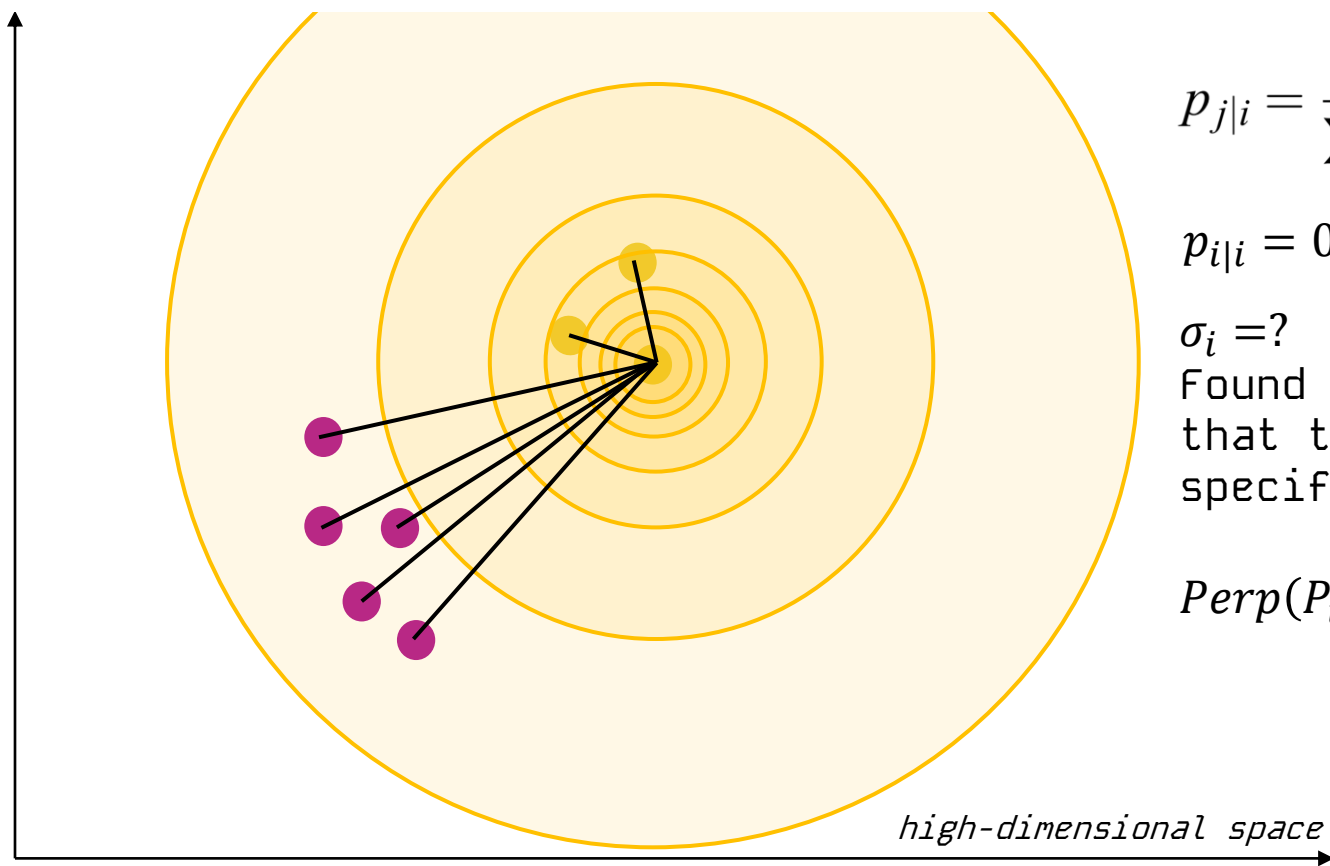
Convert euclidean distances into conditional probabilities

Similarity of x_i to x_j =
conditional probability that x_i would pick x_j as its neighbor
if neighbors are picked in proportion to their probability
density under a Gaussian centered at x_i



Similarity in high-dimensional space

Similarity of x_i to x_j = conditional probability that x_i would pick x_j as its neighbor *if neighbors are picked in proportion to their probability density under a Gaussian centered at x_i*



$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)} \quad \text{—— Gaussian pdf*}$$

—— normalization

$$p_{i|i} = 0 \quad \text{and} \quad p_{ij} = p_{ji} = \frac{p_{i|j} + p_{j|i}}{2}$$

$\sigma_i = ?$

Found with binary search for each data point such that the distribution P_i is associated with a specified *perplexity* (typically between 5 and 50)

$$\text{Perp}(P_i) = 2^{H(P_i)} \quad \text{where } H(P_i) \text{ is the Shannon entropy}$$

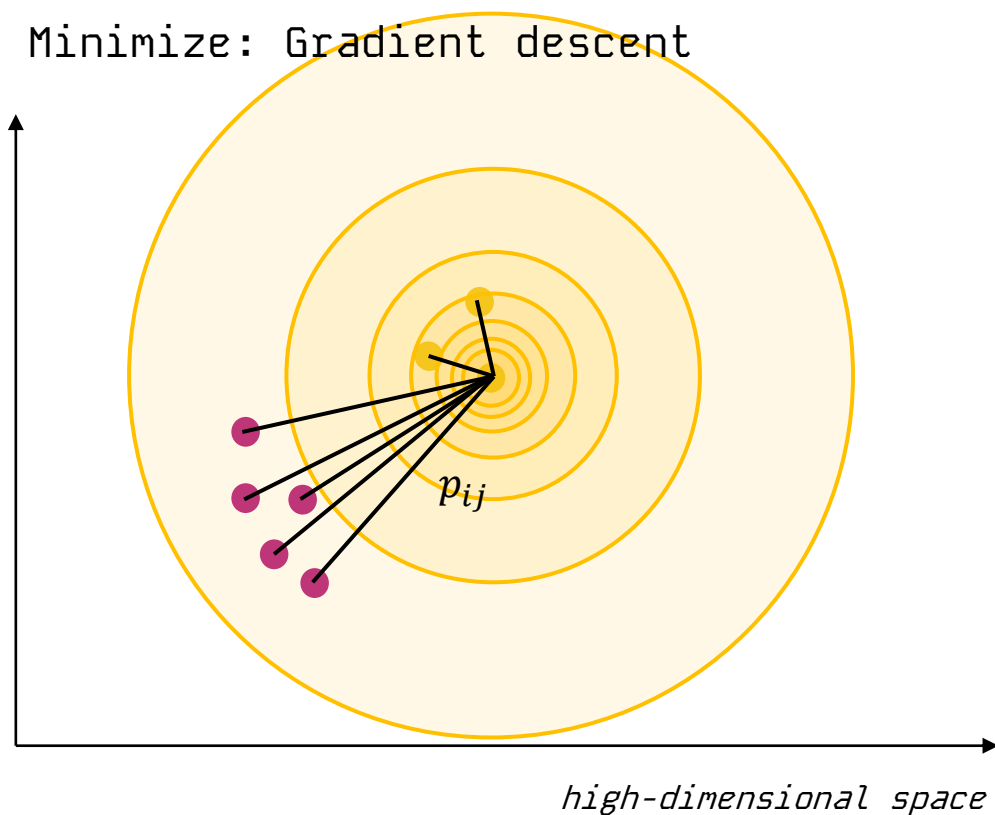
$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

Similarity

Probabilities in the high-dimensional and low-dimensional spaces should be similar

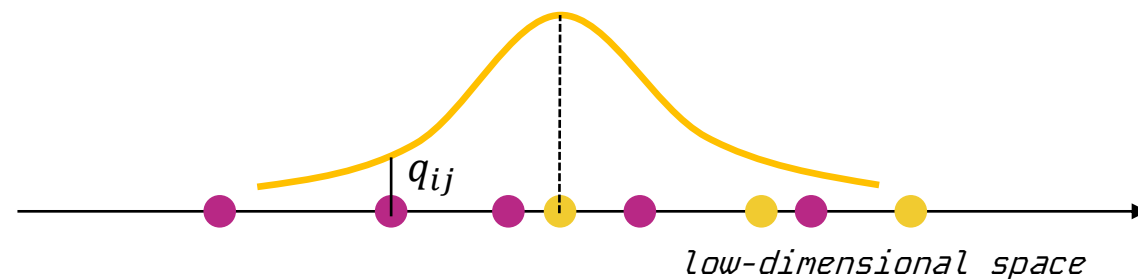
>>> Measure of distance between probability distribution
Kullback-Leibler (KL) Divergence

>>> Minimize: Gradient descent



$$C = KL(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

$$\frac{\delta C}{\delta y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1}$$



In t-SNE, perplexity is a knob that sets the *effective number of nearest neighbors* each point pays attention to. Mathematically, for each point i , t-SNE chooses a Gaussian width σ_i so that the conditional neighbor distribution $p_{j|i}$ has Shannon entropy $H(P_i)$ with

$$\text{perplexity} = 2^{H(P_i)}.$$

Bigger perplexity $\Rightarrow$ each point considers more neighbors (more *global* structure).

Smaller perplexity $\Rightarrow$ focuses on fewer neighbors (more *local* structure).

How to pick it

Works best in the 5–50 range for many problems.

Let dataset size guide you (rough rule of thumb):

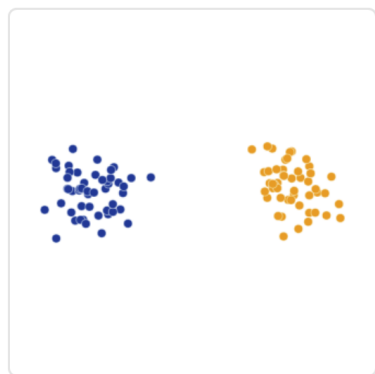
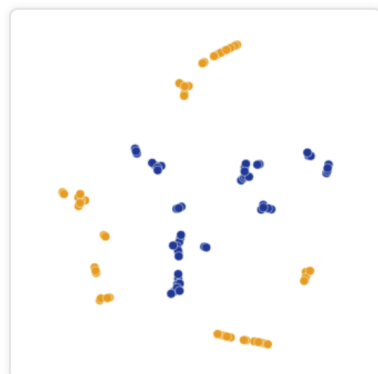
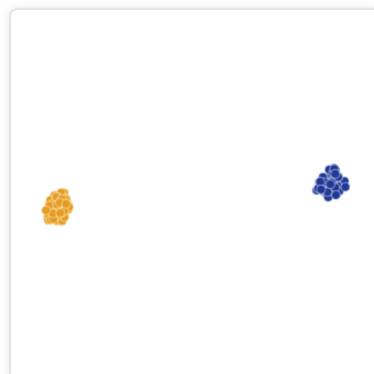
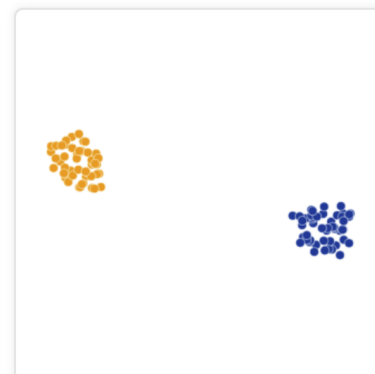
$n \leq 1,000$: 5–30

$1,000 < n \leq 10,000$: 30–50

n very large: you can push higher (e.g., 50–100), but try several values.

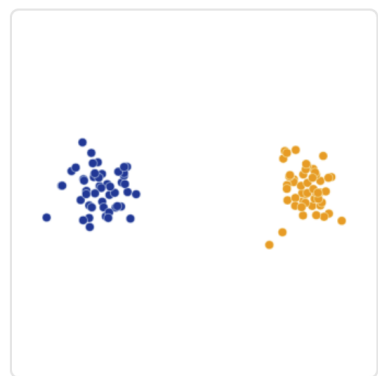
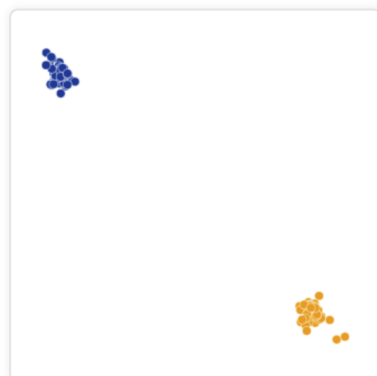
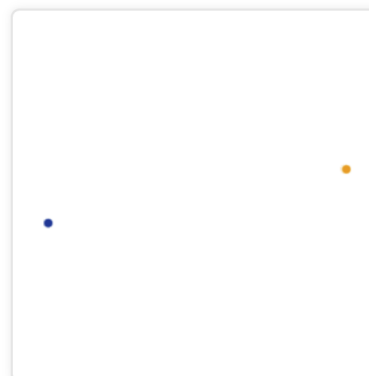
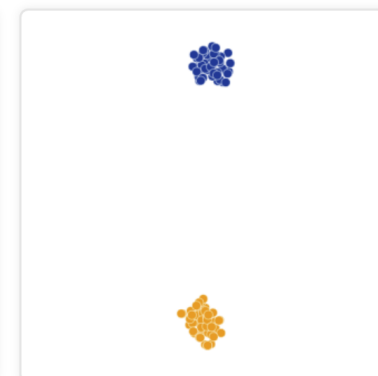
Always try a few settings and compare stability (t-SNE is stochastic).

Influence of perplexity

*Original*Perplexity: 2
Step: 5,000Perplexity: 5
Step: 5,000Perplexity: 30
Step: 5,000Perplexity: 50
Step: 5,000Perplexity: 100
Step: 5,000

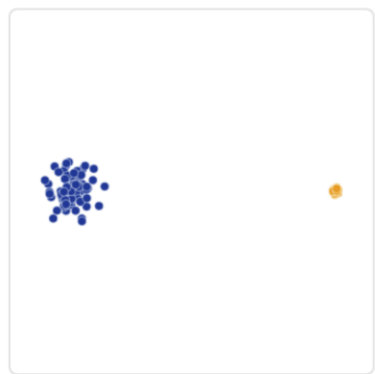
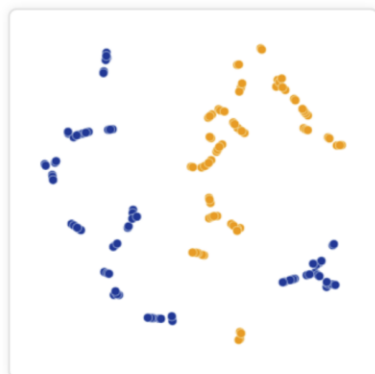
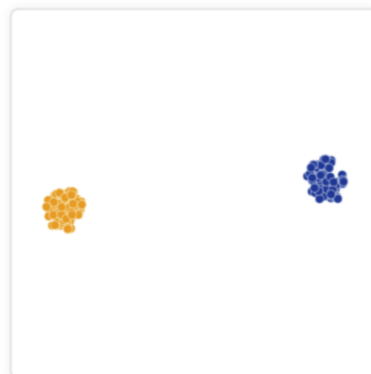
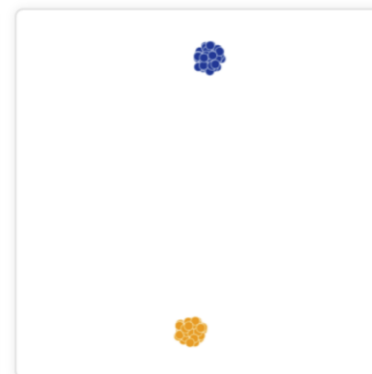
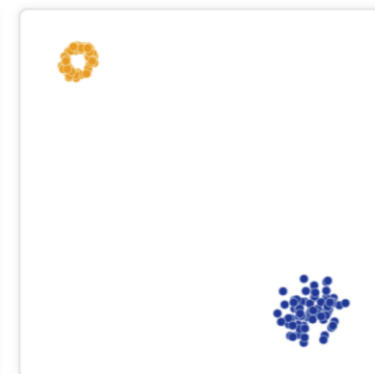
Understanding t-SNE
Wattenberg et al., 2016

Influence of number of iterations

*Original*Perplexity: 30
Step: 10Perplexity: 30
Step: 20Perplexity: 30
Step: 60Perplexity: 30
Step: 120Perplexity: 30
Step: 1,000

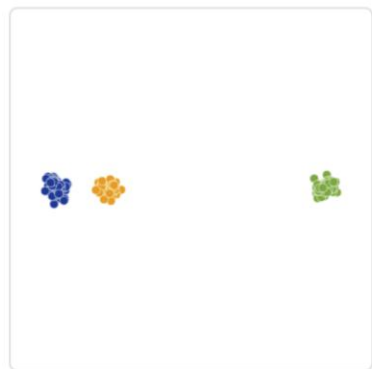
Understanding t-SNE
Wattenberg et al., 2016

Cluster size means nothing

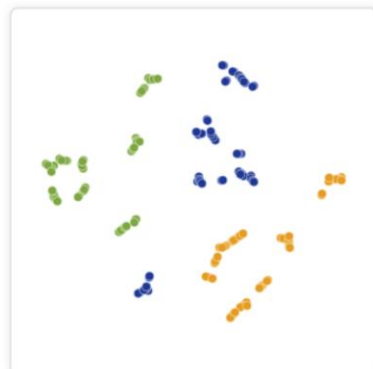
*Original*Perplexity: 2
Step: 5,000Perplexity: 5
Step: 5,000Perplexity: 30
Step: 5,000Perplexity: 50
Step: 5,000Perplexity: 100
Step: 5,000

Understanding t-SNE
Wattenberg et al., 2016

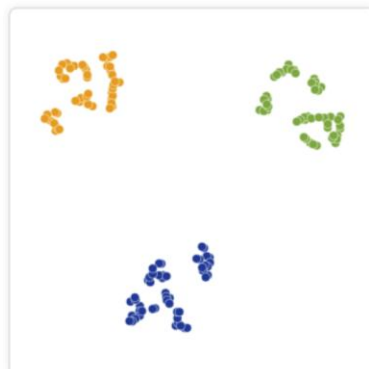
Distance may not mean anything



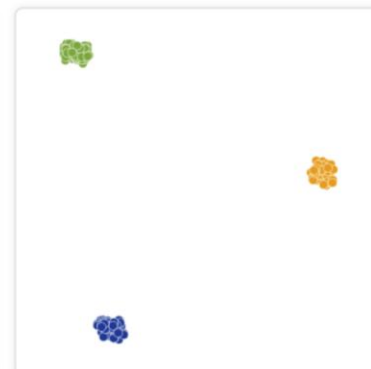
Original



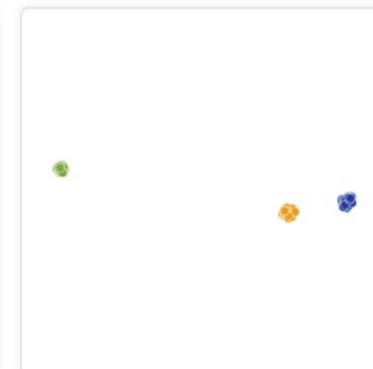
Perplexity: 2
Step: 5,000



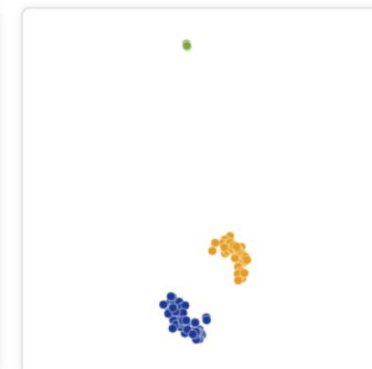
Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000



Perplexity: 50
Step: 5,000



Perplexity: 100
Step: 5,000

Understanding t-SNE
Wattenberg et al., 2016

a manifold is a shape (a space) where the data *really* lives that can be curved and embedded in a higher-dimensional space, but locally it looks flat (like ordinary Euclidean space).

Intuition

Think of a sheet of paper (2D) that's rolled into a Swiss-roll in 3D. Although it sits in 3D, any tiny patch still behaves like a flat 2D plane.

Many real datasets aren't spread through all dimensions; they lie near a lower-dimensional, smoothly varying surface inside the high-dimensional feature space.

Formal (lightweight) idea

A (topological) manifold of dimension d is a space where every point has a neighborhood that's equivalent to ordinary $\mathbb{R}^d$. So: “locally flat, globally possibly curved.”

Why t-SNE “reveals” it

t-SNE preserves local neighborhoods: if your data lies on/near a low-dimensional manifold, the 2D map often uncovers clusters/curves that reflect that underlying structure (e.g., the unrolled Swiss roll, digit families in MNIST). It's good at local structure, not exact global geometry – so distances between far-apart clusters aren't very meaningful.

For building/energy setting

High-dim vectors like [EUI, GFA, year, GHG,...] may actually vary along a few latent factors (e.g., use type, vintage, climate/operation). That implies a low-dimensional manifold inside the larger feature space. t-SNE can reveal clusters/trajectories corresponding to, say, older small multifamily → renovated → newer large office progression.

t-SNE *sometimes* reveal the shape of the underlying manifold



Original

50-dim asymmetric
Gaussian
projected on x_1
and x_2



Perplexity: 2
Step: 5,000



Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000



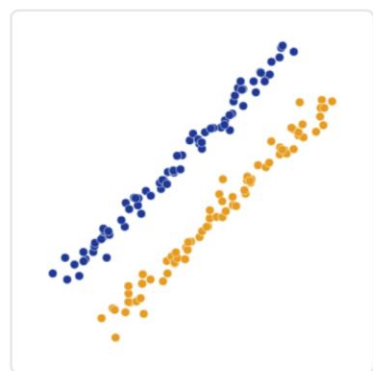
Perplexity: 50
Step: 5,000



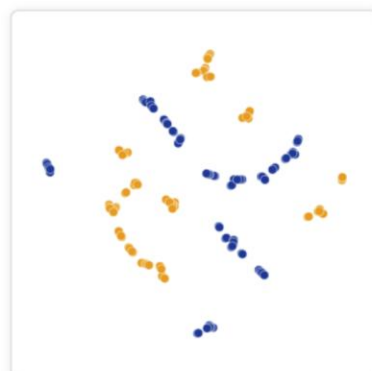
Perplexity: 100
Step: 5,000

Understanding t-SNE
Wattenberg et al., 2016

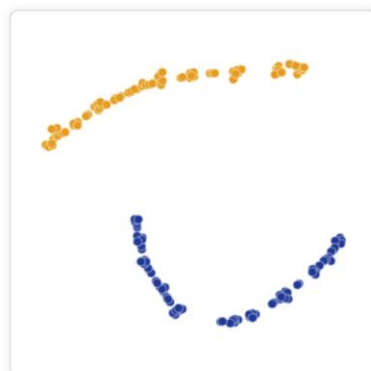
t-SNE *sometimes* reveal the shape of the underlying manifold



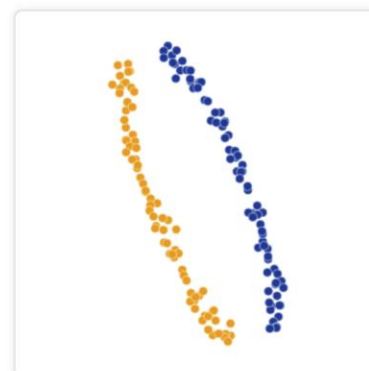
Original



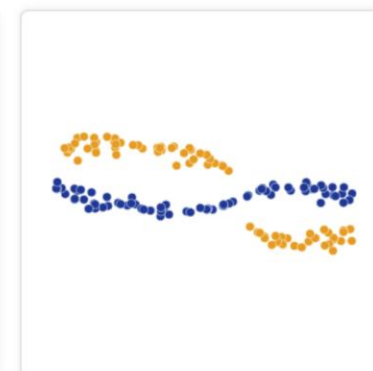
Perplexity: 2
Step: 5,000



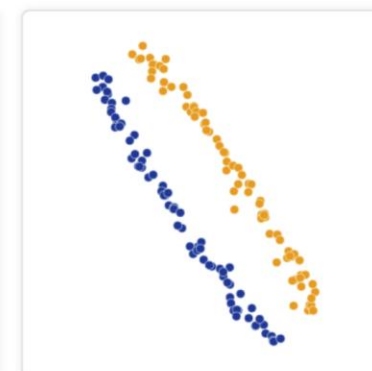
Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000



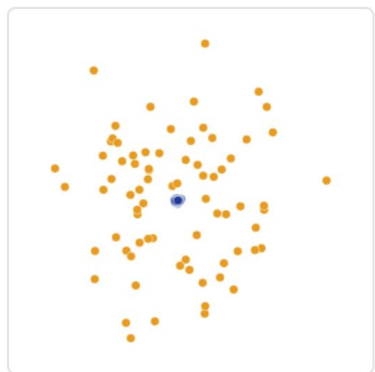
Perplexity: 50
Step: 5,000



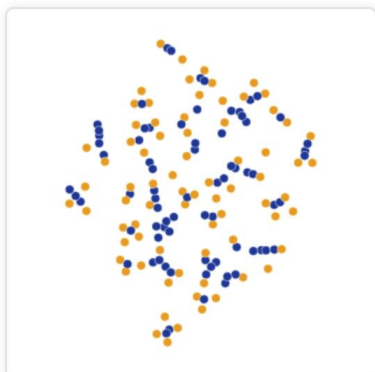
Perplexity: 100
Step: 5,000

Understanding t-SNE
Wattenberg et al., 2016

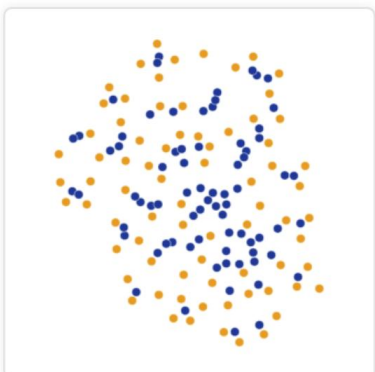
Producing multiple t-SNE plots may help to understand the underlying topology of the dataset



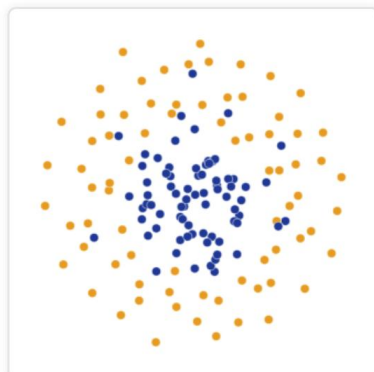
Original



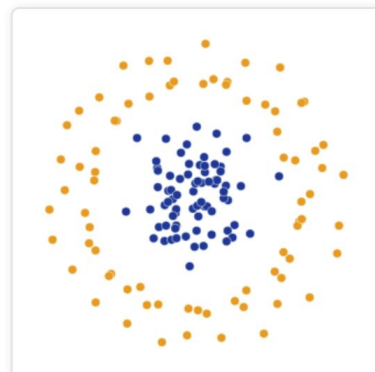
Perplexity: 2
Step: 5,000



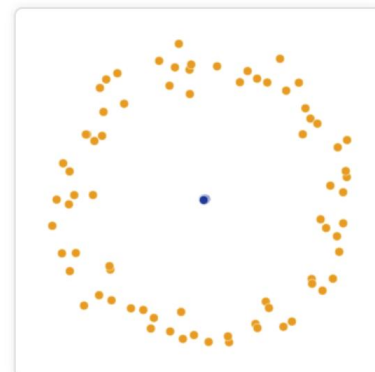
Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000



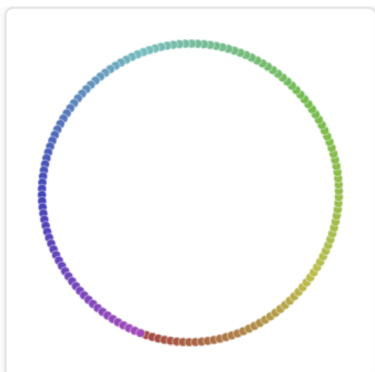
Perplexity: 50
Step: 5,000



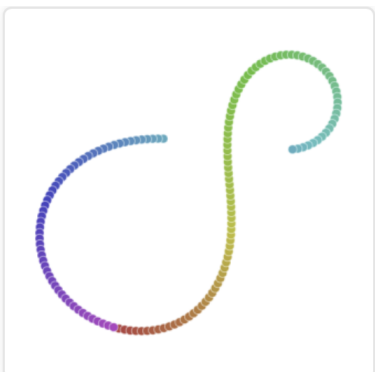
Perplexity: 100
Step: 5,000



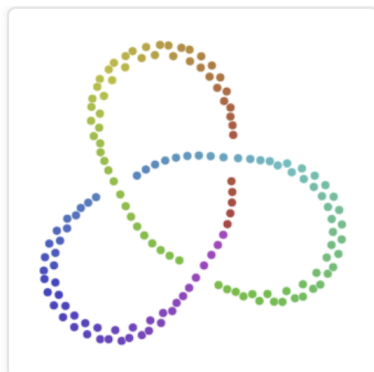
Original



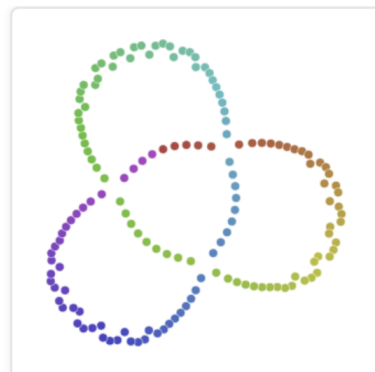
Perplexity: 2
Step: 5,000



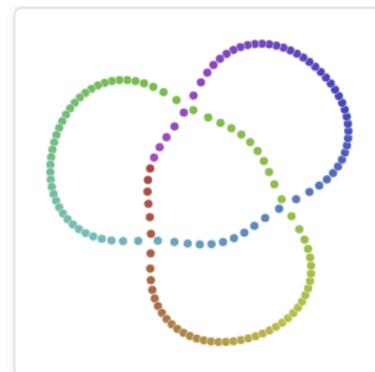
Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000

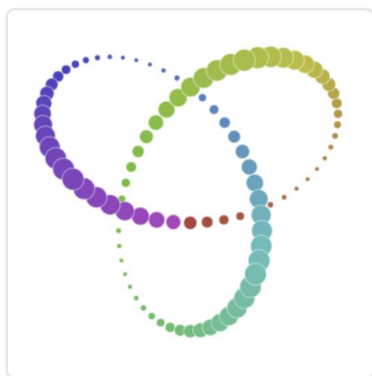


Perplexity: 50
Step: 5,000

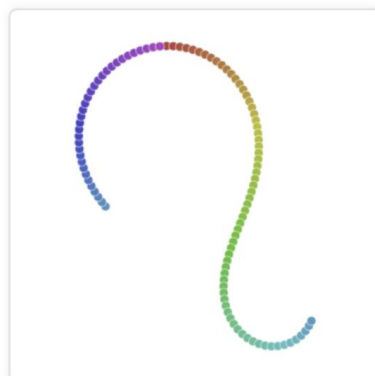


Perplexity: 100
Step: 5,000

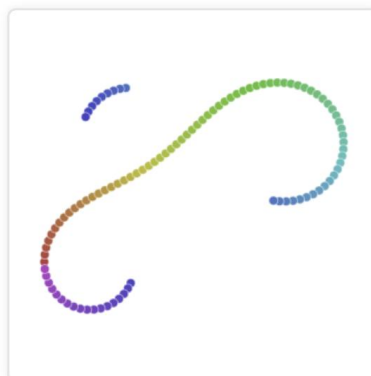
t-SNE is stochastic, and multiple runs may yield different results



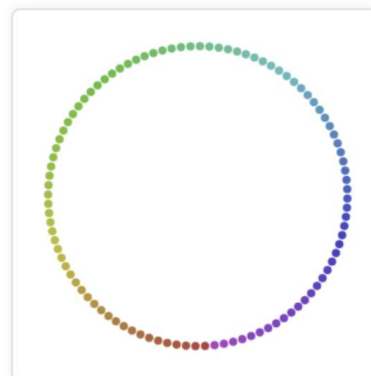
Original



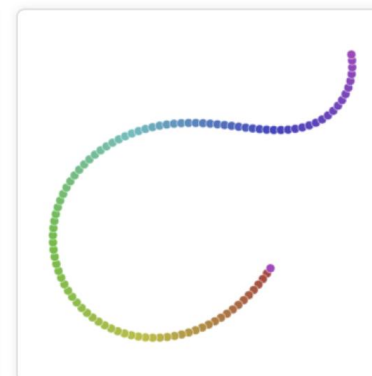
Perplexity: 2
Step: 5,000



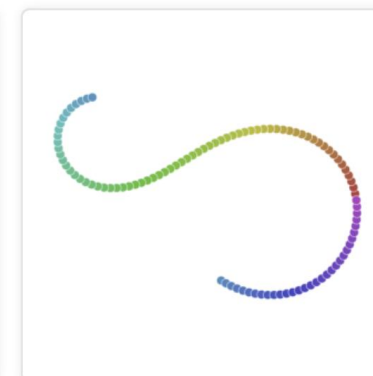
Perplexity: 2
Step: 5,000



Perplexity: 2
Step: 5,000



Perplexity: 2
Step: 5,000



Perplexity: 2
Step: 5,000

Understanding t-SNE
Wattenberg et al., 2016

<https://bit.ly/TSNE-Demo>

`https://https://projector.
tensorflow.org/`

L08.1

Unsupervised

Clustering

KMeans

Hierarchical

Principal Component Analysis

t-SNE

L08.2

Supervised 7

Convolutional Neural Networks

Saliency Maps

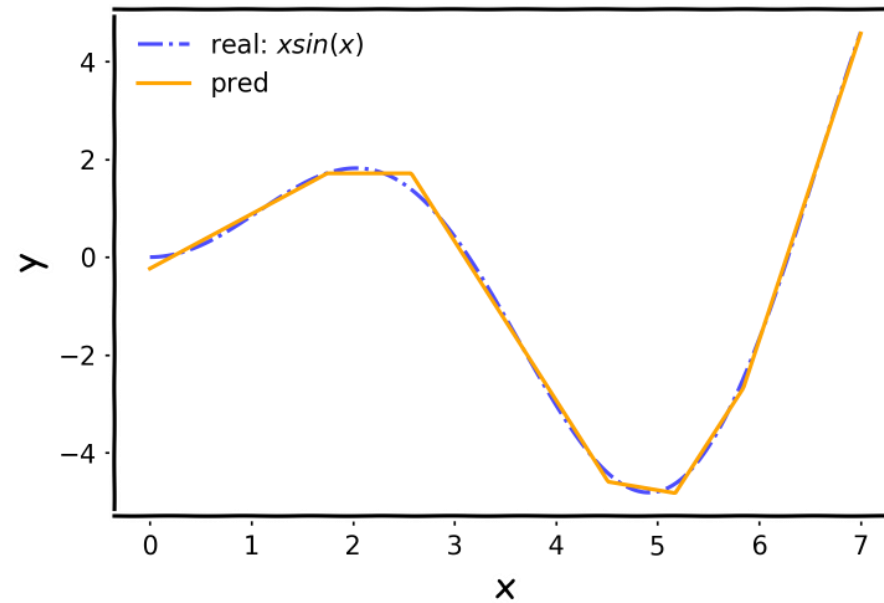
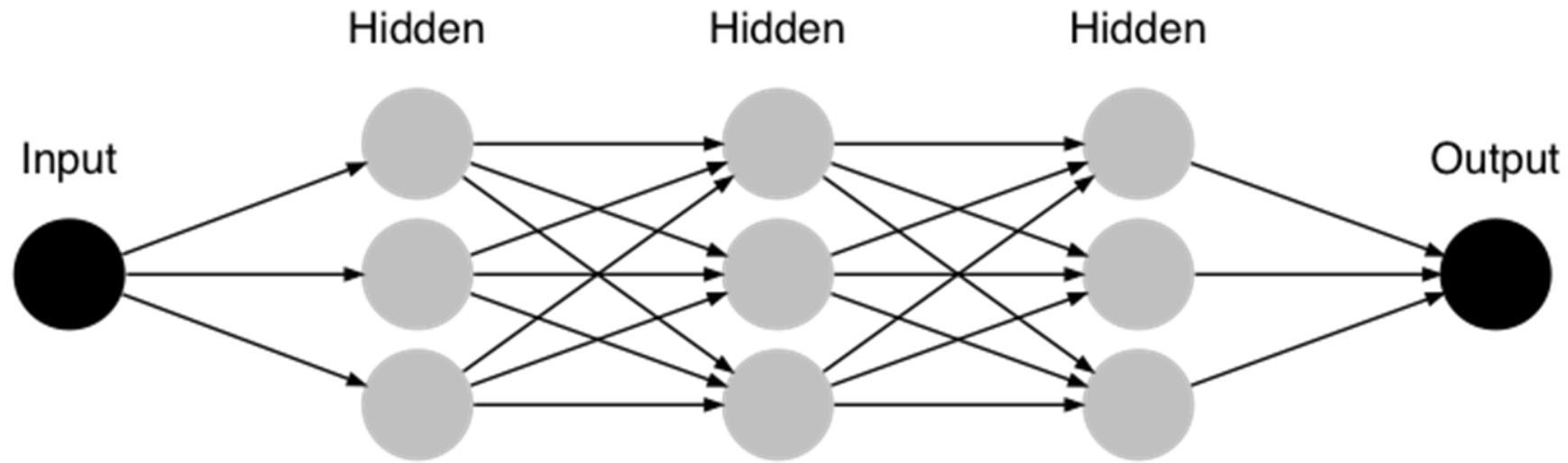
Transfer Learning

We assume that the response variable, Y , relates to the predictors, X , through some unknown function expressed generally as:

$$Y = f(X) + \varepsilon$$

Here, f is the unknown function expressing an underlying rule for relating Y to X , ε is the random amount (unrelated to X) that Y differs from the rule $f(X)$.

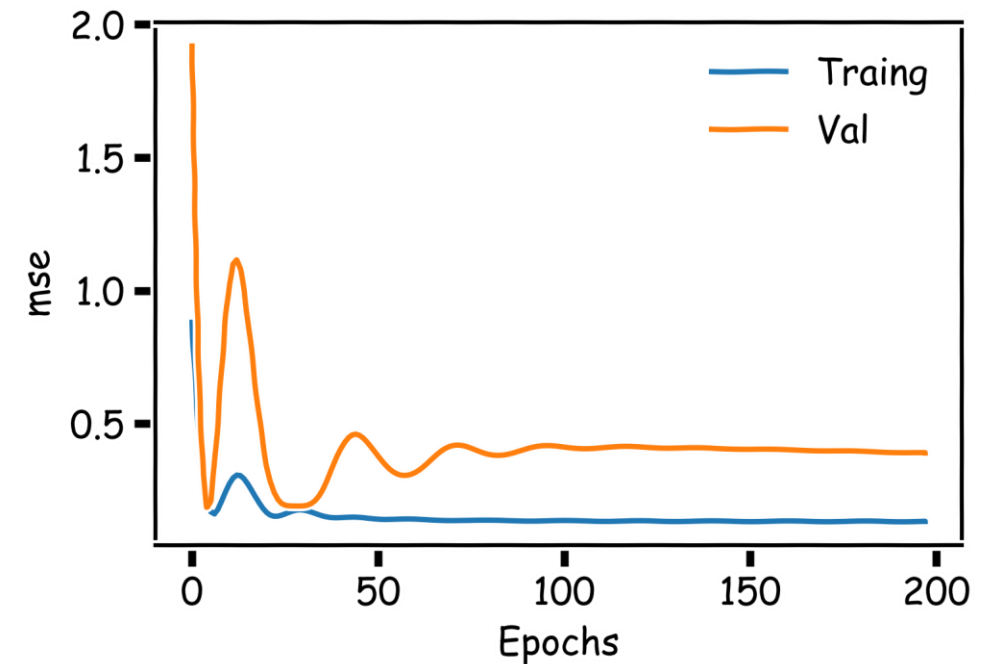
A *statistical model* is any algorithm that estimates f . We denote the estimated function as $\hat{f}$.



If we add more neurons, it clearly overfits.

Regularization of NN

- Norm Penalties
- Early Stopping
- Data Augmentation
- Sparse Representation
- Dropout



MLPs use one node for each input (e.g. pixel in an image, multiplied by 3 in RGB case). The amount of weights **rapidly becomes unmanageable** for large images.

Training difficulties arise, overfitting can appear.

MLPs react differently to an input (images) and its shifted version – **they are not translation invariant**.

MLP: Number of weights for images

Weights Explosion

Example: CIFAR10

Simple 32x32 color images
(3 channels)

Each pixel is a feature: an
MLP would have $32 \times 32 \times 3 + 1 =$
3073 weights per neuron!



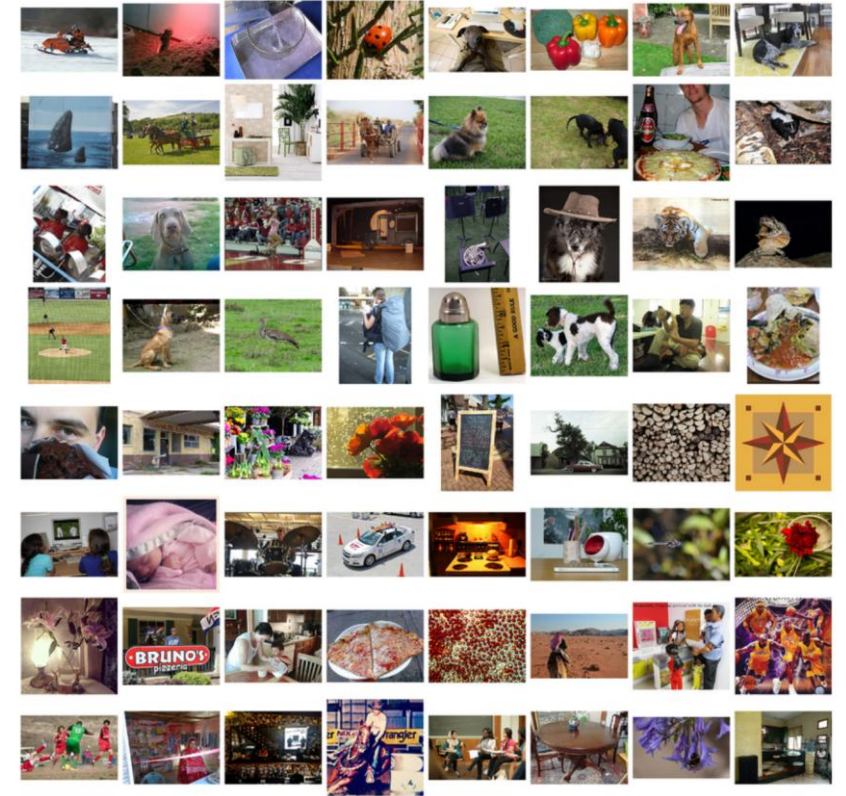
MLP: Number of weights for images

Weights Explosion

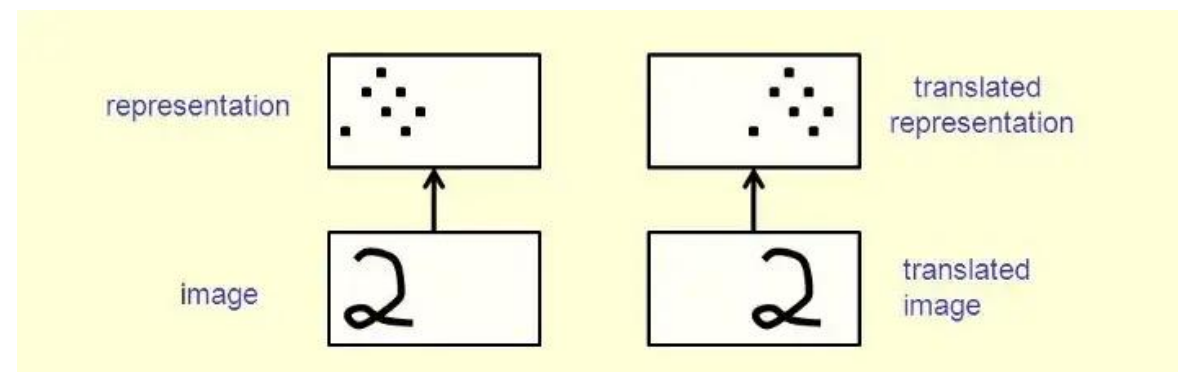
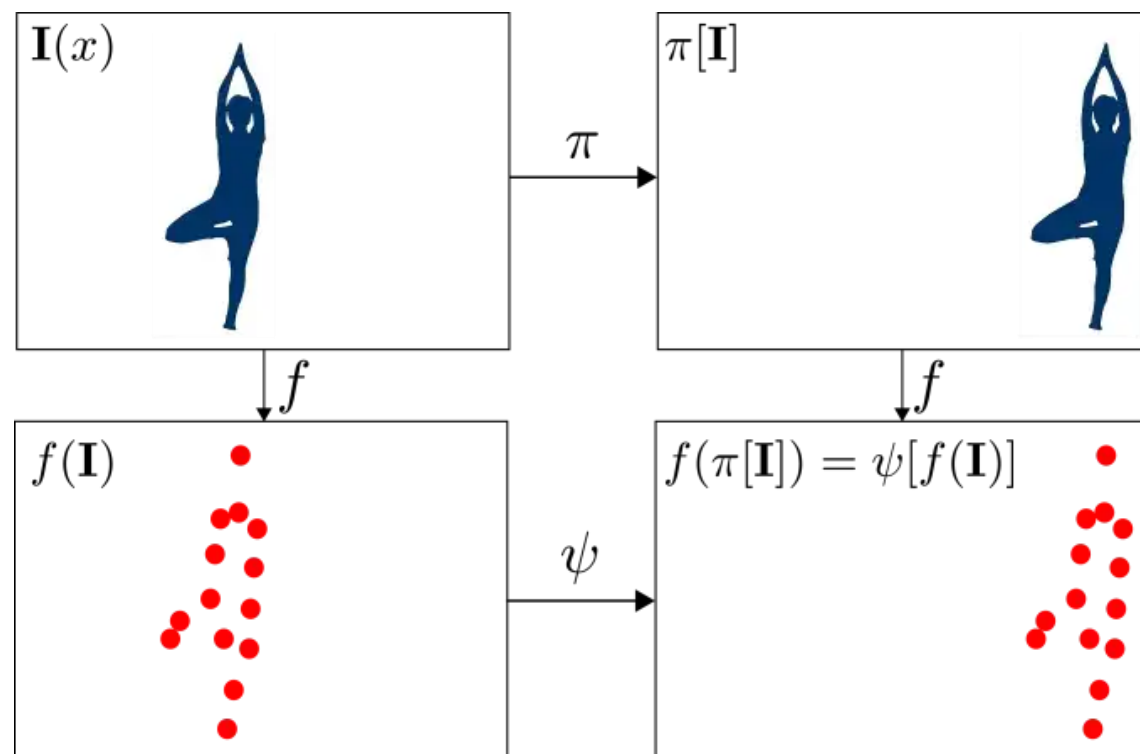
Example: ImageNet

Images are usually $224 \times 224 \times 3$: an MLP would have **150129 weights per neuron**. If the first layer of the MLP is around 128 nodes, which is small, this already becomes very heavy to calculate.

Model complexity is extremely high:
overfitting.



(Not) Translation Invariant



Imagine that we want to recognize swans in an image:

Oval-shaped
white blob
(body)



Round,
elongated oval
with orange
protuberance

Long white
rectangular
shape (neck)

Cases can be a bit more complex

Round,
elongated
head with
orange or
black beak

Long white
neck, square
shape

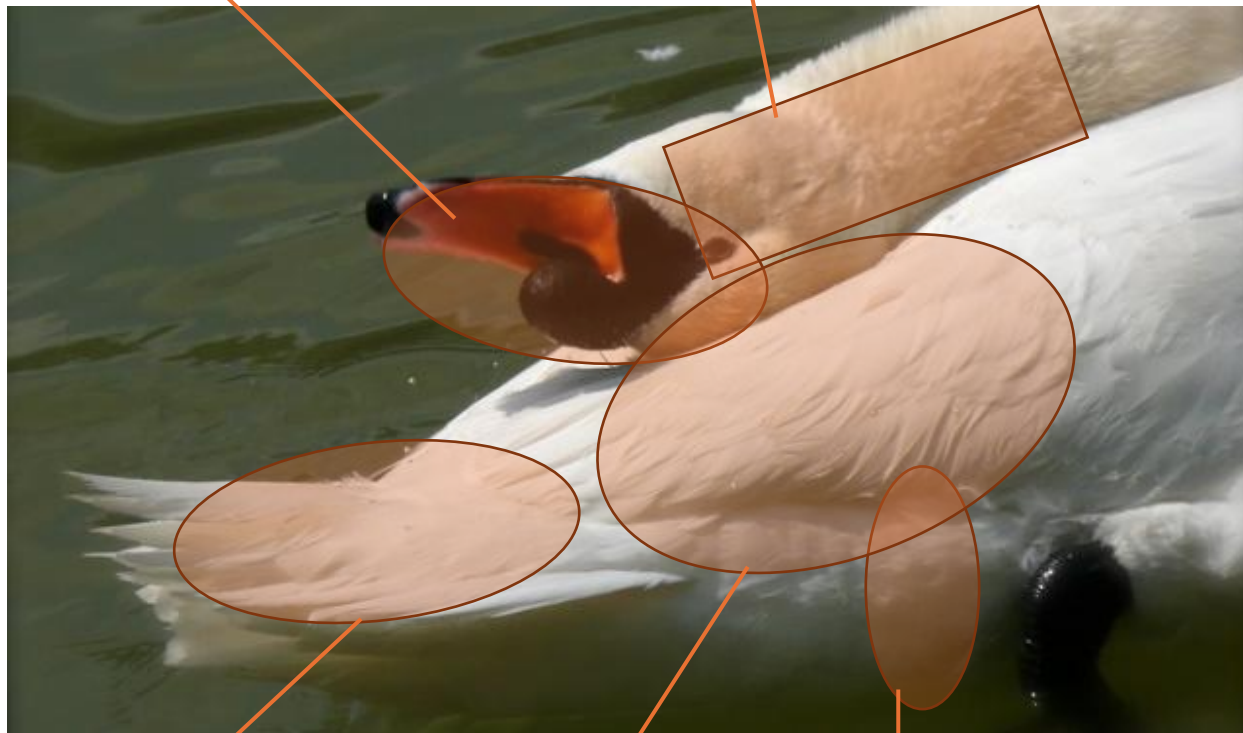


Oval-shaped
white body
with or
without large
white
symmetric
blobs (wings)

Cases can be a bit more complex

Round, elongated head with orange or black beak, can be turned backwards

Long white neck, can bend around, not necessarily straight



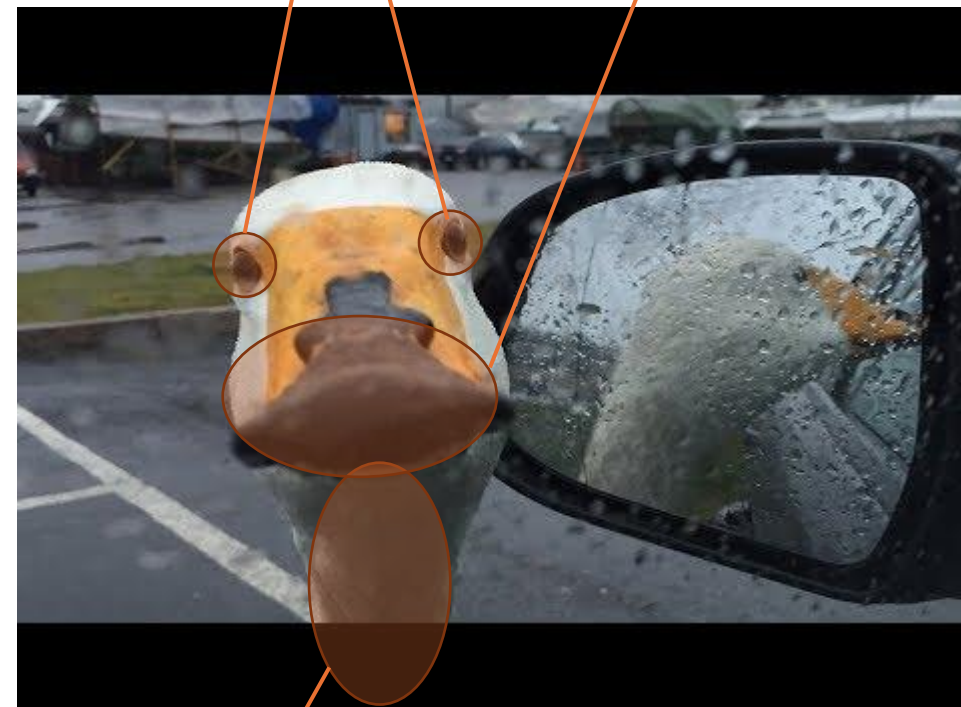
White tail, generally far from the head, looks feathery

White, oval shaped body, with or without wings visible

Black feet, under body, can have different shapes

Small black circles, can be facing the camera, sometimes can see both

Black triangular shaped form, on the head, can have different sizes



White elongated piece, can be squared or more triangular, can be obstructed sometimes

UNSUPERVISED

SUPERVISED 7

SIMULATIONS

Can we handle these?



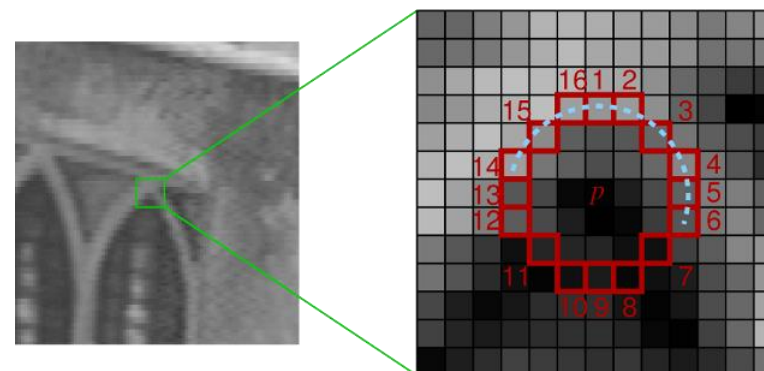
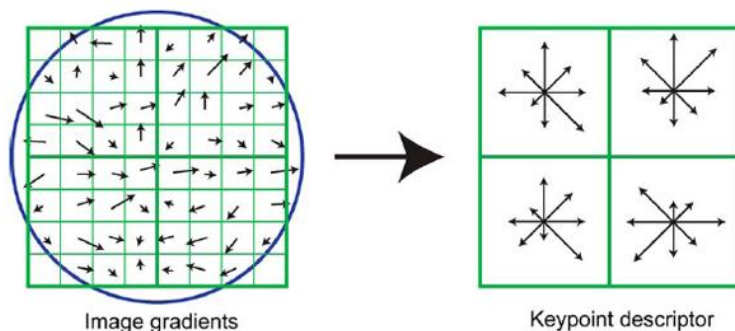
Man in swan tent photographing swans

We've been basically talking about **detecting features in images**, in a very naïve way.

Researchers built multiple computer vision techniques to deal with these issues: SIFT, FAST, SURF, BRIEF, etc.

However, similar problems arose: the detectors where either too general or too over-engineered. Humans were designing these feature detectors, and that made them either too simple or hard to generalize

SIFT feature descriptor



FAST corner detection algorithm

What if we **learned the features to detect**?

We need a system that can do Representation Learning (or Feature Learning)

Representation Learning: technique that allows a system to automatically find relevant features for a given task. Replaces manual feature engineering.

Multiple techniques for this:

Unsupervised (K-means, PCA, ...).

Supervised (Sup. Dictionary learning, **Neural Networks!**)

Image Recognition / Computer Vision Analysis



Nearby pixels are more strongly related than distant ones.

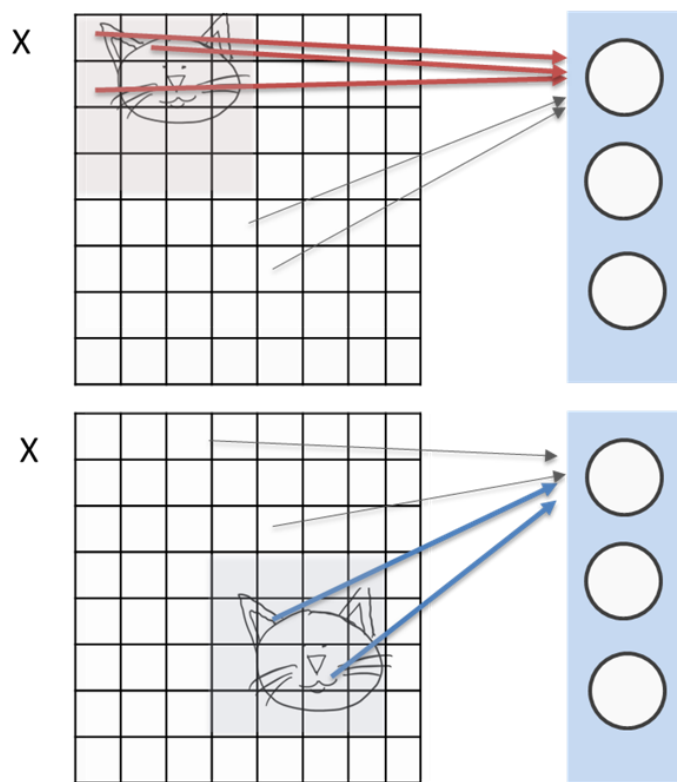
Objects are built up out of smaller parts.

Image Recognition / Computer Vision Analysis



Imagine this:

Each neuron from first layer has one weight per pixel. Recall, the importance of the predictors (here pixels) is given by the value of the coefficient (here W).



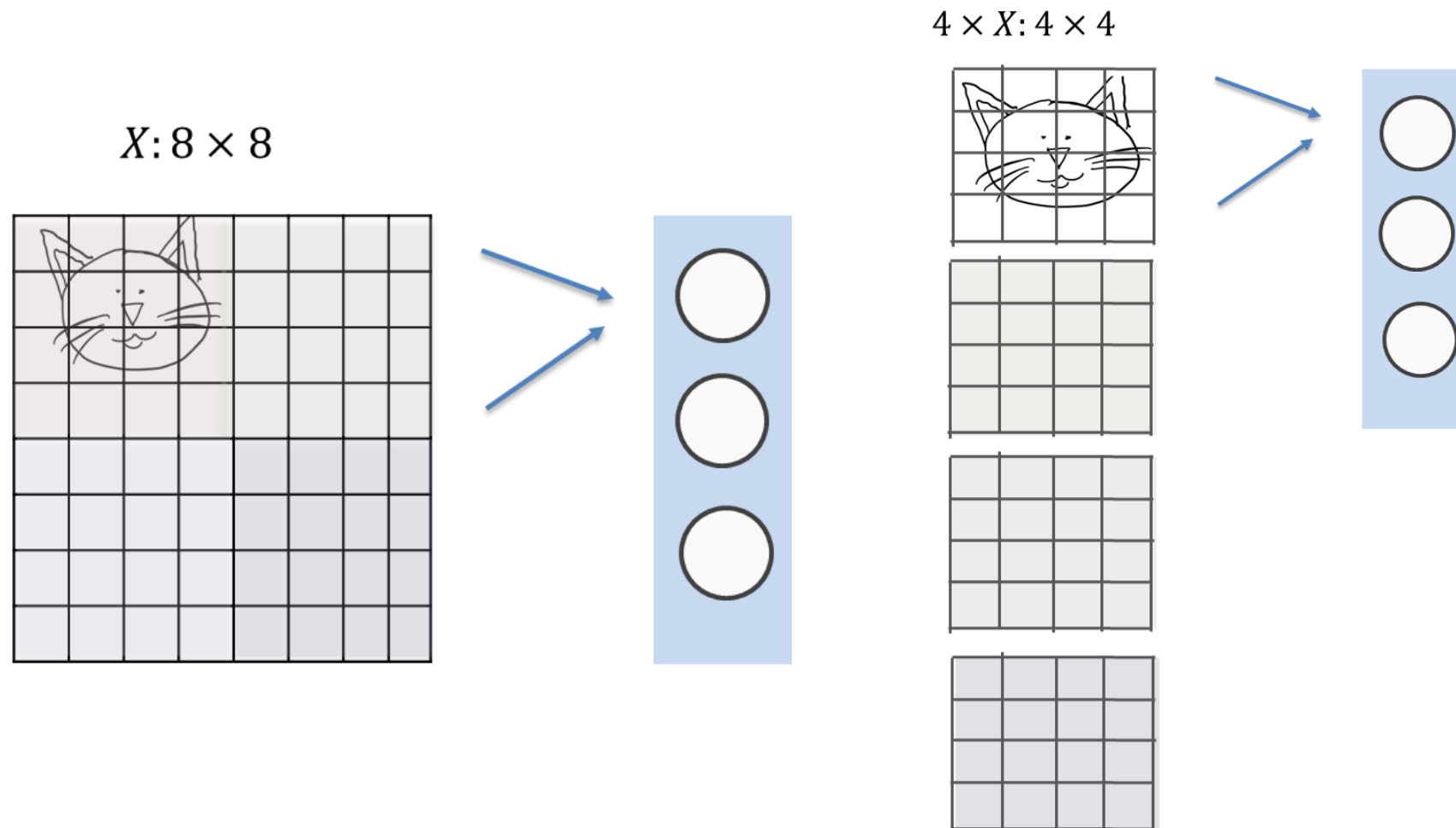
In this case, the **red weights** will be modified to better recognize cat.

We are learning **redundant** features.

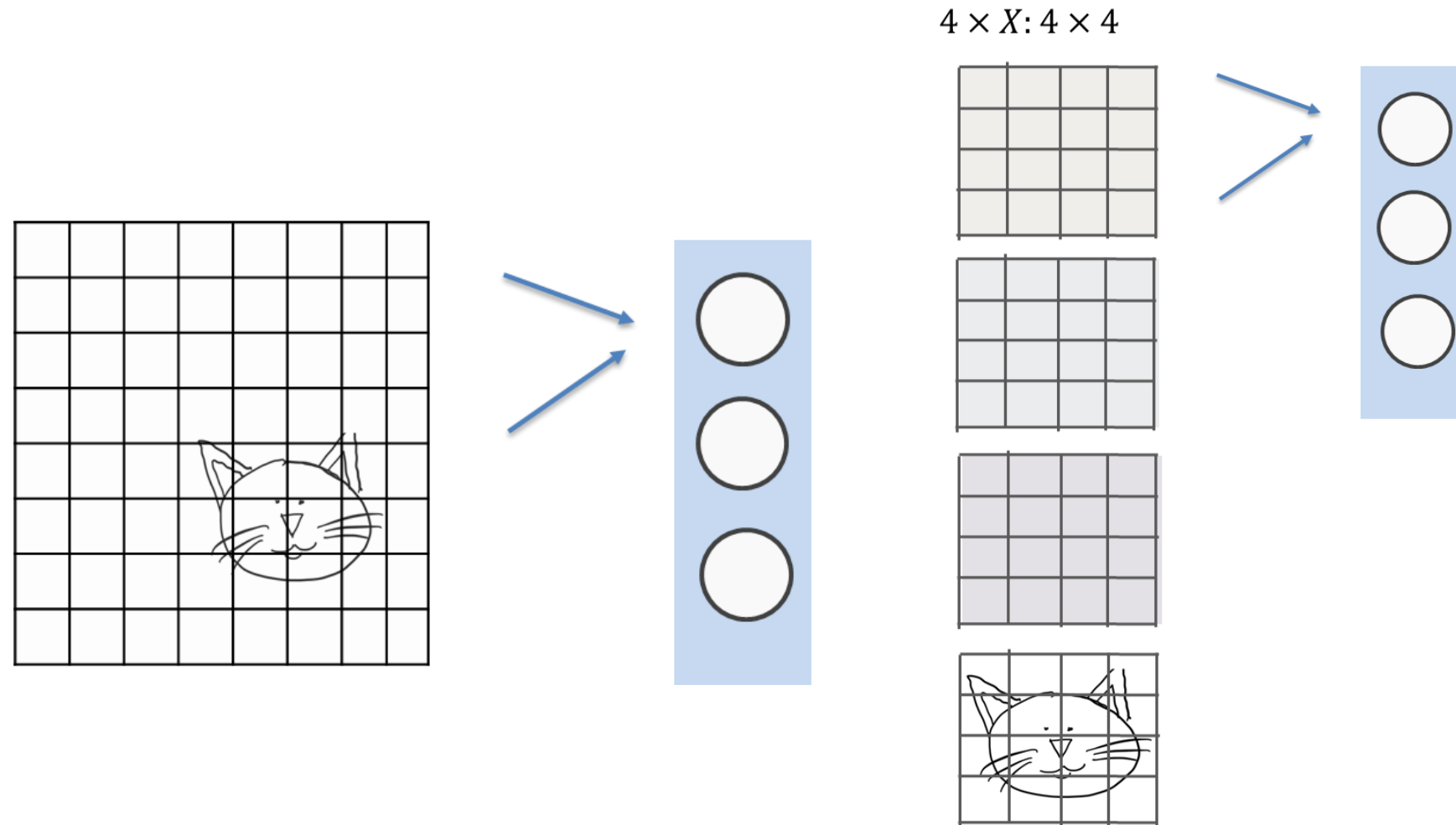
In this case, the **blue weights** will be modified.

Approach is not robust, as cats could appear in yet another position.

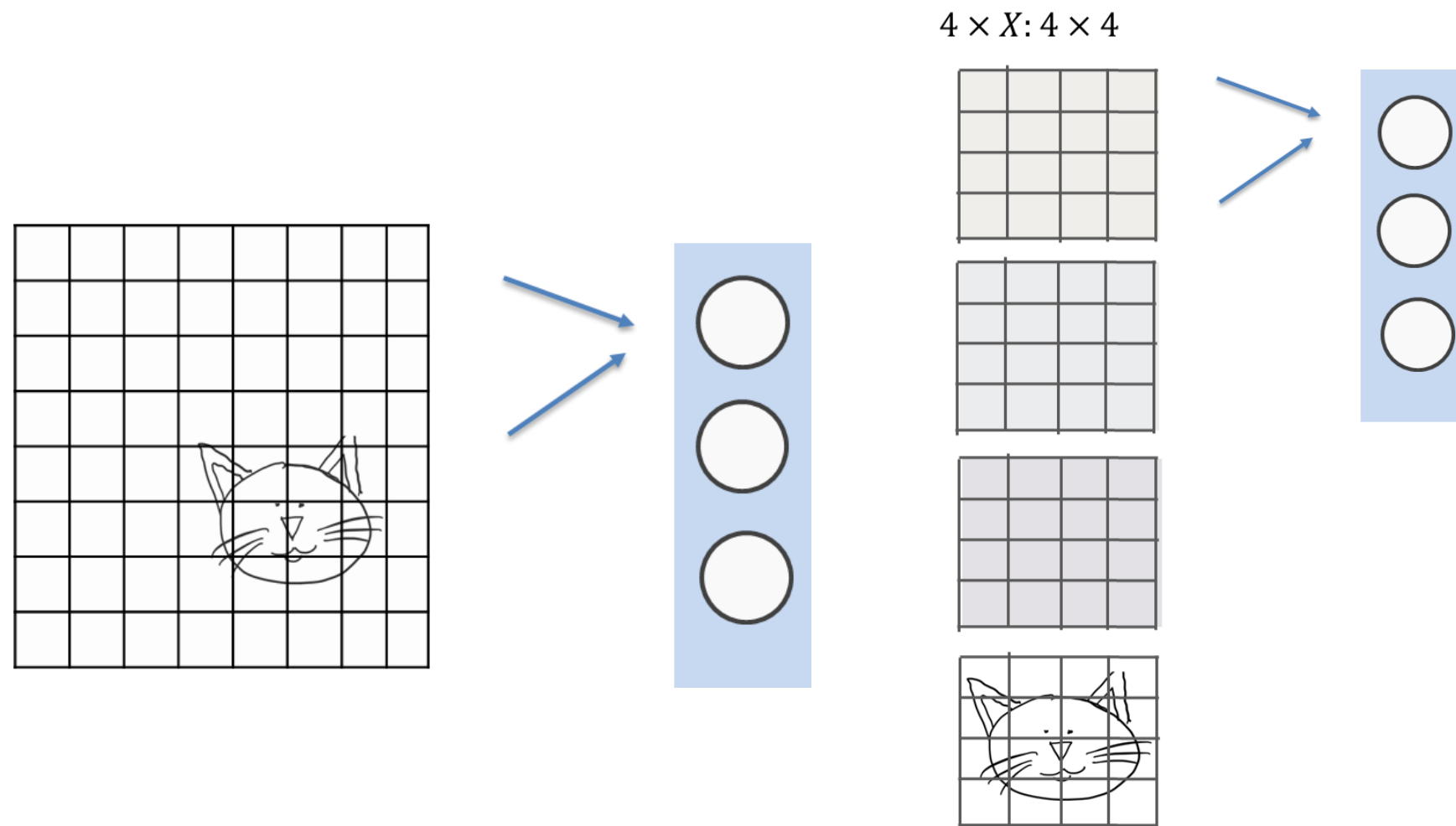
Solution: Let's cut the image into "smaller pieces"



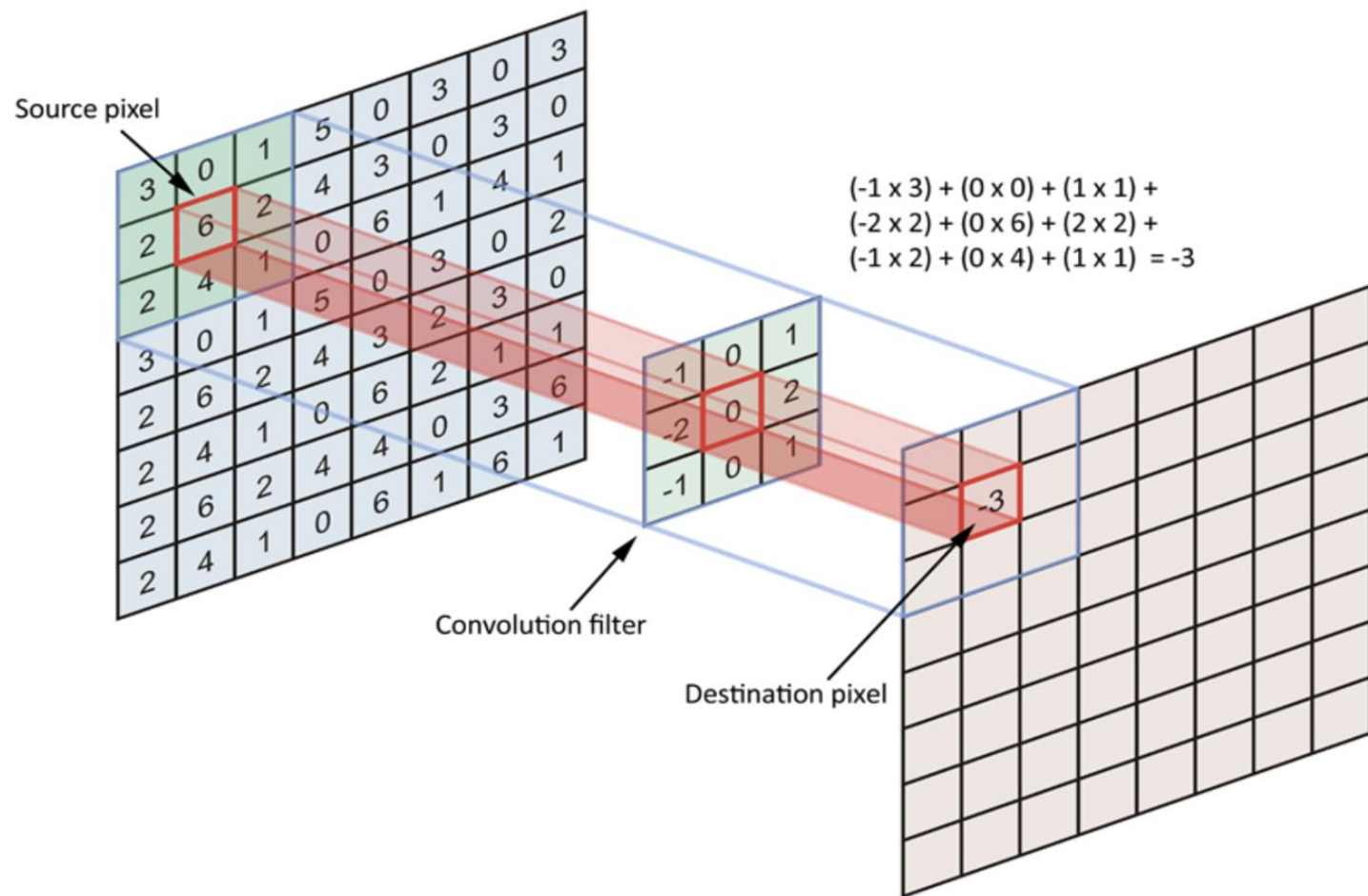
Do the same for all images



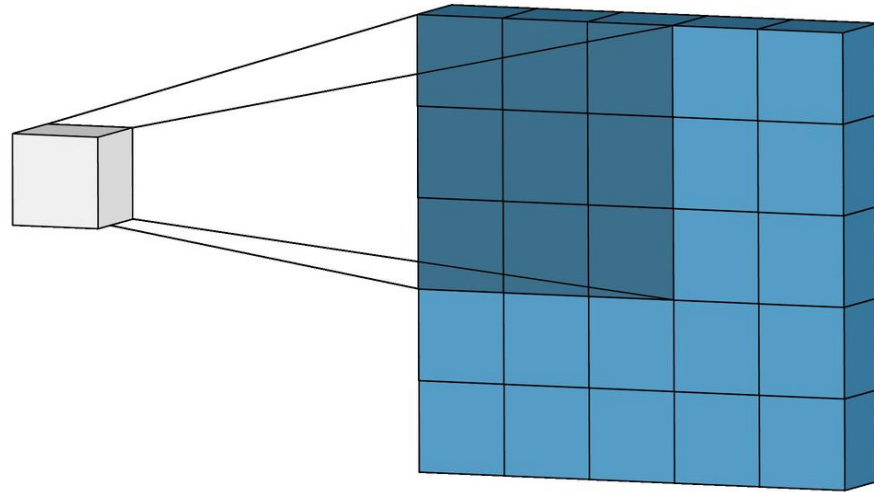
Basic Idea of Convolution



Basic Idea of Convolution



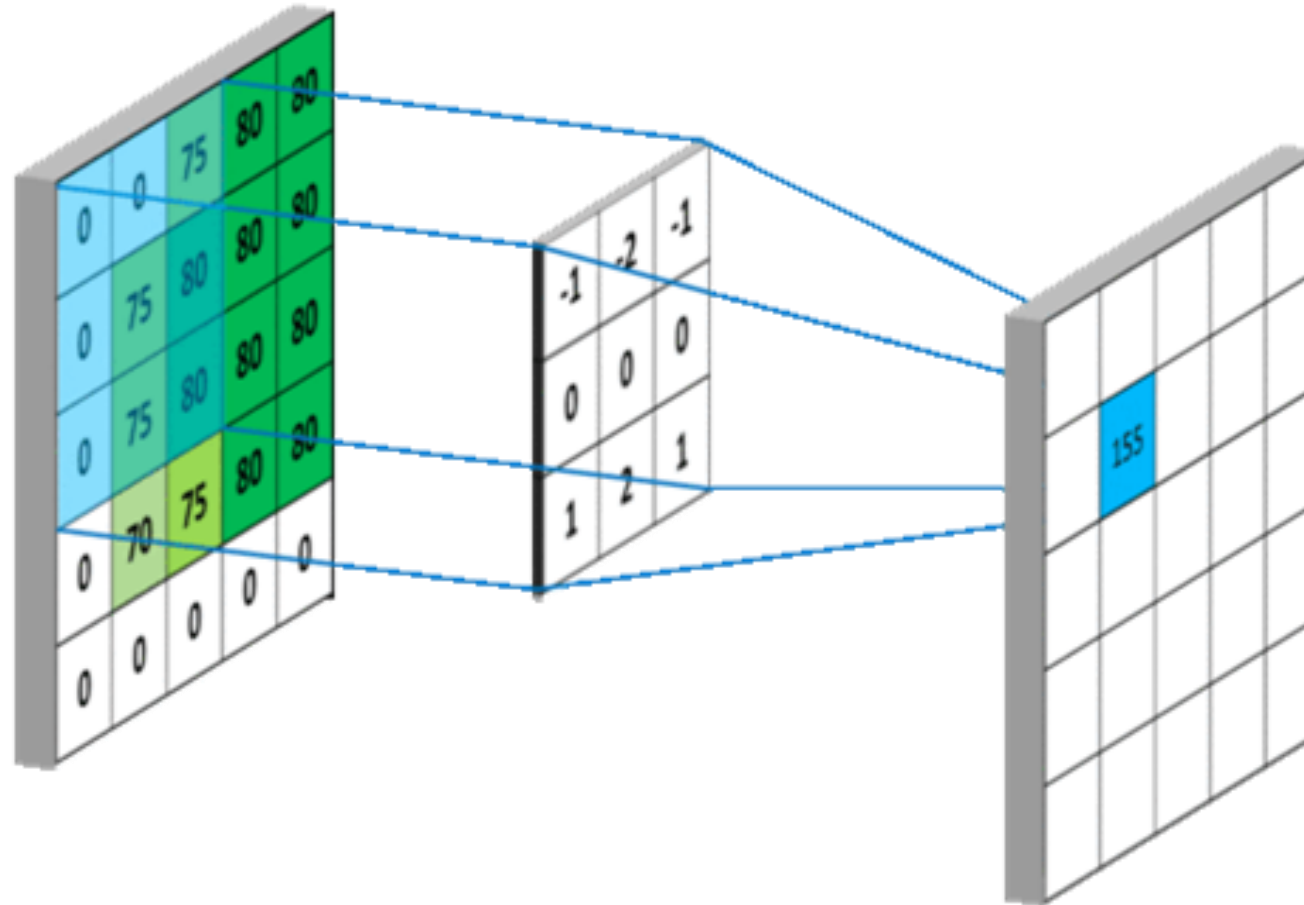
Convolution



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12	12	17
10	17	19
9	6	14

Convolution



“Convolution” Operation in action

Edge detection


$$* \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} =$$


Kernel

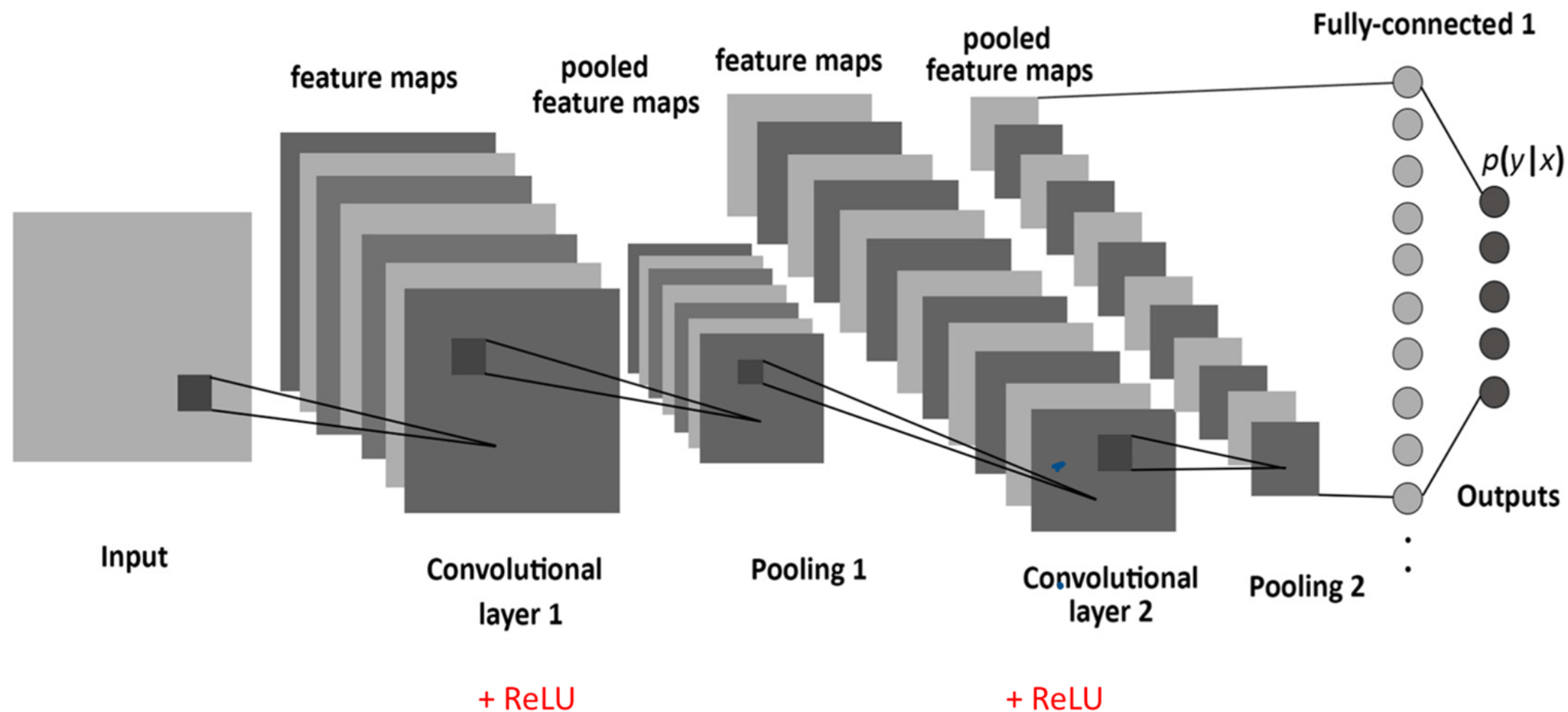
The diagram illustrates the edge detection process. It shows an original image of a squirrel's head being convolved with a 3x3 kernel. The kernel is a Laplacian operator, represented by the matrix: $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$. The result is an edge detection image where only the boundaries of the squirrel are highlighted in white against a black background. An arrow points from the word "Kernel" to the matrix.

Sharpen

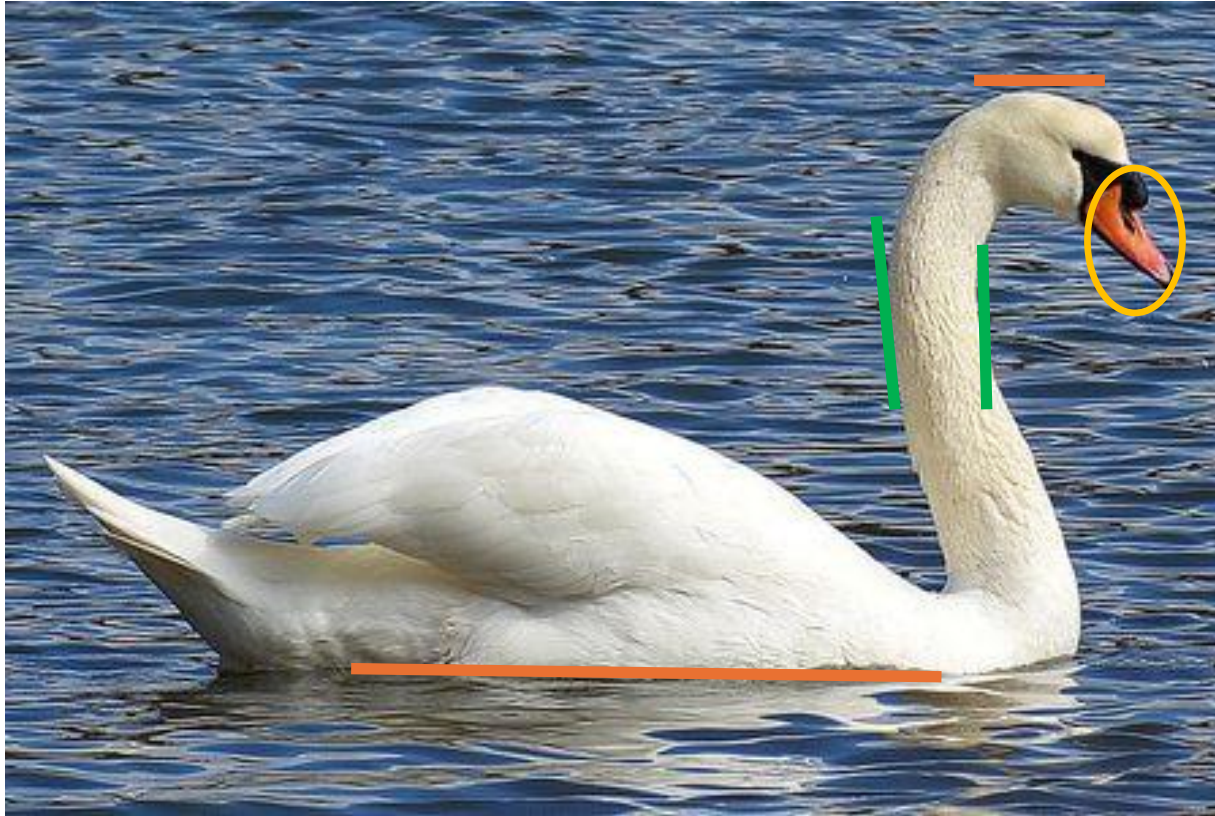

$$* \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} =$$


The diagram illustrates the sharpening process. It shows the same original image of a squirrel's head being convolved with a different 3x3 kernel. The kernel is a sharpening operator, represented by the matrix: $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$. The result is a sharpened image where the edges of the squirrel are more pronounced and the overall image appears crisper.

A Convolutional Network



Why many feature maps?



Feature 1: Horizontal Lines

Feature 2: Vertical Lines

Feature 3: Orange bulb

Why many feature maps?



Layer 2, Feature 1: Combine horizontal and vertical lines from Layer 1 produce diagonal lines.

Layer 3, Feature 1: Combine diagonal to identify shapes

We know that MLPs:

Do not scale well for images

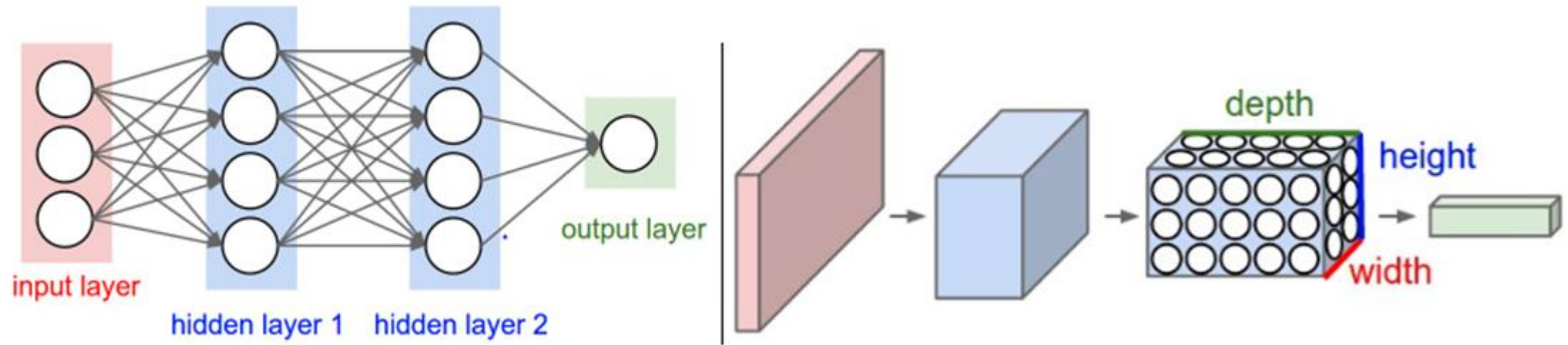
Ignore the information brought by pixel position and correlation with neighbors

Cannot handle translations

The general idea of CNNs is to intelligently adapt to properties of images:

Pixel position and neighborhood have semantic meanings.

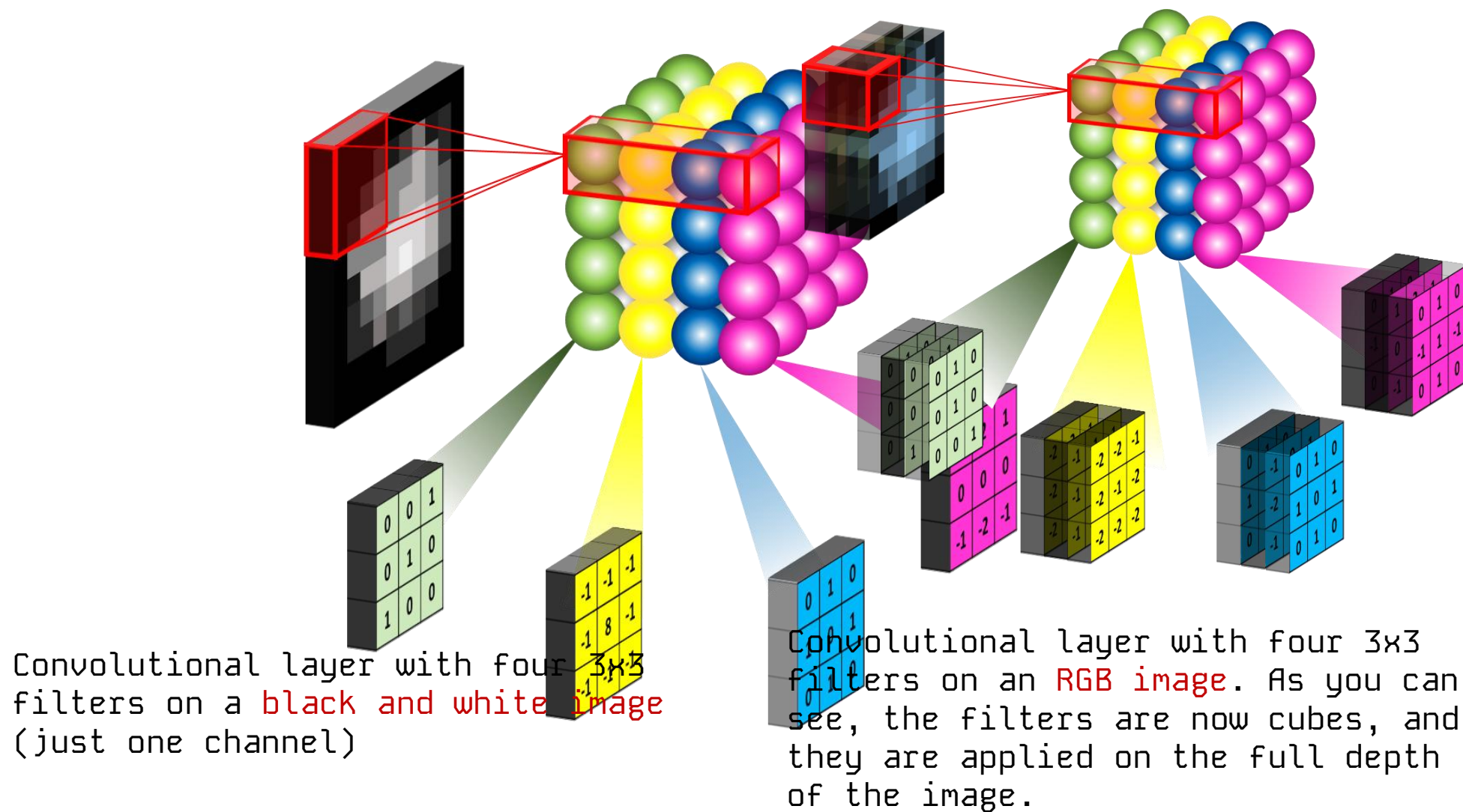
Elements of interest can appear anywhere in the image.



CNNs are also composed of layers, but those layers are not fully connected: they have **filters**, sets of cube-shaped weights that are applied throughout the image. Each 2D slice of the filters are called **kernels**.

These filters introduce **translation invariance** and **parameter sharing**.
How are they applied? Convolutions!

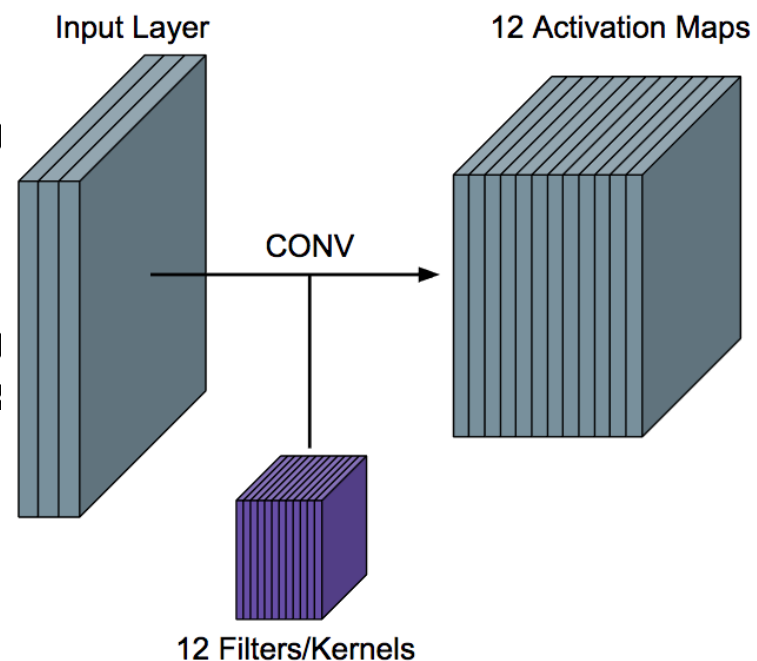
Convolutional Neural Networks (CNN)



To be clear: each filter is convolved with the entirety of the **3D input cube**, but generates a **2D feature map**.

Because we have multiple filters, we end up with a 3D **map per filter**.

The feature map dimension can **change drastically** from next: we can enter a layer with a $32 \times 32 \times 16$ input and end up with a $16 \times 16 \times 128$ output if that layer has 128 filters.





Input

A convolutional layer convolves each of its filters with the input.

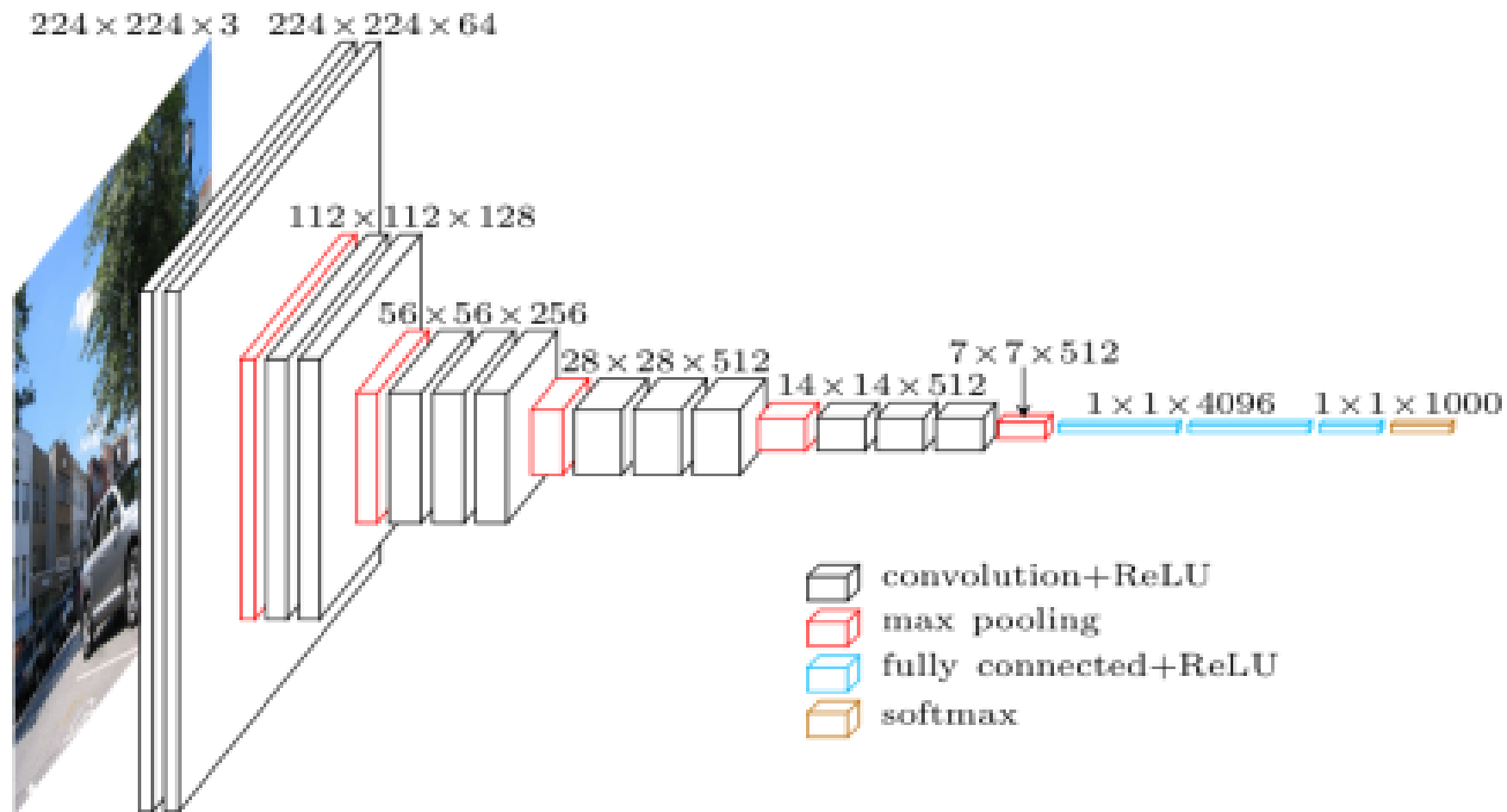
Input: a **3D tensor**, where the dimensions are Width, Height and Channels (or Feature Maps)

Output: a **3D tensor**, with dimensions Width, Height and Feature Maps (one for each filter)

Applies non-linear **activation function** (usually ReLU) over each value of the output.

Multiple **parameters to define**: number of filters, size of filters, stride, padding, activation function to use, regularization.

Fully Built CNN (VGG)

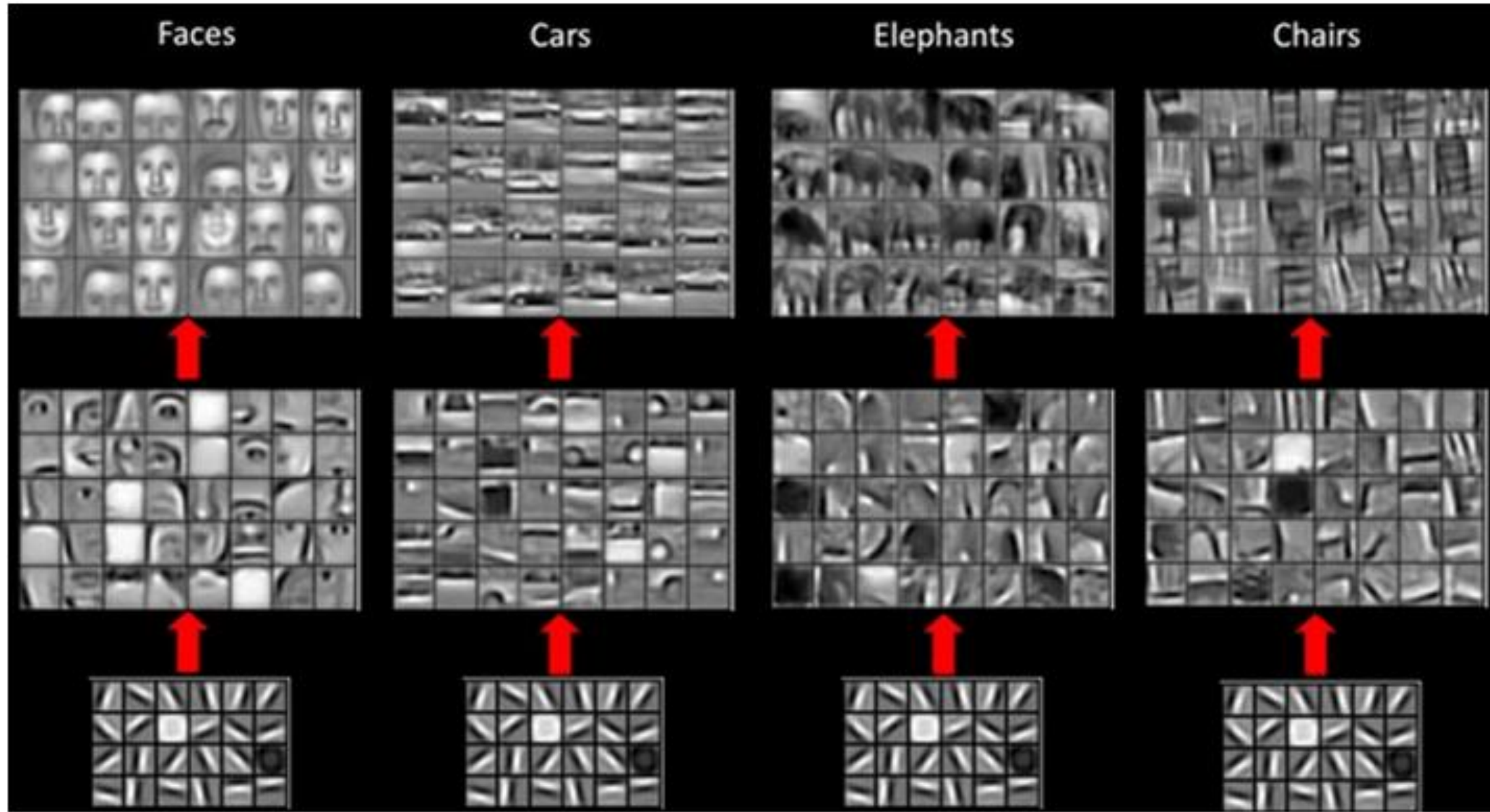


Each CNN layer learns filters of increasing complexity.

The first layers learn **basic feature detection filters**: edges, corners, etc.

The middle layers learn filters that detect **parts of objects**. For faces, they might learn to respond to eyes, noses, etc.

The last layers have higher representations: they learn to **recognize full objects**, in different shapes and positions.



UNSUPERVISED

SUPERVISED 7

SIMULATIONS

I have a convolutional layer with 12 4x4 filters that takes an RGB image as input.

What else can we define about this layer?

- Activation function
- Stride
- Padding type

How many parameters does the layer have?

$$12 \times 4 \times 4 \times 3 + 12 = 588$$

Number of filters Size of Filters Number of channels of prev layer Biases (one per filter)

Let C be a CNN with the following disposition:

- Input: 32x32x3 images
- Conv1: 8 3x3 filters, stride 1, padding=same
- Conv2: 16 5x5 filters, stride 2, padding=same
- Flatten layer
- Dense1: 512 nodes
- Dense2: 4 nodes

How many parameters does this network have?

$$(8 \times 3 \times 3 \times 3 + 8) + (16 \times 5 \times 5 \times 8 + 16) + (16 \times 16 \times 16 \times 512 + 512) + (512 \times 4 + 4)$$

Conv1

Conv2

Dense1

Dense2

Saliency Maps

If you are given an image of a dog and asked to classify it.

Most probably you will answer immediately – Dog! But your Deep Learning Network might not be as smart as you. It might classify it as a cat, a lion or rat!

What are the reasons for that?

- bias in training data
- no regularization
- or your network has seen too many rats



Saliency Maps

We want to understand what made my network give a certain class as output?

Saliency Maps, they are a way to measure the spatial support of a particular class in a given image.

“Find me pixels responsible for the class X having score $S(X)$ when the image I is passed through my network”.

Saliency Maps

Question: How do we do that?

We differentiate!

For any function $f(x, y, z)$, we can find the impact of variables x, y, z on f at any specific point (x_i, y_i, z_i) by finding its partial derivatives w.r.t these variables at that point.

Similarly, to find the responsible pixels, we take the score function S , for class x and take the partial derivatives w.r.t every pixel.

Saliency Maps

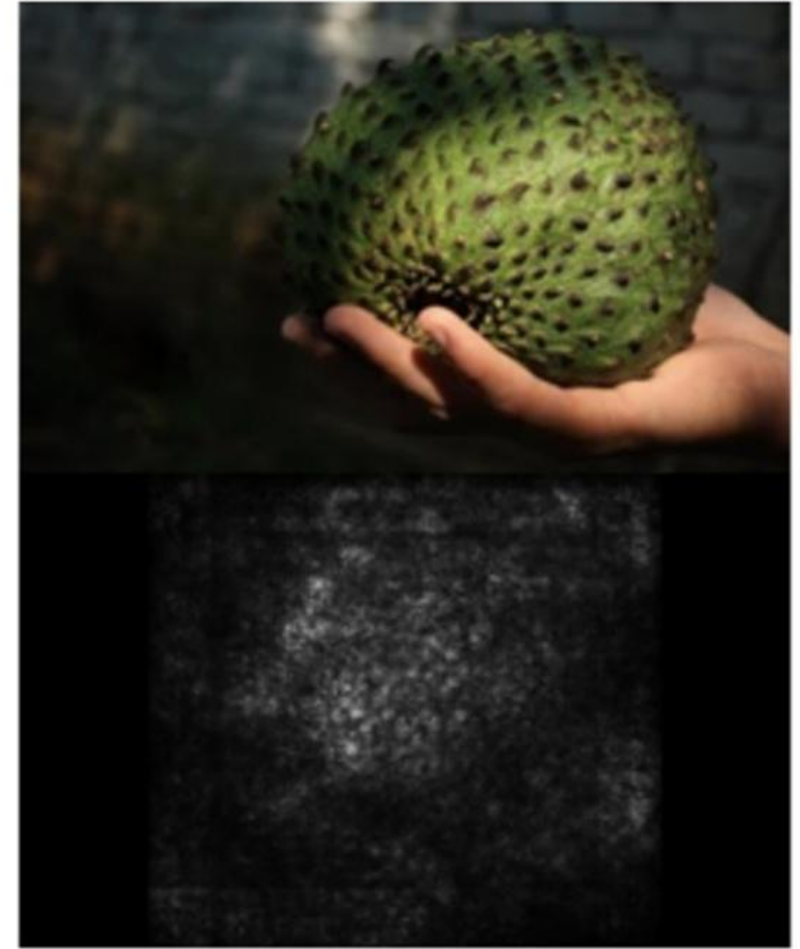
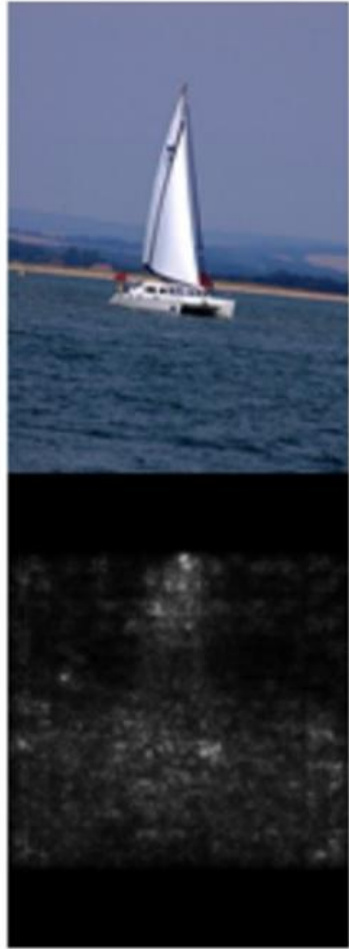
Question: Easy Peasy?

Sort of! Auto-grad can do this!

1. Forward pass of the image through the network.
2. Calculate the scores for every class.
3. Enforce derivative of score S at last layer for all classes except class K to be 0. For K , set it to 1
4. Backpropagate this derivative till the start
5. Render them and you have your Saliency Map!

Note: On step #2. Instead of doing softmax, we turn it to linear and use the logits.

Saliency Maps



UNSUPERVISED

SUPERVISED 7

SIMULATIONS

Saliency Maps – Color Images?

Take the saliency map for each channel and either take the max or average or use all 3 channels.

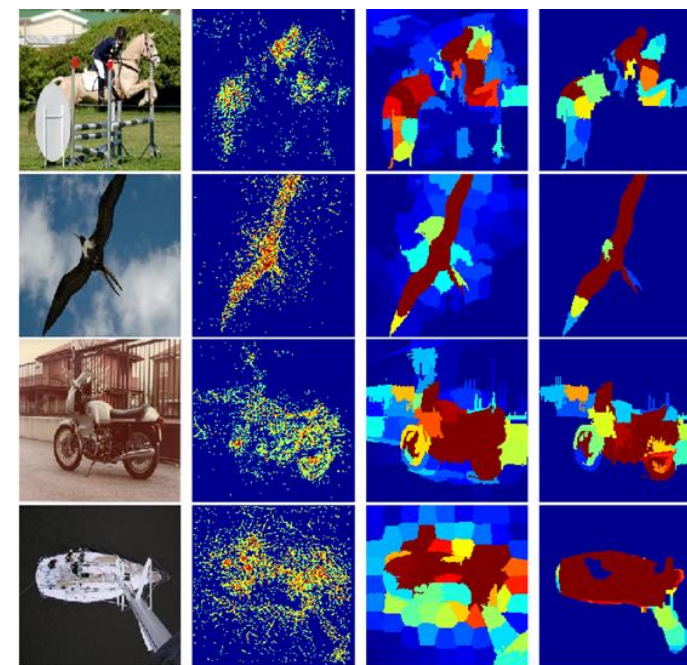
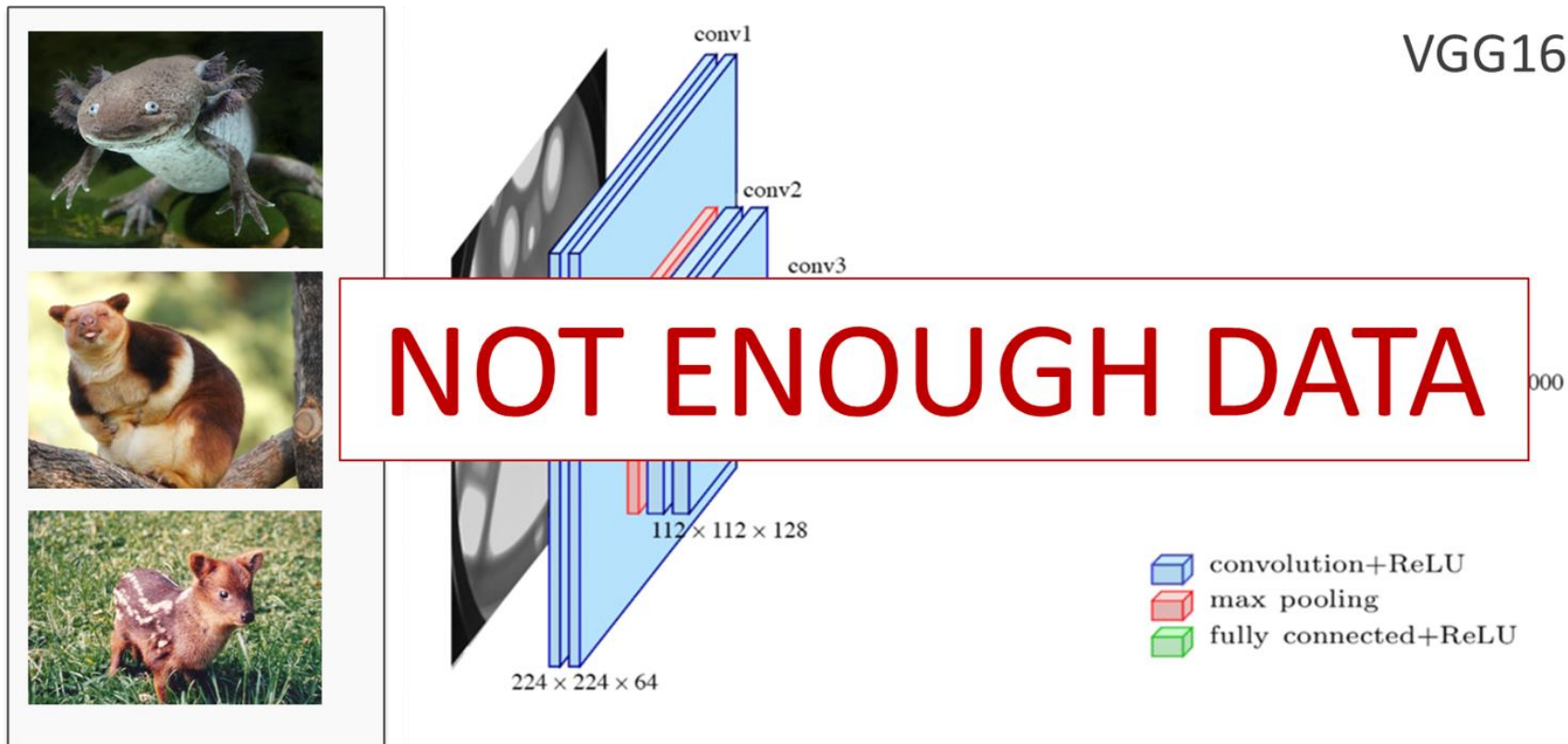


Figure 2. From left to right: original images, raw saliency maps,

<https://poloclub.github.io/cnn-explainer/>

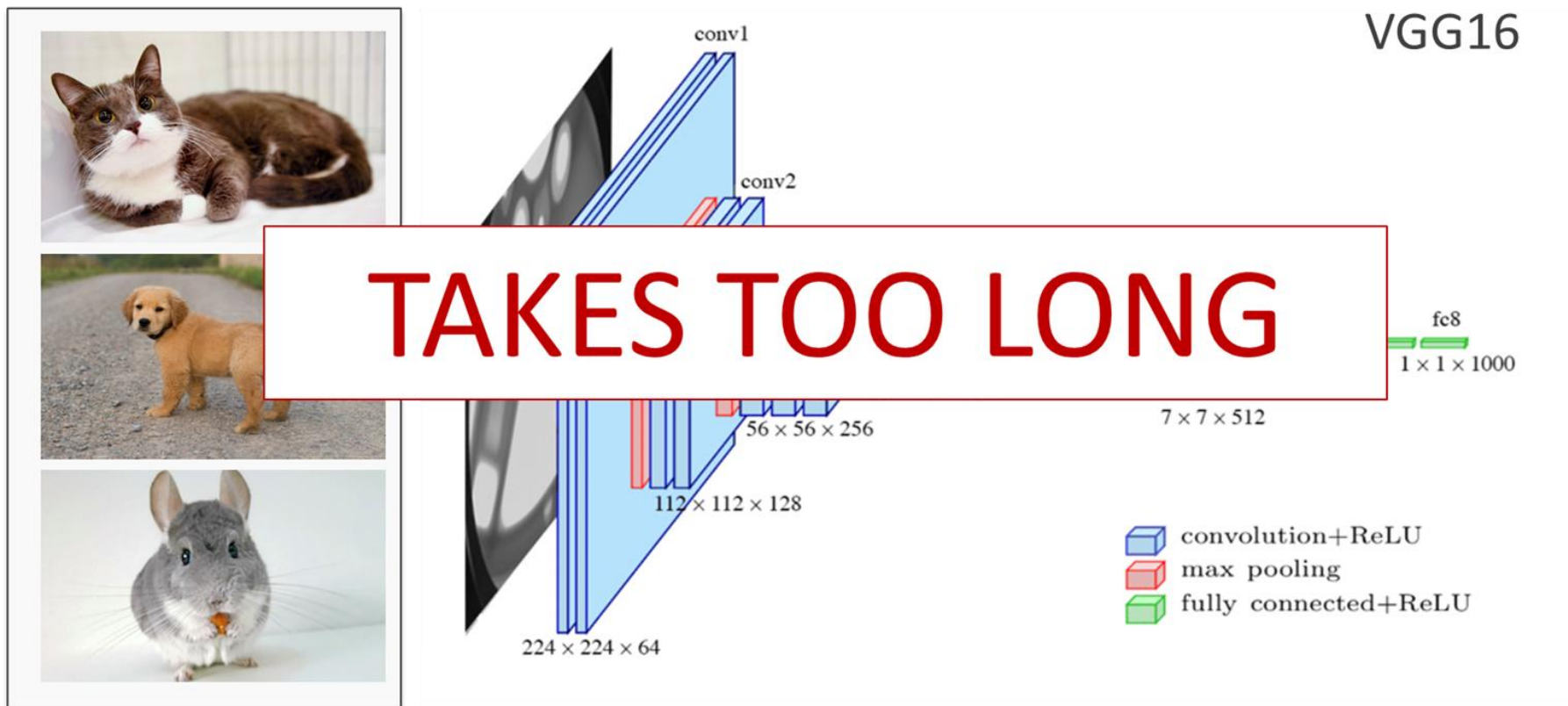
Transfer Learning



Number of parameters: 134,268,737

Data Set: Few hundred images

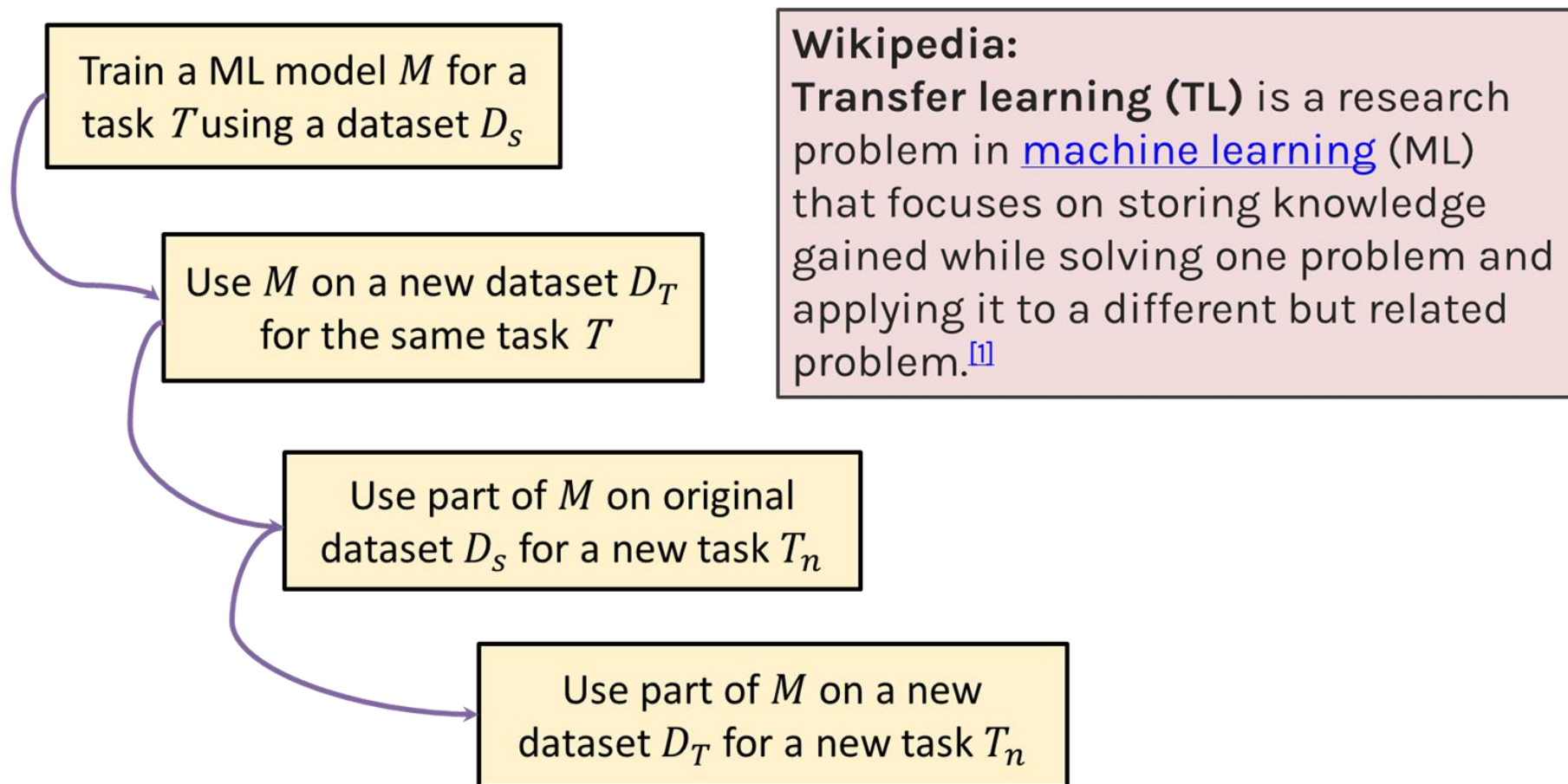
Transfer Learning



Number of parameters: 134,268,737

Enough training data. ImageNet approximate 1.2M

How do you build an image classifier that can be trained in a few minutes on a CPU with very little data?



How do you build an image classifier that can be trained in a few minutes on a CPU with very little data?

How do you make an image classifier that can be trained in a few minutes on a CPU with very little data?

Use pre-trained models, i.e., models with known weights.

Main Idea: earlier layers of a network learn low level features, which can be adapted to new domains by changing weights at later and fully-connected layers.

Example: use ImageNet trained with any sophisticated huge network. Then retrain it on a few images

train on a big "source" data set, with a big model, on one particular downstream tasks (say classification). Do it once and save the parameters. This is called a pre-trained model.

use these parameters for other smaller "target " datasets, say, for classification on new images (possibly different domain, or training distribution), or for image segmentation on old images (new task), or new images (new task and new domain).

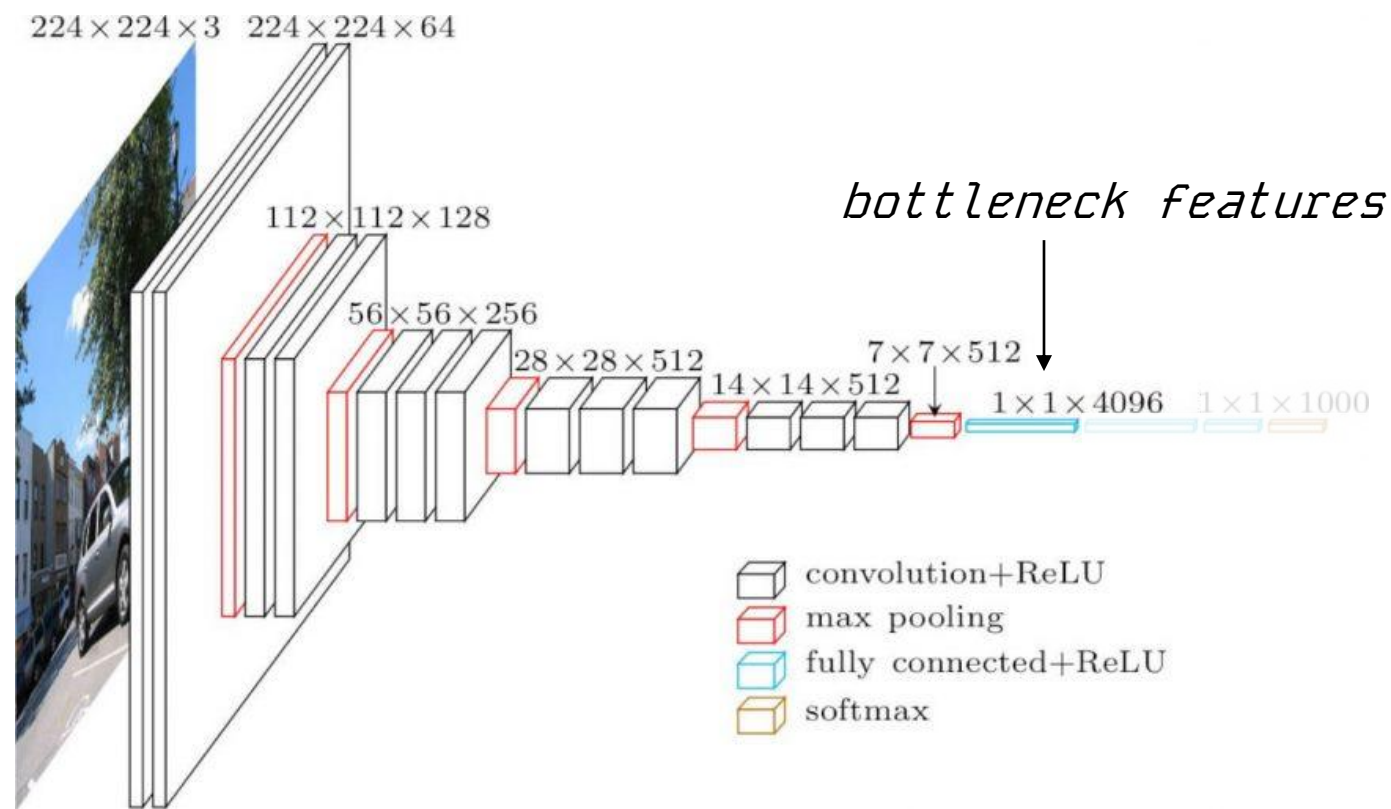
less helpful if you have a large target dataset with many labels.

will fail if source domain (where you trained big model) has nothing in common with target domain (that you want to train on smaller data set).

Idea 1

CNNs extract features from images

> use pretrained model as feature extractor and learn shallow model on top



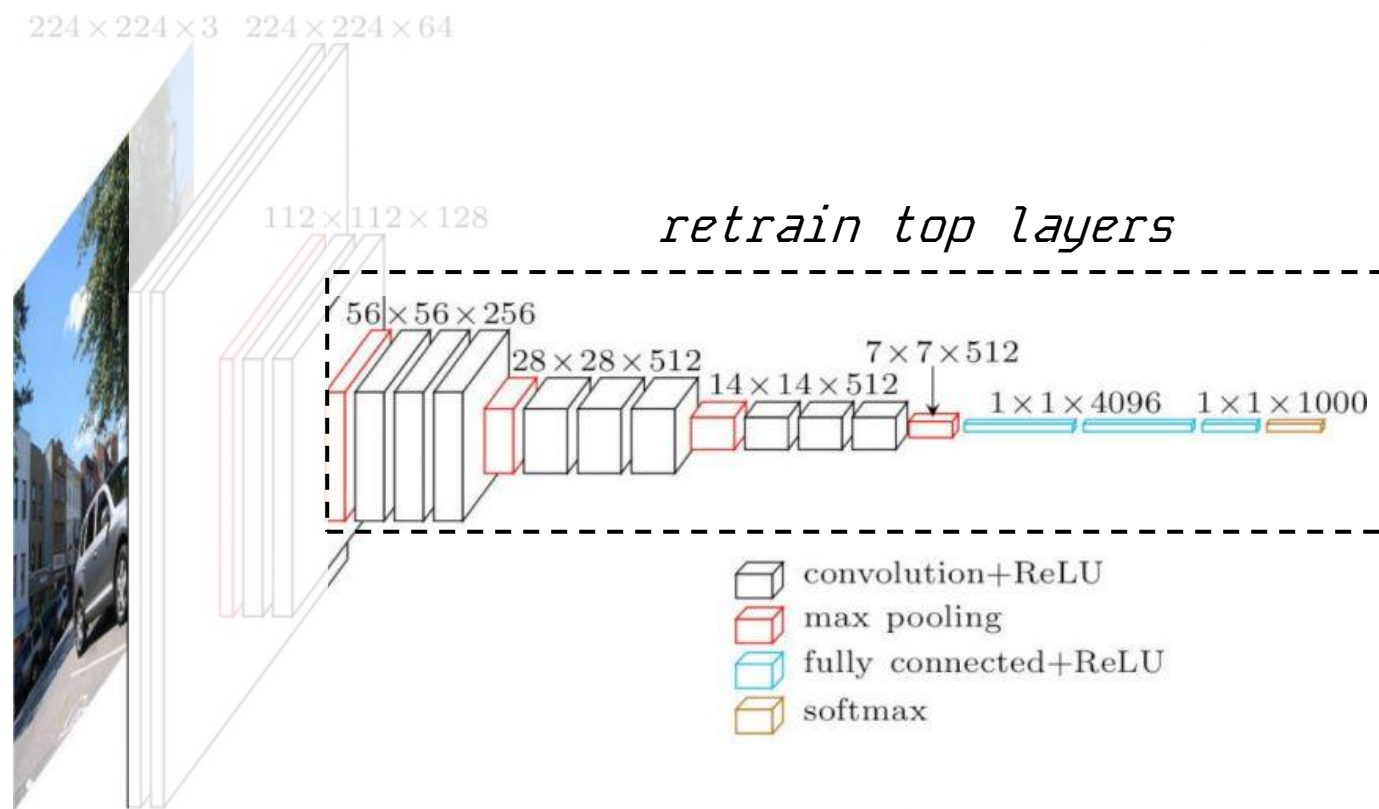
VGG16

Simonyan and Zisserman, 2014

Idea 2

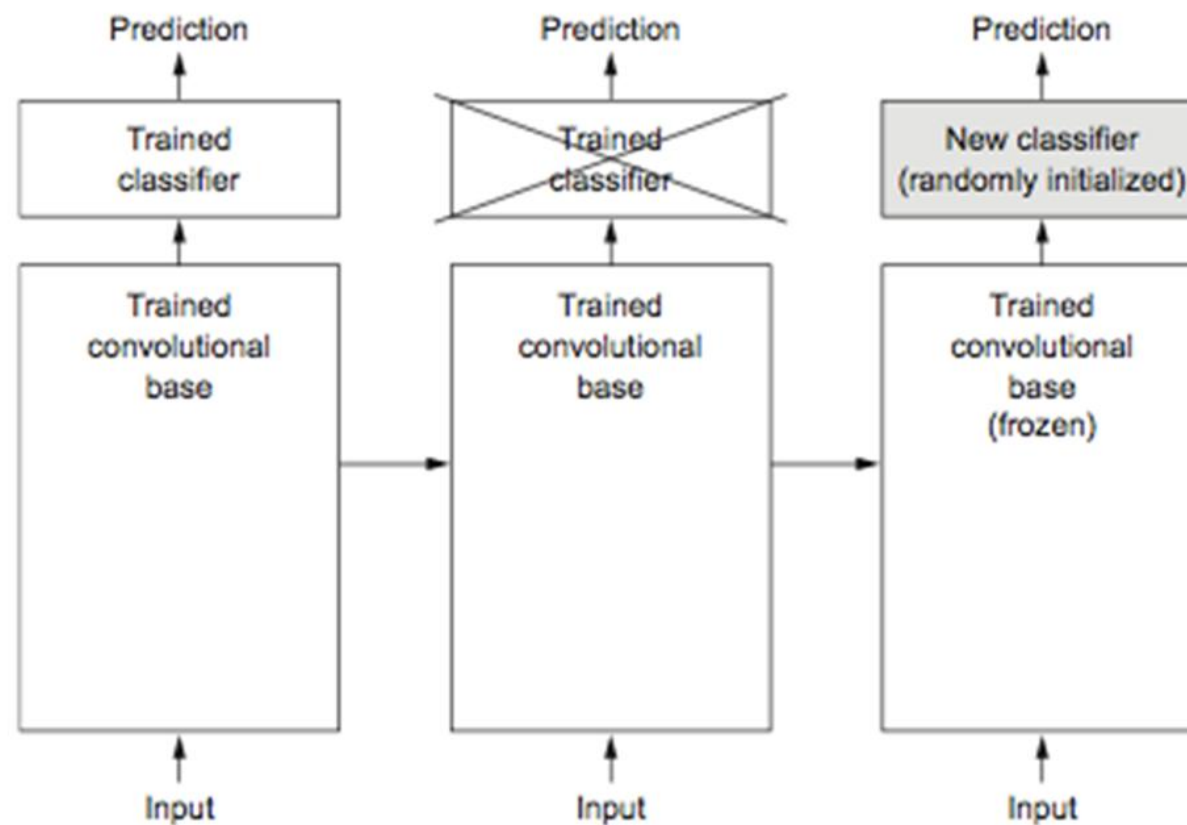
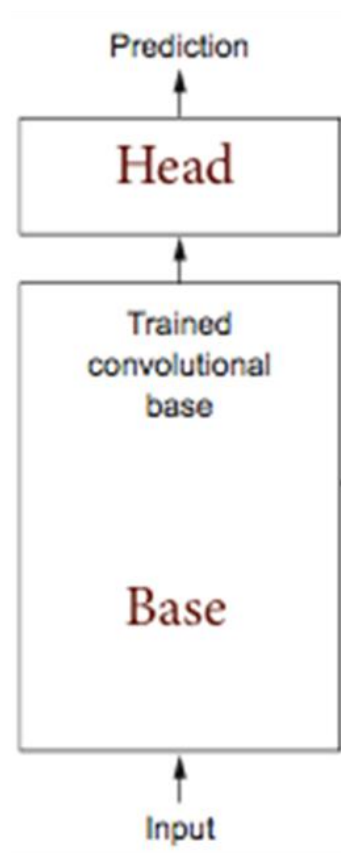
Finetune an existing model

> Re-optimize model (entirely or partially) starting from model weights.



VGG16

Simonyan and Zisserman, 2014

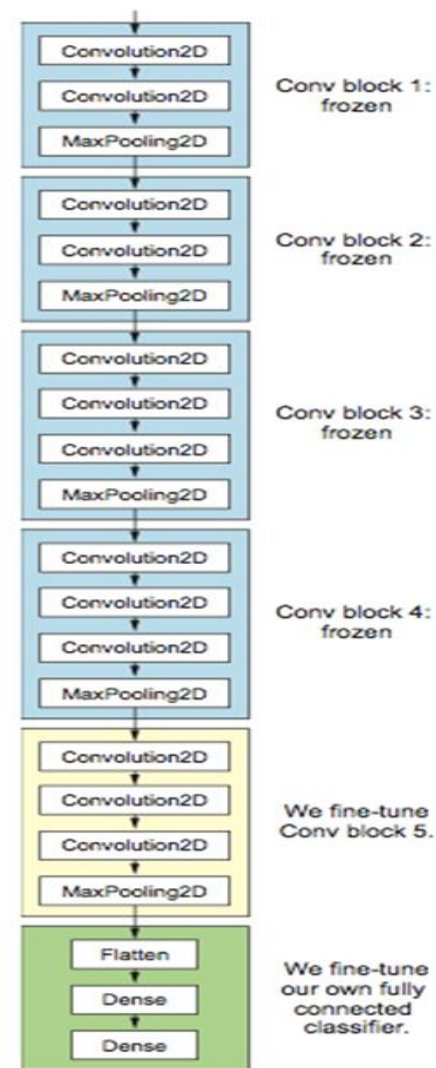


Up to now we have frozen the entire convolutional base.

Remember that earlier layers learn highly generic feature maps (edges, colors, textures).

Later layers learn abstract concepts (dog's ear).

To particularize the model to our task, it is often worth tuning the later layers as well.



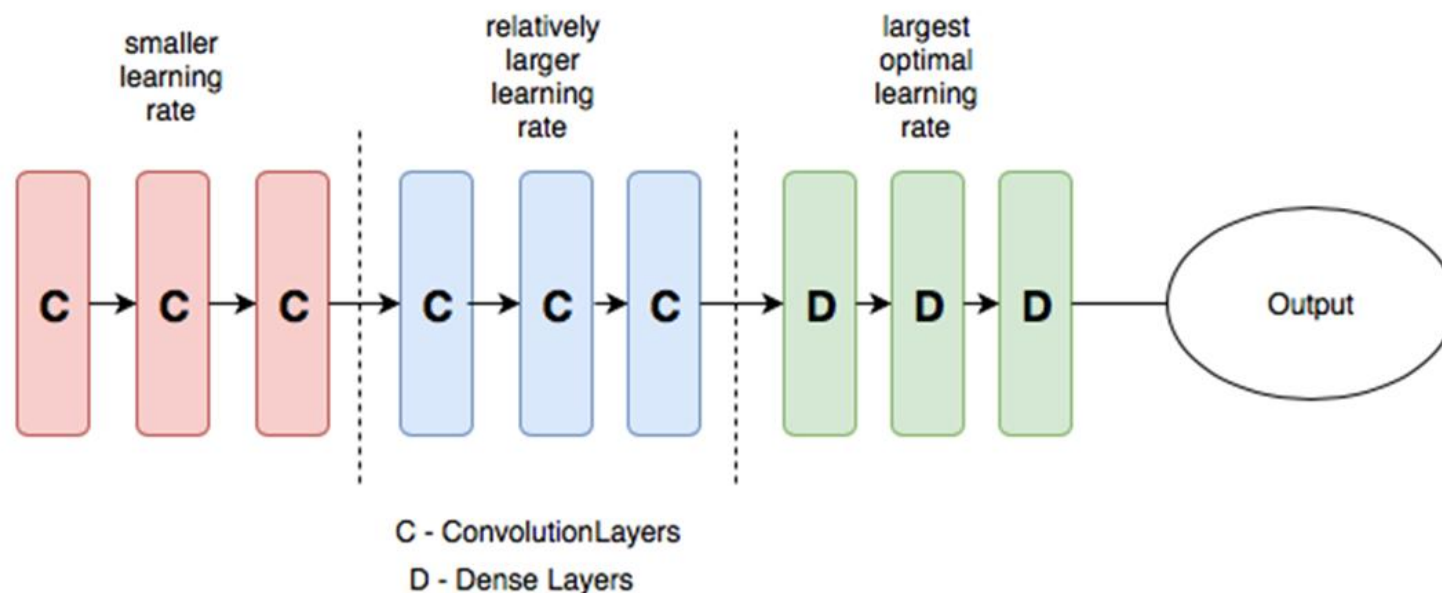
Transfer Learning

A low learning rate can take a lot of time to train on the "later" layers. Since we trained the FC head earlier, we could probably retrain them at a higher learning rate.

General Idea: Train different layers at different rates.

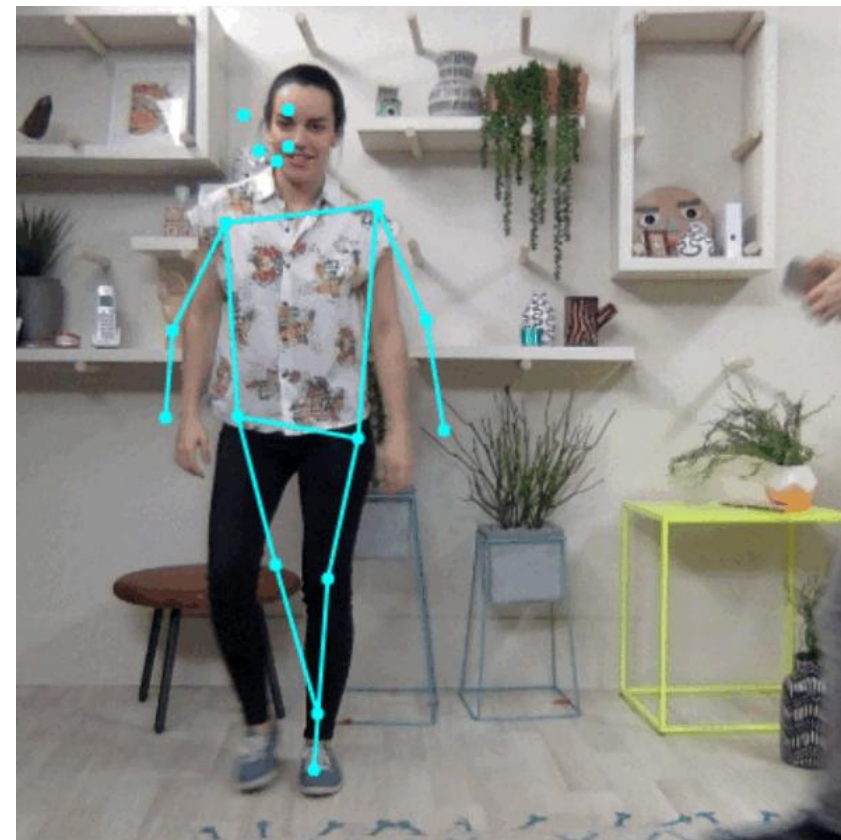
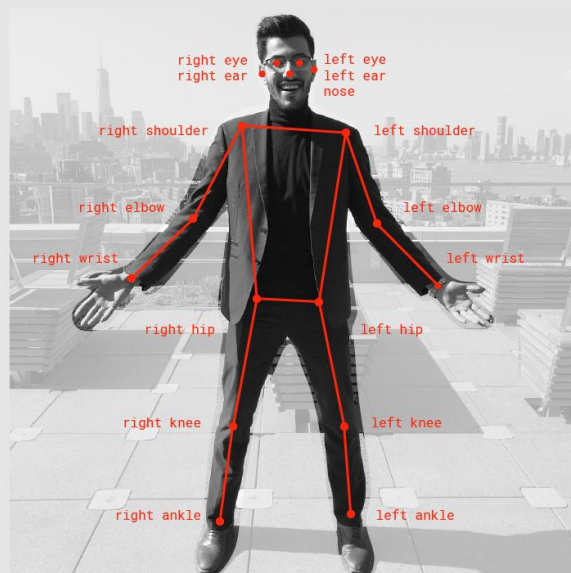
Each "earlier" layer or layer group (the color-coded layers in the image) can be trained at 3x-10x smaller learning rate than the next "later" one.

One could even train the entire network again this way until we overfit and then step back some epochs.



Built on top of ResNet with top layer changes and retrainin

17 Pose Keypoints
Returned by PoseNet



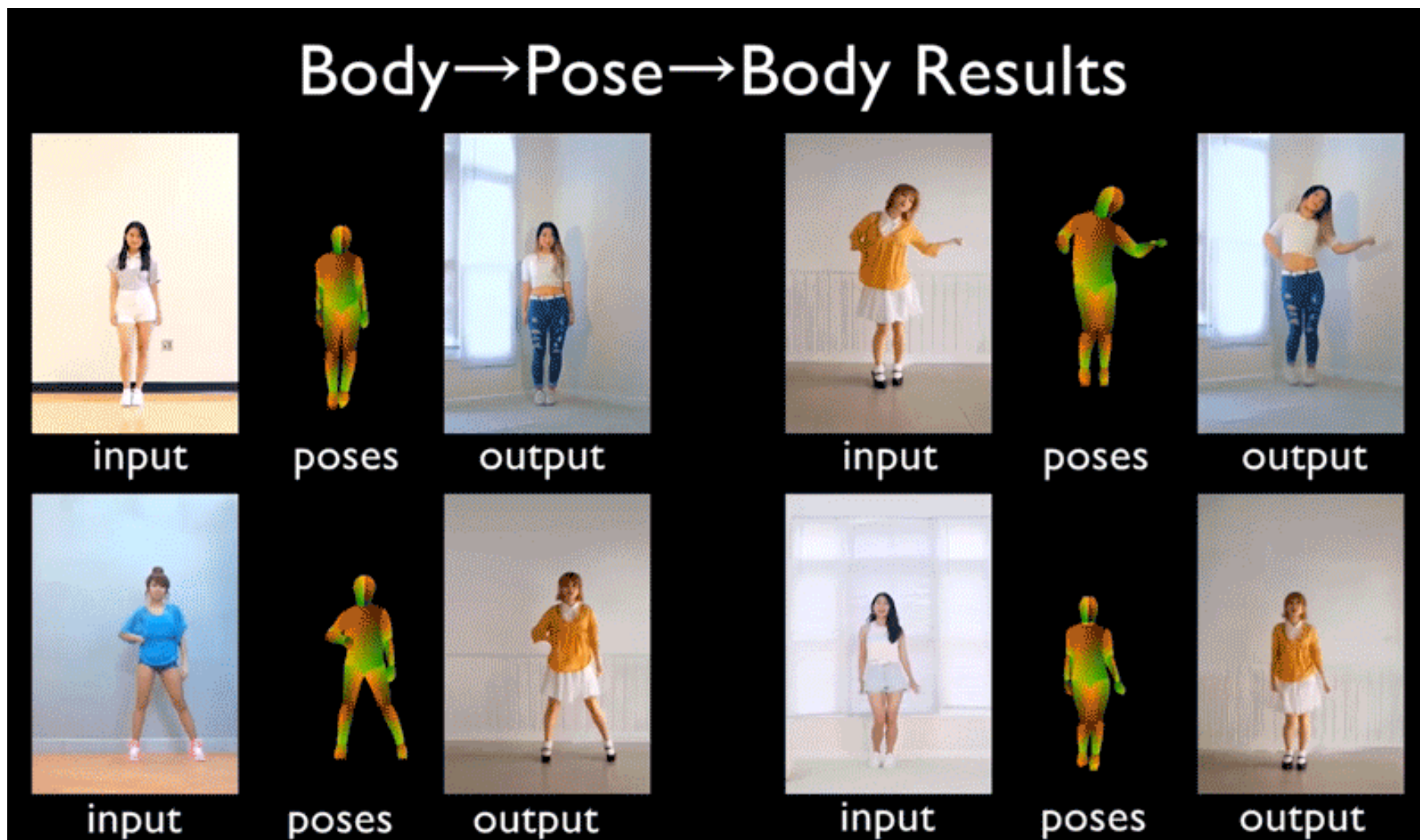
*PoseNet, Google
Papandreou et al., 2017*

UNSUPERVISED

SUPERVISED 7

SIMULATIONS

Transfer Learning – NVIDIA Video to Video Synthesis (2018)



UNSUPERVISED

SUPERVISED 7

SIMULATIONS

Google Colab
<https://bit.ly/BPS5231-L8>

<https://www.kaggle.com/models>

L08.1

Unsupervised

Clustering

KMeans

Hierarchical

Principal Component Analysis

t-SNE

L08.2

Supervised

Convolutional Neural Networks

Saliency Maps

Transfer Learning